ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO THÍ NGHIỆM

LAB 02

SYNTHESIS AND STANDARD CELL

Môn: Cơ Sở Thiết Kế Vi Mạch

Lớp: L02 – Nhóm 16

MSSV	Họ & Tên	Phân công
2310434	Lâm Chí Dĩnh	Bài Thí nghiệm 1
2313225	Trần Đình Thiên	Bài Thí nghiệm 2 & 4
2313014	Thái Tài	Bài Thí nghiệm 3

GVHD: Thầy Nguyễn Trung Hiếu

Thành phố Hồ Chí Minh, ngày 17 tháng 11 năm 2025

MỤC LỤC

MỤC LỤC 1
1. THÍ NGHIỆM 1 1
1.1. Chuẩn bị các file cần thiết để chạy trên server 1
1.1.1. HDL files 1
1.1.2. TCL files 2
1.1.3. Testbench files 3
1.2. Kết quả khi chạy tool Genus trên server 9
1.3. Chạy testbench và kết luận 14
2. THÍ NGHIỆM 2: 29
2.1. Chuẩn bị các files cần thiết để chạy trên server: 29
2.1.1. TCL file (synth.tcl) 29
2.1.2. Testbench file (alu_tb.sv) 30
2.2. Kết quả sau khi chạy tool Genus trên server 30
2.2.1. Kết quả synthesis (trước khi mapping) 30
2.2.2. Kết quả synthesis (sau khi mapping) 31
2.2.3. Đọc các file outputs 31
2.3. Kiểm thử và kết luận 36
2.3.1. Các trường hợp kiểm thử 36
2.3.2. Chạy trên Xcelium và phân tích dạng sóng trên Simvision 36
2.3.3. Tổng kết 50
3. THÍ NGHIỆM 3: 52
1.1. Chuẩn bị các files cần thiết để chạy trên server 52
1.1.1. HDL file 52
1.1.2. TCL files 52
1.1.3. Testbench file 53
1.2. Kết quả khi chạy tool Genus trên server 56

1.2.1. Kết quả Area và Power 56
1.2.2. Kết quả Delay 58
1.3. Chạy testbench và kết luận 58
1.3.1. Testcases 58
1.3.2. Chạy trên Xcelium và phân tích dạng sóng trên Simvision 60
4. THÍ NGHIỆM 4: 66
4.1. Chuẩn bị các files cần thiết để chạy trên server: 66
4.1.1. TCL file (synth.tcl) 66
4.1.2. Testbench file (counter_tb.sv) 67
4.2. Kết quả sau khi chạy tool Genus trên server 67
4.2.1. Kết quả synthesis 68
4.2.2. Đọc các file outputs 68
4.3. Kiểm thử và kết luận 71
4.3.1. Testcases 71
4.3.2. Chạy trên Xcelium và phân tích dạng sóng trên Simvision 71
4.3.3. Tổng kết 78

DANH MỤC HÌNH ẢNH

Hình 1.1: Mô phỏng cổng NOT tại corner ff_1v0_hvt 14

Hình 1.2: Mô phỏng cổng NOT tại corner ff_1v2_hvt 14

Hình 1.3: Mô phỏng cổng NOT tại corner sl_1v0_hvt 15

Hình 1.4: Mô phỏng cổng NOT tại corner sl_1v2_hvt 15

Hình 1.5: Mô phỏng cổng NOT tại corner ff_1v0_lvt 15

Hình 1.6: Mô phỏng cổng NOT tại corner ff_1v2_lvt 15

Hình 1.7: Mô phỏng cổng NOT tại corner sl_1v0_lvt 16

Hình 1.8: Mô phỏng cổng NOT tại corner sl_1v2_lvt 16

Hình 1.9 : Mô phỏng cổng NAND2 tại corner ff_1v0_hvt 16

Hình 1.10: Mô phỏng cổng NAND2 tại corner ff_1v2_hvt 17

Hình 1.11: Mô phỏng cổng NAND2 tại corner sl_1v0_hvt 17

Hình 1.12: Mô phỏng cổng NAND2 tại corner sl_1v2_hvt 18

Hình 1.13: Mô phỏng cổng NAND2 tại corner ff_1v0_lvt 18

Hình 1.14: Mô phỏng cổng NAND2 tại corner ff_1v2_lvt 18

Hình 1.15: Mô phỏng cổng NAND2 tại corner sl_1v0_lvt 19

Hình 1.16: Mô phỏng cổng NAND2 tại corner sl_1v2_lvt 19

Hình 1.17: Mô phỏng cổng OR2 tại corner ff_1v0_hvt 19

Hình 1.18: Mô phỏng cổng OR2 tại corner ff_1v2_hvt 20

Hình 1.19: Mô phỏng cổng OR2 tại corner sl_1v0_hvt 20

Hình 1.20: Mô phỏng cổng OR2 tại corner sl_1v2_hvt 20

Hình 1.21: Mô phỏng cổng OR2 tại corner ff_1v0_lvt 21

Hình 1.22: Mô phỏng cổng OR2 tại corner ff_1v2_lvt 21

Hình 1.23: Mô phỏng cổng OR2 tại corner sl_1v0_lvt 21

Hình 1.24: Mô phỏng cổng OR2 tại corner sl_1v2_lvt 21

Hình 1.25: Mô phỏng cổng NOR2 tại corner ff_1v0_hvt 21

Hình 1.26: Mô phỏng cổng NOR2 tại corner ff_1v2_hvt 22

Hình 1.27: Mô phỏng cổng NOR2 tại corner sl_1v0_hvt 22
Hình 1.28: Mô phỏng cổng NOR2 tại corner sl_1v2_hvt 23
Hình 1.29: Mô phỏng cổng NOR2 tại corner ff_1v0_lvt 23
Hình 1.30: Mô phỏng cổng NOR2 tại corner ff_1v2_lvt 23
Hình 1.31: Mô phỏng cổng NOR2 tại corner sl_1v0_lvt 23
Hình 1.32: Mô phỏng cổng NOR2 tại corner sl_1v2_lvt 23
Hình 1.33: Mô phỏng cổng MUX21 tại corner ff_1v0_hvt 23
Hình 1.34: Mô phỏng cổng MUX21 tại corner ff_1v2_hvt 24
Hình 1.35: Mô phỏng cổng MUX21 tại corner sl_1v0_hvt 24
Hình 1.36: Mô phỏng cổng MUX21 tại corner sl_1v2_hvt 25
Hình 1.37: Mô phỏng cổng MUX21 tại corner ff_1v0_lvt 26
Hình 1.38: Mô phỏng cổng MUX21 tại corner ff_1v2_lvt 26
Hình 1.39: Mô phỏng cổng MUX21 tại corner sl_1v0_lvt 26
Hình 1.40: Mô phỏng cổng MUX21 tại corner sl_1v2_lvt 26
Hình 1.41: Mô phỏng cổng FULL_ADDER tại corner ff_1v0_hvt 26
Hình 1.42: Mô phỏng cổng FULL_ADDER tại corner ff_1v2_hvt 27
Hình 1.43: Mô phỏng cổng FULL_ADDER tại corner sl_1v0_hvt 27
Hình 1.44: Mô phỏng cổng FULL_ADDER tại corner sl_1v2_hvt 28
Hình 1.45: Mô phỏng cổng FULL_ADDER tại corner ff_1v0_lvt 28
Hình 1.46: Mô phỏng cổng FULL_ADDER tại corner ff_1v2_lvt 28
Hình 1.47: Mô phỏng cổng FULL_ADDER tại corner sl_1v0_lvt 28
Hình 1.48: Mô phỏng cổng FULL_ADDER tại corner sl_1v2_lvt 28
Hình 2.1: Kết quả schematic sau khi synthesis (không mapping) 31
Hình 2.2: Kết quả schematic sau khi synthesis (có mapping) 31
Hình 2.3: dạng sóng khi không áp dụng delay vào testbench 37
Hình 2.4: Dạng sóng phóng to tại trường hợp 120 ns 37
Hình 2.5: Dạng sóng của toàn bộ bài kiểm thử với T = 20 ns 37

Hình 2.6: Trạng thái các ngõ vào/ra và các đường dây tại giai đoạn khởi tạo.....38

Hình 2.7: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 20 ns.....39

Hình 2.8: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 40 ns.....39

Hình 2.9: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 60 ns.....40

Hình 2.10: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 80 ns.....41

Hình 2.11: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 100 ns....41

Hình 2.12: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 120 ns....42

Hình 2.13: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 140 ns....42

Hình 2.14: Tổng quan dạng sóng khi đặt T = 100 ns 43

Hình 2.15: Dạng sóng tại giai đoạn khởi tạo. 43

Hình 2.16: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 100 ns....44

Hình 2.17: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 200 ns....44

Hình 2.18: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 300 ns....44

Hình 2.19: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 400 ns....45

Hình 2.20: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 500 ns....45

Hình 2.21: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 600 ns....46

Hình 2.22: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 700 ns....46

Hình 2.23: Tổng quan dạng sóng khi đặt T = 100 ns 46

Hình 2.24: Dạng sóng tại giai đoạn khởi tạo. 47

Hình 2.25: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 200 ns....47

Hình 2.26: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 400 ns....48

Hình 2.27: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 600 ns....48

Hình 2.28: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 800 ns....48

Hình 2.29: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 1000 ns..49

Hình 2.30: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 1200 ns..49

Hình 2.31: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 1400 ns..49

Hình 3.1: Case 1 Normal operation 60

Hình 3.2: Case 2 Setup Violation 60

Hình 3.3: Case 4 Boundary condition 61

Hình 3.4: Case 1 Normal operation 61

Hình 3.5: Case 2 Set up violation 61

Hình 3.6: Case 3 Hold violation 62

Hình 3.7: Case 4 Boundary condition 62

Hình 3.8: Case 1 Normal operation 63

Hình 3.9: Case 2 Set up violation 63

Hình 3.10: Case 4 Boundary condition 63

Hình 3.11: slow_vdd1v2_basicCells_hvt.lib 63

Hình 3.12: fast_vdd1v0_basicCells_lvt.lib 64

Hình 3.13: fast_vdd1v2_basicCells_lvt.lib 64

Hình 3.14: slow_vdd1v0_basicCells_lvt.lib 64

Hình 3.15: slow_vdd1v2_basicCells_lvt.lib 65

Hình 4.1: Kết quả synthesis bộ Counter 68

Hình 4.2: Dạng sóng lý tưởng (không có delay) 71

Hình 4.3: Dạng sóng phóng to tại thời điểm 100 ns 72

Hình 4.4: Dạng sóng của toàn bộ bài kiểm thử với T = 50 ns 72

Hình 4.5: Trạng thái các ngõ vào/ra và các đường dây tại giai đoạn khởi tạo 72

Hình 4.6: Trạng thái các biến tại đầu chu kỳ 2, t = 100 ns 73

Hình 4.7: Trạng thái các biến tại đầu chu kỳ 3, t = 150 ns 73

Hình 4.8: Trạng thái các biến tại đầu chu kỳ 4, t = 200 ns 74

Hình 4.9: Trạng thái các biến tại đầu chu kỳ 5, t = 250 ns 74

Hình 4.10: Dạng sóng của toàn bộ bài kiểm thử với T = 100 ns 75

Hình 4.11: Trạng thái các biến tại đầu chu kỳ 1, t = 100 ns 75

Hình 4.12: Trạng thái các biến tại đầu chu kỳ 2, t = 200 ns 75

Hình 4.13: Trạng thái các biến tại đầu chu kỳ 3, t = 300 ns 76

Hình 4.14: Trạng thái các biến tại đầu chu kỳ 4, t = 400 ns 76

Hình 4.15: Trạng thái các biến tại đầu chu kỳ 5, t = 500 ns 76

Hình 4.16: Dạng sóng của toàn bộ bài kiểm thử với T = 100 ns..... 77

Hình 4.17: Trạng thái các biến tại đầu chu kỳ 1, t = 200 ns 77

Hình 4.18: Trạng thái các biến tại đầu chu kỳ 2, t = 400 ns 77

Hình 4.19: Trạng thái các biến tại đầu chu kỳ 3, t = 600 ns 78

Hình 4.20: Trạng thái các biến tại đầu chu kỳ 4, t = 800 ns 78

Hình 4.21: Trạng thái các biến tại đầu chu kỳ 5, t = 1000 ns 78

DANH MỤC BẢNG

Bảng 2.1: Các trường hợp kiểm thử khối ALU 36

Bảng 2.2: Tổng hợp các kết quả kiểm thử..... 50

Bảng 3.1: Kết quả Area và Power của DFF ở từng corner..... 56

Bảng 3.2: Ảnh hưởng của VDD lên Power từng corner 57

Bảng 3.3: Ảnh hưởng của Vt lên Power từng corner 57

Bảng 3.4: Ảnh hưởng của Process lên Power từng corner 57

Bảng 3.5: Kết quả Delay của DFF ở từng corner 58

Bảng 4.1: Các trường hợp kiểm thử của khối Counter 71

Bảng 4.2: Tổng hợp các kết quả kiểm thử của khối Counter..... 79

1. THÍ NGHIỆM 1

Ở thí nghiệm này chúng ta sẽ synthesis các standard cell NOT, OR2, NAND2, NOR2, MUX, FULL ADDER để khảo sát các thông số "Area", "Power", "Delay".

1.1. Chuẩn bị các file cần thiết để chạy trên server

1.1.1. HDL files

- Standard cell NOT

```
module NOT (input  logic A, output logic Y);
    assign Y = ~A;
endmodule
```

- Standard cell OR2

```
module OR2 (input  logic A, input  logic B, output logic Y);
    assign Y = A | B;
endmodule
```

- Standard cell NOR2

```
module NOR2 (input  logic A, input  logic B, output logic Y);
    assign Y = ~(A | B);
endmodule
```

- Standard cell NAND2

```
module NAND2 (input  logic A, input  logic B, output logic Y);
    assign Y = ~(A & B);
endmodule
```

- Standard MUX21

```
module MUX21 (input  logic A, input  logic B, input  logic S, output logic Y);
    assign Y = (S) ? B : A;
endmodule
```

- Standard cell FULL ADDER

```
module full_adder (input  logic A, input  logic B, input  logic Cin, output logic
S, output logic Cout);
    assign {Cout, S} = A + B + Cin;
endmodule
```

1.1.2. TCL files

Với mục tiêu bài thí nghiệm là khảo sát hết 8 library corner cho từng standard cell, nên nhóm sẽ chuẩn bị 8 file TCL tương tự nhau, chỉ thay đổi tên thư viện, đường dẫn thư mục chứa các file reports và outputs tương ứng mỗi library.

Dưới đây là file TCL của *fast_vdd1v0_basicCells_hvt.lib* của standard cell NOT, các thư viện còn lại cho các standard cell còn lại làm tương tự.

```
set_db init_lib_search_path {/home/yellow/ee3201_16/LAB2/gpdk045_lib}
set_db init_hdl_search_path {/home/yellow/ee3201_16/LAB2/Exp1/00_src}

read_libs fast_vdd1v0_basicCells_hvt.lib
read_hdl  -sv not.sv
elaborate

set_db syn_generic_effort medium
set_db syn_map_effort     medium

syn_generic
syn_map

report_area   > ./../02_reports/not/ff_1v0_hvt/area.rpt
report_power  > ./../02_reports/not/ff_1v0_hvt/power.rpt
report_timing > ./../02_reports/not/ff_1v0_hvt/timing.rpt
report_qor    > ./../02_reports/not/ff_1v0_hvt/qor.rpt

write_hdl > ./../03_outputs/not/ff_1v0_hvt/not_netlist.sv
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -
setuphold split > ./../03_outputs/not/ff_1v0_hvt/delay.sdf
```

Vì cấu trúc thư mục project của nhóm, lệnh chạy file từng file TCL trên Genus sẽ như sau:

```
genus -f 04_scripts/genus/<cell_lib_corner>.tcl
```

Trong đó:

- <cell_lib_corner.tcl>: Tên file TCL của từng cell trong từngcorner

1.1.3. Testbench files

a. NOT

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
    `include "03_outputs/not/sl_1v2_lvt/not_netlist.sv"
    `include "../gpdk045_verilog/slow_vdd1v2_basicCells_lvt.v"
`else
    `include "00_src/not.sv"
`endif

module tb_not;
    logic A;
    logic Y;

    NOT DUT (
        .A (A),
        .Y (Y)
    );

    `ifdef ANNOTATION
    initial begin
        $sdf_annotate("03_outputs/not/sl_1v2_lvt/delay.sdf", DUT, ,
"annotate_not_sl_1v2_lvt.log", "MAXIMUM");
    end
    `endif

    initial begin
        $dumpfile("not_sl_1v2_lvt.vcd");
        $dumpvars(0, tb_not.DUT);
    end

    initial begin
        A = 0;
        #0.1 A = 1;
        #0.1 A = 0;
        #0.1 A = 1;
        #0.1 A = 0;
        #0.1 $finish;
    end
endmodule
```

b. NOR2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
    `include "03_outputs/nor2/ff_1v2_lvt/nor2_netlist.sv"
    `include "../gpdk045_verilog/fast_vdd1v2_basicCells_lvt.v"
`else
    `include "00_src/nor2.sv"
`endif

module tb_nor2;
    logic A, B;
    logic Y;
    NOR2 DUT ( .A(A), .B(B), .Y(Y) );

    `ifdef ANNOTATION
    initial begin
        $sdf_annotate("03_outputs/nor2/ff_1v2_lvt/delay.sdf", DUT, ,
"annotate_nor2.log", "MAXIMUM");
    end
    `endif

    initial begin
        $dumpfile("nor2_ff_1v2_lvt.vcd");
        $dumpvars(0, tb_nor2.DUT);
    end

    initial begin
        // 1. Do Delay A (Giu B=0)
        A = 0; B = 0;
        #1;
        A = 1;
        #1;
        A = 0;
        #1;
        // 2. Do Delay B (Giu A=0)
        A = 0; B = 0;
        #1;
        B = 1;
        #1;
        B = 0;
        #1;
        $finish;
    end
endmodule
```

c. NAND2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
    `include "03_outputs/nand2/sl_1v0_hvt/nand2_netlist.sv"
    `include "../gpdk045_verilog/sl_vdd1v0_basicCells_hvt.v"
`else
    `include "00_src/nand2.sv"
`endif

module tb_nand2;
    logic A, B; // 2 Input
    logic Y;    // Output
    NAND2 DUT (
        .A (A),
        .B (B),
        .Y (Y)
    );

    `ifdef ANNOTATION
    initial begin
        // 3. Load file Delay SDF
        $sdf_annotate(
            "03_outputs/nand2/sl_1v0_hvt/delay.sdf",
            DUT,
            ,
            "annotate_nand2.log",
            "MAXIMUM"
        );
    end
    `endif

    initial begin
        $dumpfile("nand2_sl_1v0_hvt.vcd");
        $dumpvars(0, tb_nand2.DUT);
    end

    initial begin

        A = 0; B = 0;
        #0.2;
        A = 1; B = 0;
        #0.2;
        A = 1; B = 1;
        #0.2;
        A = 1; B = 0;
        #0.2;
        A = 0; B = 1;
        #0.2;
```

```
        A = 1; B = 1;
        #0.2;
        A = 0; B = 1;
        #0.2;
        $finish;
    end
endmodule
```

d. OR2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
    `include "03_outputs/or2/sl_1v0_hvt/or2_netlist.sv"
    `include "../gpdk045_verilog/slow_vdd1v0_basicCells_hvt.v"
`else
    `include "00_src/or2.sv"
`endif

module tb_or2;
    logic A, B; // 2 Input
    logic Y;    // Output
    OR2 DUT (
        .A (A),
        .B (B),
        .Y (Y)
    );

    `ifdef ANNOTATION
    initial begin
        $sdf_annotate(
            "03_outputs/or2/sl_1v0_hvt/delay.sdf", DUT, , "annotate_or2.log",
"MAXIMUM"
        );
    end
    `endif

    initial begin
        $dumpfile("or2_sl_1v0_hvt.vcd");
        $dumpvars(0, tb_or2.DUT);
    end

    initial begin
        A = 0; B = 0;
        #0.2;
        A = 1; B = 0;
        #0.2;
        A = 0; B = 0;
        #0.2;
```

```
        A = 0; B = 1;
        #0.2;
        A = 0; B = 0;
        #0.2;
        $finish;
    end
endmodule
```

e. MUX

```
`timescale 1ns/1fs
`ifdef GATE_LEVEL
    `include "03_outputs/mux21/sl_1v2_hvt/mux21_netlist.sv"
    `include "../gpdk045_verilog/slow_vdd1v2_basicCells_hvt.v"
`else
    `include "00_src/mux21.sv"
`endif

module tb_mux21;
    logic A, B, S;
    logic Y;
    MUX21 DUT ( .A(A), .B(B), .S(S), .Y(Y) );

    `ifdef ANNOTATION
    initial begin
        $sdf_annotate("03_outputs/mux21/sl_1v2_hvt/delay.sdf", DUT, ,
"annotate_mux21_sl_1v2_hvt.log", "MAXIMUM");
    end
    `endif

    initial begin
        $dumpfile("mux21_sl_1v2_hvt.vcd");
        $dumpvars(0, tb_mux21.DUT);
    end

    initial begin
        // 1. Do Delay A (Giu S=0)
        S = 0; B = 0; A = 0;
        #1;
        A = 1;
        #1;
        A = 0;
        #1;
        // 2. Do Delay B (Giu S=1)
        S = 1; A = 0; B = 0;
        #1;
        B = 1;
        #1;
```

```
        B = 0;
        #1;
        // 3. Do Delay S (Chuyen tu A=0 sang B=1)
        A = 0; B = 1; S = 0;
        #1;
        S = 1;
        #1;
        S = 0;
        #1;
        $finish;
    end
endmodule
```

f. FULL ADDER

```
`timescale 1ns/1fs
`ifdef GATE_LEVEL
    `include "03_outputs/full_adder/ff_1v0_hvt/full_adder_netlist.sv"
    `include "../gpdk045_verilog/fast_vdd1v0_basicCells_hvt.v"
`else
    `include "00_src/full_adder.sv"
`endif

module tb_full_adder;
    logic A, B, Cin;
    logic Sum, Cout;
    full_adder DUT (
        .A(A),
        .B(B),
        .Cin(Cin),
        .S(Sum),
        .Cout(Cout)
    );

    `ifdef ANNOTATION
    initial begin
        $sdf_annotate("03_outputs/full_adder/ff_1v0_hvt/delay.sdf", DUT, ,
"annotate_full_adder_ff_1v0_hvt.log", "MAXIMUM");
    end
    `endif

    initial begin
        $dumpfile("full_adder_ff_1v0_hvt.vcd");
        $dumpvars(0, tb_full_adder.DUT);
    end

    initial begin
        A = 1; B = 0; Cin = 0;
```

```
        #1;
        Cin = 1; // Cin len 1 -> Cout len 1 (Delay Rise)
        #1;
        Cin = 0; // Cin xuong 0 -> Cout xuong 0 (Delay Fall)
        #1;
        A = 0; B = 0; Cin = 0;
        #1;
        A = 1; // A len 1 -> S len 1 (Delay Rise)
        #1;
        A = 0; // A xuong 0 -> S xuong 0 (Delay Fall)
        #1;
        $finish;
    end
endmodule
```

1.2. Kết quả khi chạy tool Genus trên server

Dưới đây là bảng kết quả các giá trị Area, Power và Delay của 6 standard cell trong 8 library corner khác nhau

- **Kết quả của NOT**

Cell	Corner	Path	t_rise(ns)	t_fall(ns)	Area(um^2)	Power(nW)
NOT	slow_vdd1v2_basicCells_lvt	A -> Y	0.004	0.005	0,684	6.20997
	slow_vdd1v2_basicCells_hvt	A -> Y	0.006	0.008		5.82108
	slow_vdd1v0_basicCells_lvt	A -> Y	0.006	0.007		4.28894
	slow_vdd1v0_basicCells_hvt	A -> Y	0.009	0.015		3.99470
	fast_vdd1v2_basicCells_lvt	A -> Y	0.003	0.003		10.7250
	fast_vdd1v2_basicCells_hvt	A -> Y	0.003	0.003		8.61296
	fast_vdd1v0_basicCells_lvt	A -> Y	0.003	0.003		7.80289
	fast_vdd1v0_basicCells_hvt	A -> Y	0.004	0.004		5.92740

- **Kết quả của OR2**

Cell	Corner	Path	t_rise(ns)	t_fall(ns)	Area(um^2)	Power(nF)
OR2	slow_vdd1v2_basicCells_lvt	A -> Y	0.022	0.034	1.368	18.4242
		B -> Y	0.018	0.030		
	slow_vdd1v2_basicCells_hvt	A -> Y	0.037	0.050		16.8838
		B -> Y	0.031	0.044		
	slow_vdd1v0_basicCells_lvt	A -> Y	0.033	0.052		12.64
		B -> Y	0.027	0.045		
	slow_vdd1v0_basicCells_hvt	A -> Y	0.077	0.099		11.4746
		B -> Y	0.065	0.088		
	fast_vdd1v2_basicCells_lvt	A -> Y	0.009	0.014		33.4466
		B -> Y	0.008	0.012		
	fast_vdd1v2_basicCells_hvt	A -> Y	0.013	0.021		25.3194
		B -> Y	0.011	0.019		
	fast_vdd1v0_basicCells_lvt	A -> Y	0.011	0.017		22.3841
		B -> Y	0.009	0.015		
	fast_vdd1v0_basicCells_hvt	A -> Y	0.017	0.029		17.3733
		B -> Y	0.015	0.026		

- **Kết quả của NOR2**

Cell	Corner	Path	t_rise(ns)	t_fall(ns)	Area(um^2)	Power(nF)
	slow_vdd1v2_basicCells_lvt	A -> Y	0.013	0.01		11.3492

NOR2		B -> Y	0.008	0.006	1.026	
	slow_vdd1v2_basicCells_hvt	A -> Y	0.019	0.016		10.6519
		B -> Y	0.013	0.01		
	slow_vdd1v0_basicCells_lvt	A -> Y	0.02	0.014		7.823
		B -> Y	0.013	0.008		
	slow_vdd1v0_basicCells_hvt	A -> Y	0.036	0.03		7.3089
		B -> Y	0.025	0.02		
	fast_vdd1v2_basicCells_lvt	A -> Y	0.006	0.004		20.336
		B -> Y	0.004	0.003		
	fast_vdd1v2_basicCells_hvt	A -> Y	0.009	0.006		15.663
		B -> Y	0.006	0.004		
	fast_vdd1v0_basicCells_lvt	A -> Y	0.007	0.005		14.402
		B -> Y	0.005	0.003		
	fast_vdd1v0_basicCells_hvt	A -> Y	0.011	0.007		10.754
		B -> Y	0.008	0.005		

- **Kết quả của NAND**

Cell	Corner	Path	t_rise(ns)	t_fall(ns)	Area(um^2)	Power(nF)
NAND2	slow_vdd1v2_basicCells_lvt	A -> Y	0.008	0.015	1.026	11.1834
		B -> Y	0.006	0.012		
	slow_vdd1v2_basicCells_hvt	A -> Y	0.01	0.029		10.467
		B -> Y	0.007	0.021		
	slow_vdd1v0_basicCells_lvt	A -> Y	0.01	0.024		7.7373
		B -> Y	0.007	0.018		
	slow_vdd1v0_basicCells_hvt	A -> Y	0.017	0.066		7.2072
		B -> Y	0.012	0.05		
	fast_vdd1v2_basicCells_lvt	A -> Y	0.004	0.005		20.0986
		B -> Y	0.003	0.004		
	fast_vdd1v2_basicCells_hvt	A -> Y	0.006	0.008		15.4143
		B -> Y	0.004	0.006		
	fast_vdd1v0_basicCells_lvt	A -> Y	0.005	0.006		14.249
		B -> Y	0.004	0.005		
	fast_vdd1v0_basicCells_hvt	A -> Y	0.007	0.011		10.5962
		B -> Y	0.005	0.008		

- Kết quả của MUX

Cell	Corner	Path	t_rise(ns)	t_fall(ns)	Area(um^2)	Power(nW)
MUX	slow_vdd1v2_basicCells_lvt	A -> Y	0.048	0.044	2.394	40.5591
		B -> Y	0.048	0.045		
		S->Y	0.051	0.048		
	slow_vdd1v2_basicCells_hvt	A -> Y	0.088	0.064		36.2320
		B -> Y	0.087	0.065		
		S->Y	0.089	0.074		
	slow_vdd1v0_basicCells_lvt	A -> Y	0.077	0.066		27.1963
		B -> Y	0.076	0.068		
		S->Y	0.080	0.073		
	slow_vdd1v0_basicCells_hvt	A -> Y	0.203	0.126		23.9599
		B -> Y	0.202	0.130		
		S->Y	0.199	0.152		
	fast_vdd1v2_basicCells_lvt	A -> Y	0.015	0.018		64.8883
		B -> Y	0.015	0.018		
		S->Y	0.017	0.019		
	fast_vdd1v2_basicCells_hvt	A -> Y	0.024	0.027		54.7016
		B -> Y	0.024	0.027		
		S->Y	0.027	0.029		
	fast_vdd1v0_basicCells_lvt	A -> Y	0.020	0.022		46.1418
		B -> Y	0.019	0.022		
		S->Y	0.022	0.023		
	fast_vdd1v0_basicCells_hvt	A -> Y	0.034	0.036		36.5866
		B -> Y	0.034	0.037		
		S->Y	0.038	0.040		

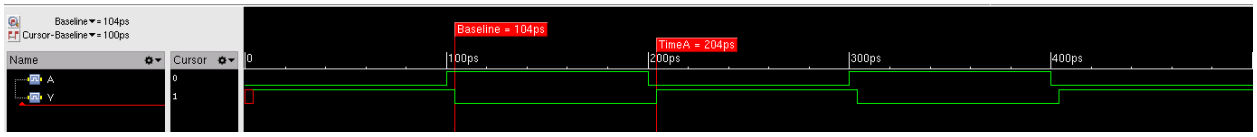
- **Kết quả của FULL ADDER**

Cell	Corner	Path	t_rise(ns)	t_fall(ns)	Area (um^2)	Power (nW)
FA	slow_vdd1v2_basicCells_lvt	A -> S	0.104	0.107	5.13	99.0057
		A -> Co	0.068	0.062		
		B -> S	0.108	0.106		
		B -> Co	0.068	0.061		
		Ci -> S	0.106	0.097		
		Ci -> Co	0.062	0.059		
	slow_vdd1v2_basicCells_hvt	A -> S	0.175	0.181		84.0672
		A -> Co	0.124	0.089		
		B -> S	0.176	0.176		
		B -> Co	0.121	0.089		
		Ci -> S	0.160	0.163		
		Ci -> Co	0.112	0.086		
	slow_vdd1v0_basicCells_lvt	A -> S	0.163	0.171		64.4167
		A -> Co	0.109	0.094		
		B -> S	0.170	0.168		
		B -> Co	0.108	0.094		
		Ci -> S	0.166	0.154		
		Ci -> Co	0.100	0.092		
	slow_vdd1v0_basicCells_hvt	A -> S	0.418	0.400		53.6985
		A -> Co	0.288	0.174		
		B -> S	0.403	0.389		
		B -> Co	0.282	0.176		
		Ci -> S	0.383	0.362		
		Ci -> Co	0.267	0.173		
	fast_vdd1v2_basicCells_lvt	A -> S	0.038	0.037		164.422
		A -> Co	0.022	0.026		
		B -> S	0.038	0.037		
		B -> Co	0.022	0.025		
		Ci -> S	0.038	0.036		
		Ci -> Co	0.020	0.024		
	fast_vdd1v2_basicCells_hvt	A -> S	0.058	0.057		135.372
		A -> Co	0.034	0.039		
		B -> S	0.059	0.056		
		B -> Co	0.034	0.038		
		Ci -> S	0.054	0.054		
		Ci -> Co	0.031	0.036		
	fast_vdd1v0_basicCells_lvt	A -> S	0.047	0.047		113.149

		A -> Co	0.027	0.032		
		B -> S	0.048	0.046		
		B -> Co	0.028	0.031		
		Ci -> S	0.047	0.044		
		Ci -> Co	0.026	0.030		
	fast_vdd1v0_basicCells_hvt	A -> S	0.080	0.081		87.2063
		A -> Co	0.048	0.053		
		B -> S	0.083	0.080		
		B -> Co	0.048	0.054		
		Ci -> S	0.075	0.077		
		Ci -> Co	0.044	0.051		

1.3. Chạy testbench và kết luận

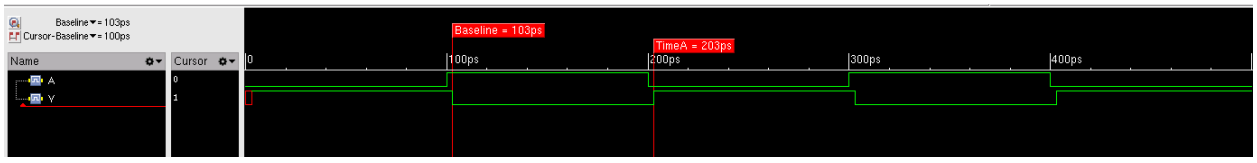
a. NOT



Hình 1.1: Mô phỏng cổng NOT tại corner ff_1v0_hvt

Tại thời điểm 100ps: Ngõ vào A chuyển từ mức thấp (0) lên mức cao (1). Ngõ ra Y bắt đầu chuyển từ cao xuống thấp tại 104ps. => $\Delta t = 4ps$

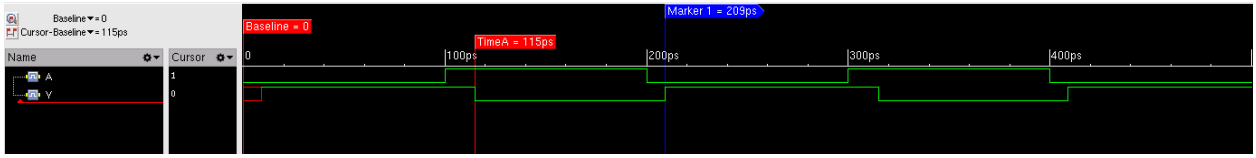
Tại thời điểm 200ps: Ngõ vào A chuyển từ mức cao (1) xuống mức thấp (0). Ngõ ra Y bắt đầu chuyển từ thấp lên cao tại 204ps. => $\Delta t = 4ps$



Hình 1.2: Mô phỏng cổng NOT tại corner ff_1v2_hvt

Trường hợp 1: A thay đổi, Y thay đổi từ 1 sang 0 Dựa trên dạng waveform (tại thời điểm 100ps đến 103ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 3ps(t_fall).

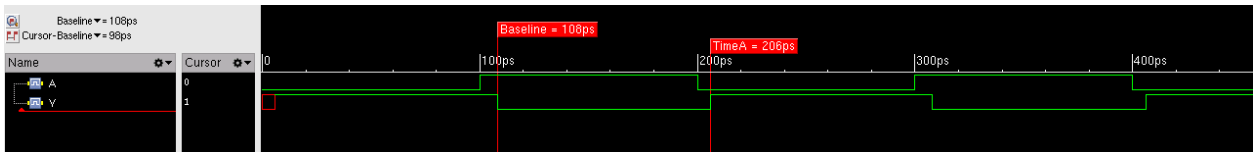
Trường hợp 2: A thay đổi, Y thay đổi từ 0 sang 1 Dựa trên dạng waveform (tại thời điểm 200ps đến 203ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 3ps(t_rise).



Hình 1.3: Mô phỏng cổng NOT tại corner sl_1v0_hvt

Tại thời điểm 100ps: Ngõ vào A chuyển từ mức thấp (0) lên mức cao (1). Ngõ ra Y bắt đầu chuyển từ cao xuống thấp tại 115ps. => $\Delta t = 5ps$

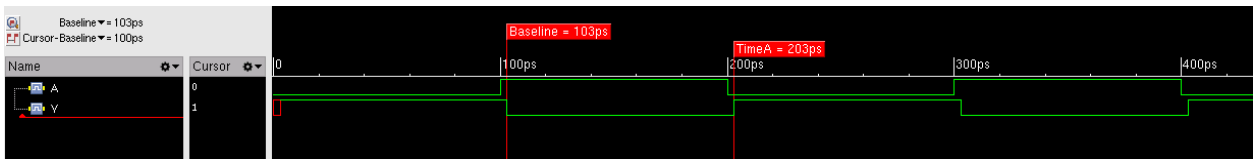
Tại thời điểm 200ps: Ngõ vào A chuyển từ mức cao (1) xuống mức thấp (0). Ngõ ra Y bắt đầu chuyển từ thấp lên cao tại 209ps. => $\Delta t = 9ps$



Hình 1.4: Mô phỏng cổng NOT tại corner sl_1v2_hvt

Trường hợp 1: A thay đổi, Y thay đổi từ 1 sang 0 Dựa trên dạng waveform (tại thời điểm 100ps đến 108ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 8ps(t_fall).

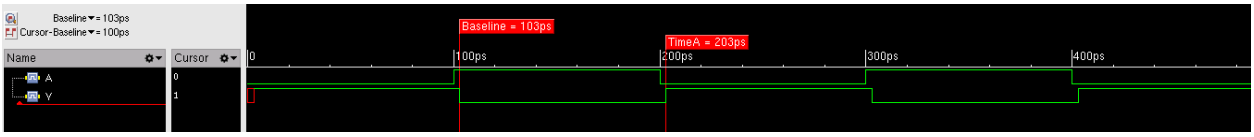
Trường hợp 2: A thay đổi, Y thay đổi từ 0 sang 1 Dựa trên dạng waveform (tại thời điểm 200ps đến 206ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 6ps(t_rise).



Hình 1.5: Mô phỏng cổng NOT tại corner ff_1v0_lvt

Trường hợp 1: A thay đổi, Y thay đổi từ 1 sang 0 Dựa trên dạng waveform (tại thời điểm 100ps đến 103ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 3ps(t_fall).

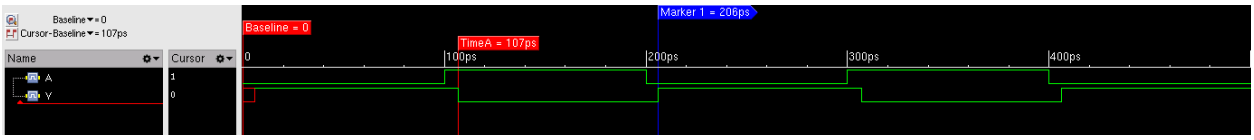
Trường hợp 2: A thay đổi, Y thay đổi từ 0 sang 1 Dựa trên dạng waveform (tại thời điểm 200ps đến 203ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 3ps(t_rise).



Hình 1.6: Mô phỏng cổng NOT tại corner ff_1v2_lvt

Trường hợp 1: A thay đổi, Y thay đổi từ 1 sang 0 Dựa trên dạng waveform (tại thời điểm 100ps đến 103ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 3ps(t_fall).

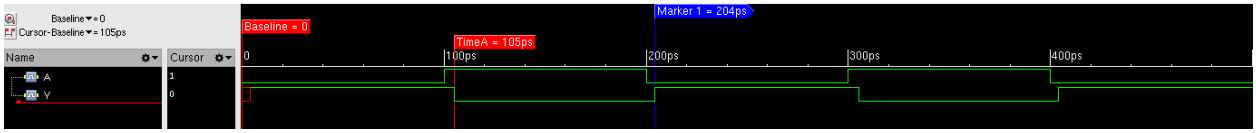
Trường hợp 2: A thay đổi, Y thay đổi từ 0 sang 1 Dựa trên dạng waveform (tại thời điểm 200ps đến 203ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 3ps(t_rise).



Hình 1.7: Mô phỏng cổng NOT tại corner sl_1v0_lvt

Trường hợp 1: A thay đổi, Y thay đổi từ 1 sang 0 Dựa trên dạng waveform (tại thời điểm 100ps đến 107ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 7ps(t_fall).

Trường hợp 2: A thay đổi, Y thay đổi từ 0 sang 1 Dựa trên dạng waveform (tại thời điểm 200ps đến 206ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 6ps(t_rise).

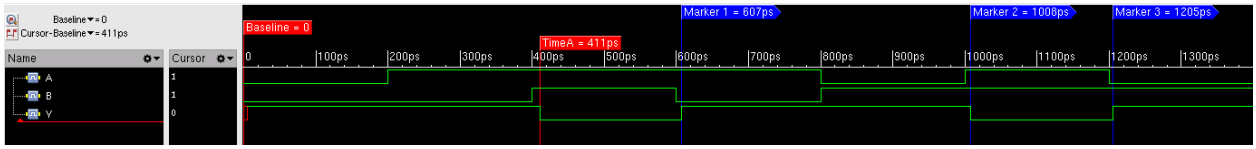


Hình 1.8: Mô phỏng cổng NOT tại corner sl_1v2_lvt

Trường hợp 1: A thay đổi, Y thay đổi từ 1 sang 0 Dựa trên dạng waveform (tại thời điểm 100ps đến 105ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 5ps(t_fall).

Trường hợp 2: A thay đổi, Y thay đổi từ 0 sang 1 Dựa trên dạng waveform (tại thời điểm 200ps đến 204ps), ta nhận thấy ngõ ra Y bị delay 1 khoảng 4ps(t_rise).

b. NAND2



Hình 1.9 : Mô phỏng cổng NAND2 tại corner ff_1v0_hvt

Trường hợp 1: B giữ mức 1, A thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

Tại thời điểm 400ps, A lên mức 1. Tại TimeA = 411ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 411ps - 00ps = 11ps (t_{fall}).

Trường hợp 2: A giữ mức 1, B thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 600ps, B xuống mức 0. Tại Marker 1 = 607ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 607ps - 600ps = 7ps (t_{rise}).

Trường hợp 3: A giữ mức 1, B thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

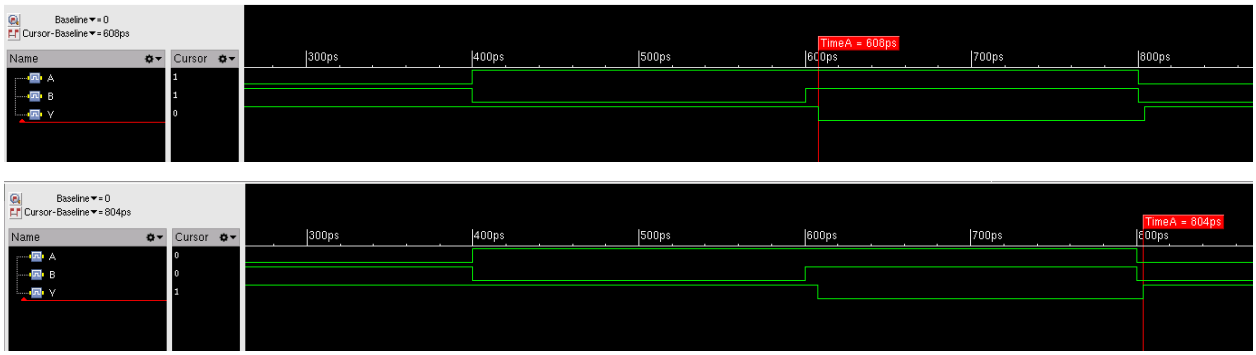
Tại thời điểm 1000ps, B lên mức 1. Tại Marker 2 = 1008ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 1008ps - 1000ps =8ps (t_{fall})

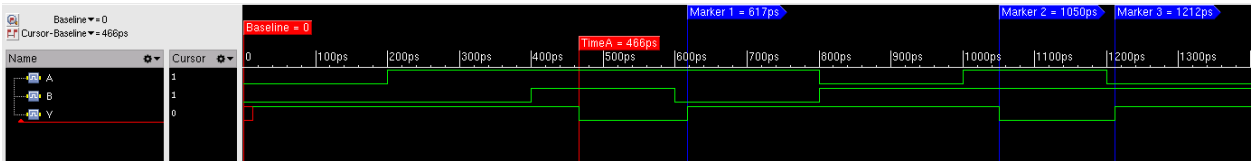
Trường hợp 4: B giữ mức 1, A thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 1200ps, A xuống mức 0. Tại Marker 3 = 1205ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 1205ps - 1200ps =5ps (t_{rise}).



Hình 1.10: Mô phỏng cổng NAND2 tại corner ff_1v2_hvt



Hình 1.11: Mô phỏng cổng NAND2 tại corner sl_1v0_hvt

Trường hợp 1: B giữ mức 1, A thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

Tại thời điểm 400ps, A lên mức 1. Tại TimeA = 466ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 466ps - 400ps = 66ps (t_{fall})

Trường hợp 2: B giữ mức 1, A thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 600ps, A xuống mức 0. Tại Marker 1 = 617ps, Y lên mức 1.

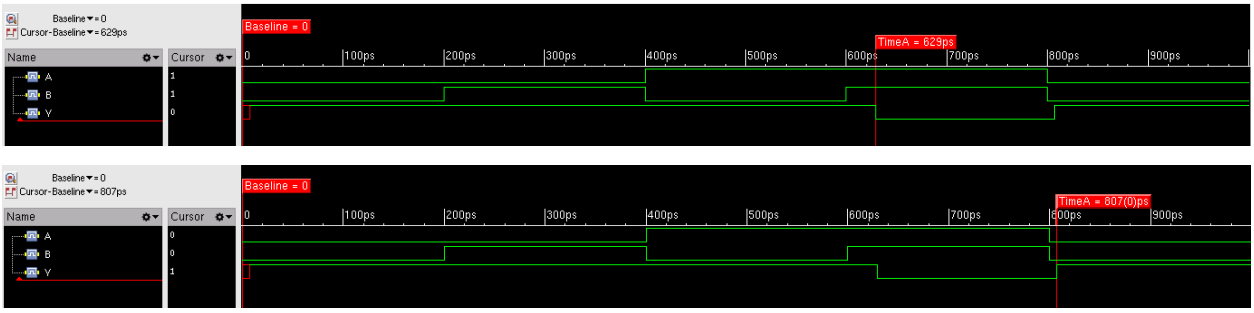
Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng $617ps - 600ps =$ 17ps (t_{rise})

Trường hợp 3: A giữ mức 1, B thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

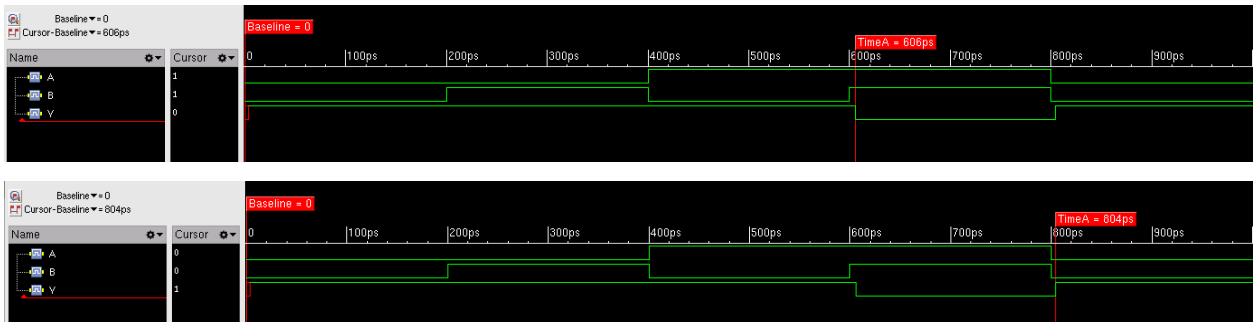
Tại thời điểm 1000ps, B xuống mức 0. Tại Marker 2 = 1050ps, Y lên mức 1. Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng $1050ps - 1000ps =$ 50ps (t_{rise}).

Trường hợp 4: A giữ mức 1, B thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

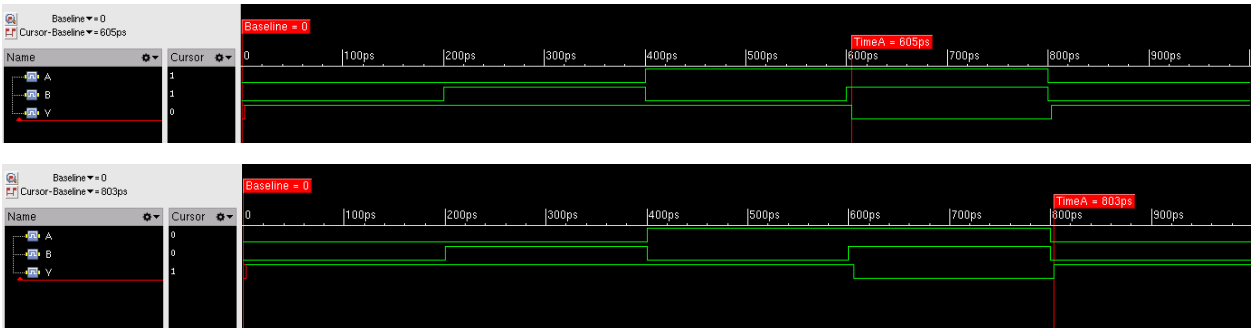
Tại thời điểm 1200ps, B lên mức 1. Tại Marker 3 = 1212ps, Y xuống mức 0. Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng $1212ps - 1200ps =$ 12ps (t_{fall})



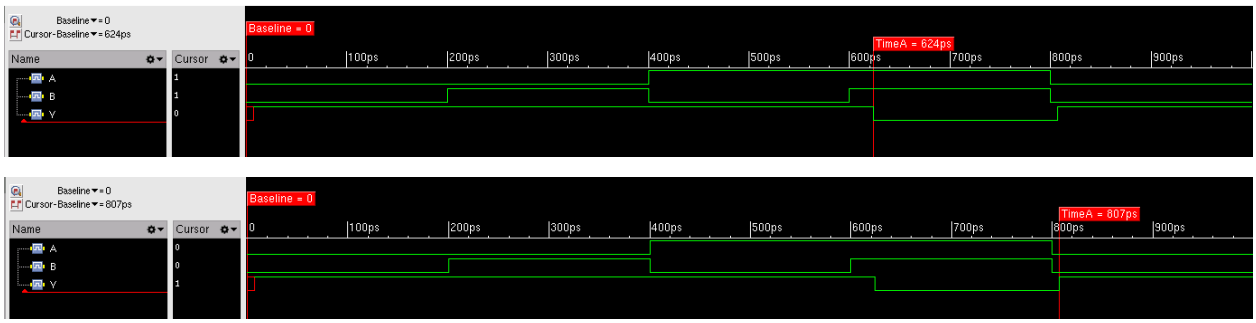
Hình 1.12: Mô phỏng cổng NAND2 tại corner sl_1v2_hvt



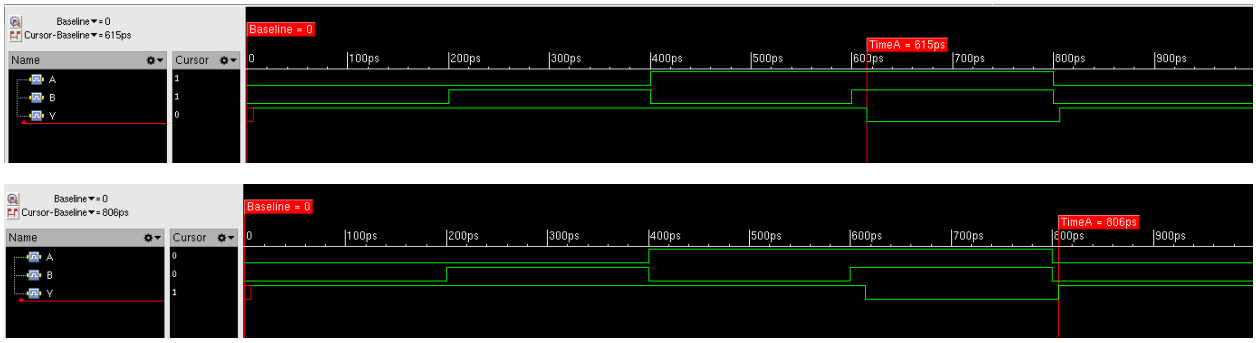
Hình 1.13: Mô phỏng cổng NAND2 tại corner ff_1v0_lvt



Hình 1.14: Mô phỏng cổng NAND2 tại corner ff_1v2_lvt

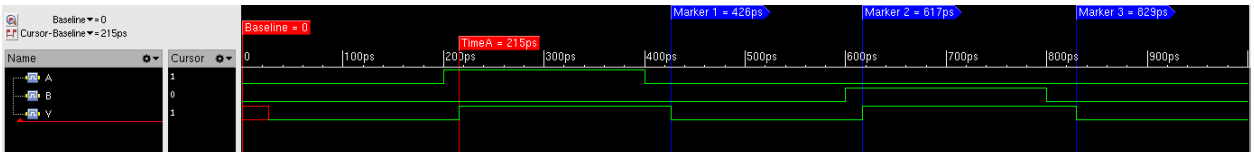


Hình 1.15: Mô phỏng cổng NAND2 tại corner sl_1v0_lvt



Hình 1.16: Mô phỏng cổng NAND2 tại corner sl_1v2_lvt

c. OR2



Hình 1.17: Mô phỏng cổng OR2 tại corner ff_1v0_hvt

Trường hợp 1: B giữ mức 0, A thay đổi từ 0 lên 1, Y thay đổi từ 0 lên 1

Tại thời điểm 200ps, A lên mức 1. Tại TimeA = 215ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 15ps (t_rise).

Trường hợp 2: B giữ mức 0, A thay đổi từ 1 xuống 0, Y thay đổi từ 1 xuống 0

Tại thời điểm 400ps, A xuống mức 0. Tại Marker 1 = 426ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 26ps (t_fall).

Trường hợp 3: A giữ mức 0, B thay đổi từ 0 lên 1, Y thay đổi từ 0 lên 1

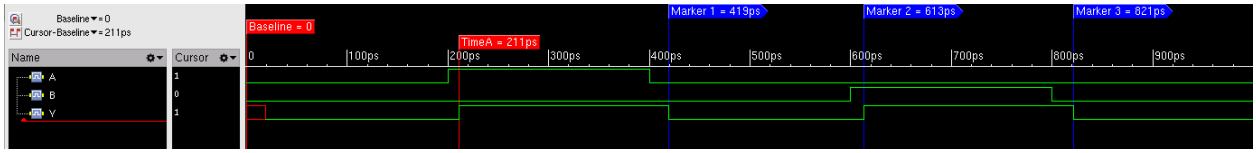
Tại thời điểm 600ps, B lên mức 1. Tại Marker 2 = 617ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 17ps (t_rise).

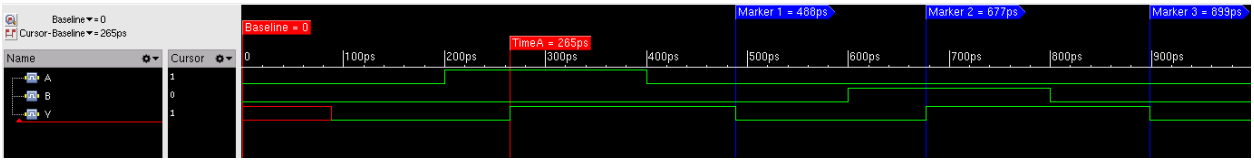
Trường hợp 4: Đo kiểm biên (Boundary case) / Trường hợp khác

Tại thời điểm 800ps, có sự thay đổi trạng thái đầu vào. Tại Marker 3 = 829ps, xác định điểm đo lường.

Dựa trên marker, ta tính được độ trễ là 829ps - 800ps = 29ps. Giá trị này tương ứng với thông số t_tall (A -> Y) trong bảng dữ liệu của corner này.



Hình 1.18: Mô phỏng cổng OR2 tại corner ff_1v2_hvt



Hình 1.19: Mô phỏng cổng OR2 tại corner sl_1v0_hvt

Trường hợp 1: B giữ mức 0, A thay đổi từ 0 lên 1, Y thay đổi từ 0 lên 1

Tại thời điểm 200ps, A lên mức 1. Tại TimeA = 265ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 65ps (t_rise).

Trường hợp 2: B giữ mức 0, A thay đổi từ 1 xuống 0, Y thay đổi từ 1 xuống 0

Tại thời điểm 400ps, A xuống mức 0. Tại Marker 1 = 488ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 88ps (t_fall).

Trường hợp 3: A giữ mức 0, B thay đổi từ 0 lên 1, Y thay đổi từ 0 lên 1

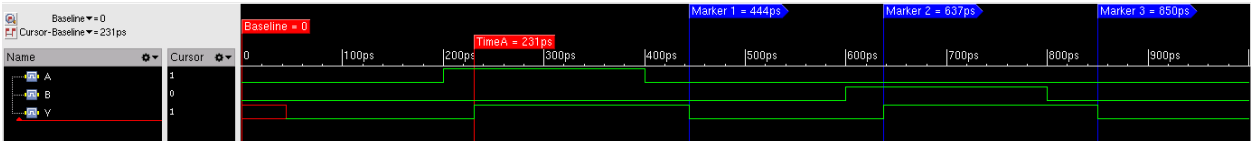
Tại thời điểm 600ps, B lên mức 1. Tại Marker 2 = 677ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 77ps (t_rise).

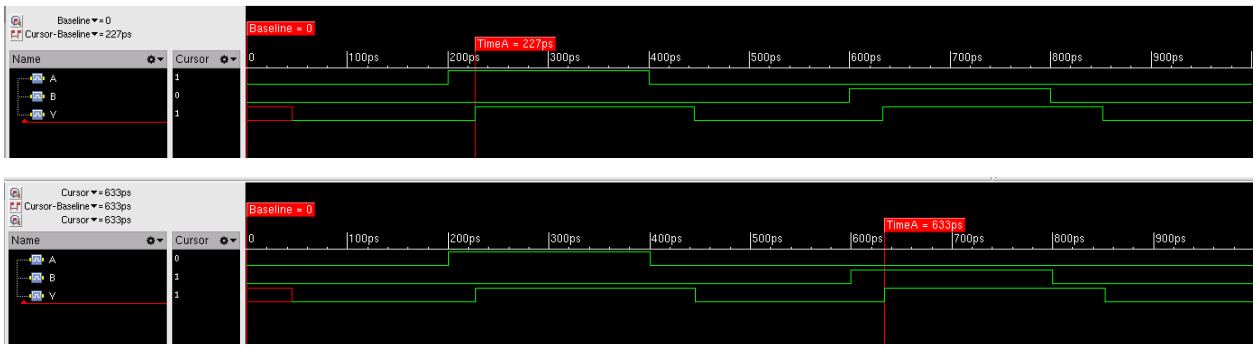
Trường hợp 4: A giữ mức 0, B thay đổi từ 1 xuống 0, Y thay đổi từ 1 xuống 0

Tại thời điểm 800ps, B xuống mức 0. Tại Marker 3 = 899ps, Y xuống mức 0.

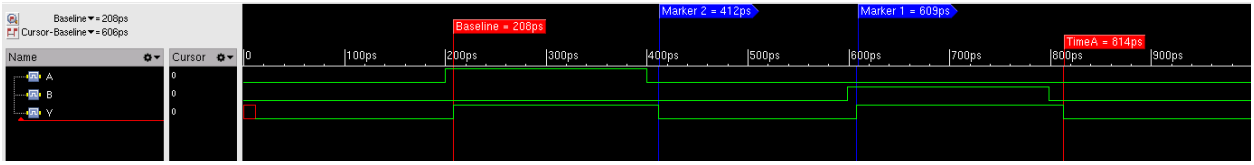
Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 99ps (t_fall)



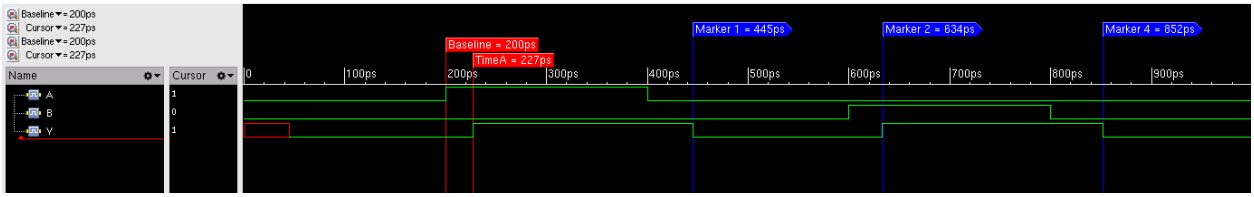
Hình 1.20: Mô phỏng cổng OR2 tại corner sl_1v2_hvt



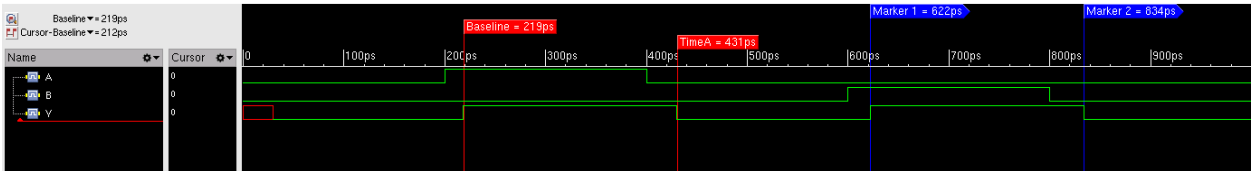
Hình 1.21: Mô phỏng cổng OR2 tại corner ff_1v0_lvt



Hình 1.22: Mô phỏng cổng OR2 tại corner ff_1v2_lvt

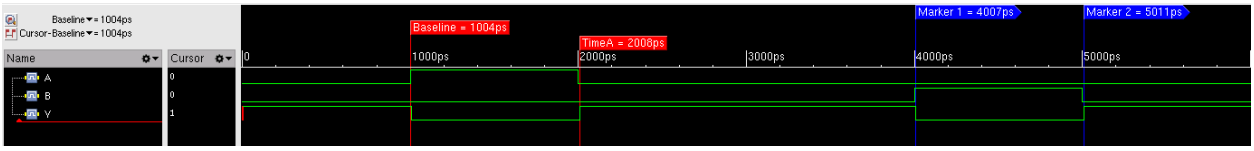


Hình 1.23: Mô phỏng cổng OR2 tại corner sl_1v0_lvt



Hình 1.24: Mô phỏng cổng OR2 tại corner sl_1v2_lvt

d. NOR2



Hình 1.25: Mô phỏng cổng NOR2 tại corner ff_1v0_hvt

Trường hợp 1: B giữ mức 0, A thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

Tại thời điểm 1000ps, A lên mức 1. Tại Baseline = 1004ps, Y xuống mức 0. Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 1004ps - 1000ps = 4ps (t_fall).

Trường hợp 2: B giữ mức 0, A thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 2000ps, A xuống mức 0. Tại TimeA = 2008ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 2008ps - 2000ps =8ps (t_rise).

Trường hợp 3: A giữ mức 0, B thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

Tại thời điểm 4000ps, B lên mức 1. Tại Marker 1 = 4007ps, Y xuống mức 0.

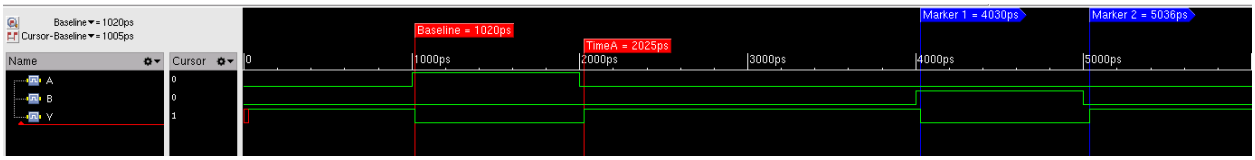
Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 4007ps - 4000ps =7ps (t_fall).

Trường hợp 4: A giữ mức 0, B thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 5000ps, B xuống mức 0. Tại Marker 2 = 5011ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 5011ps - 5000ps =11ps (t_rise).

Hình 1.26: Mô phỏng cổng NOR2 tại corner ff_1v2_hvt



Hình 1.27: Mô phỏng cổng NOR2 tại corner sl_1v0_hvt

Trường hợp 1: B giữ mức 0, A thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

Tại thời điểm 1000ps, A lên mức 1. Tại Baseline = 1020ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 1020ps - 1000ps = 20ps (t_fall).

Trường hợp 2: B giữ mức 0, A thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 2000ps, A xuống mức 0. Tại TimeA = 2025ps, Y lên mức 1.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 2025ps - 2000ps = 25ps (t_rise).

Trường hợp 3: A giữ mức 0, B thay đổi từ 0 lên 1, Y thay đổi từ 1 xuống 0

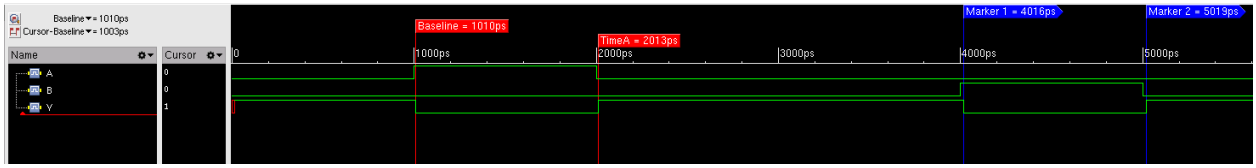
Tại thời điểm 4000ps, B lên mức 1. Tại Marker 1 = 4030ps, Y xuống mức 0.

Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 4030ps - 4000ps = 30ps (t_fall).

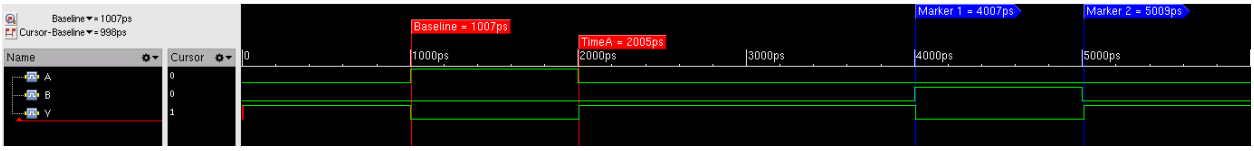
Trường hợp 4: A giữ mức 0, B thay đổi từ 1 xuống 0, Y thay đổi từ 0 lên 1

Tại thời điểm 5000ps, B xuống mức 0. Tại Marker 2 = 5036ps, Y lên mức 1.

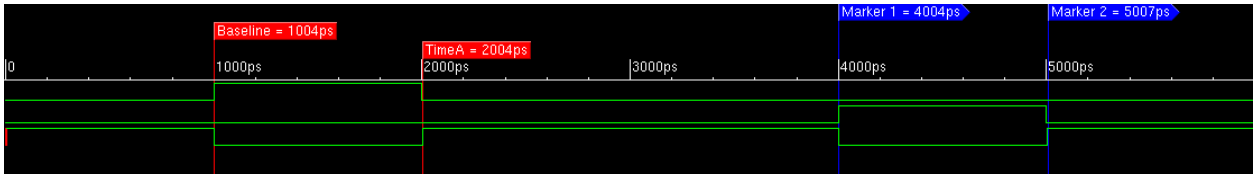
Dựa trên dạng waveform, ta nhận thấy ngõ ra Y bị delay 1 khoảng 5036ps - 5000ps = 36ps (t_rise).



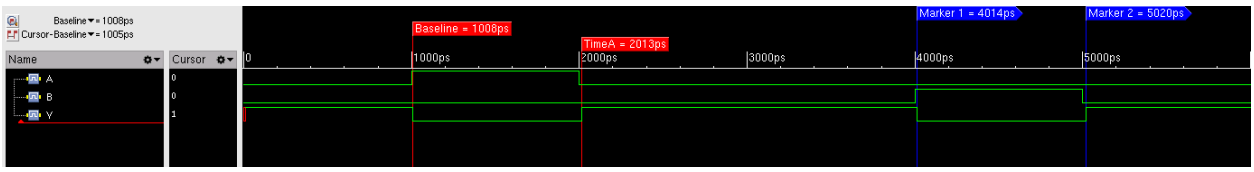
Hình 1.28: Mô phỏng cổng NOR2 tại corner sl_1v2_hvt



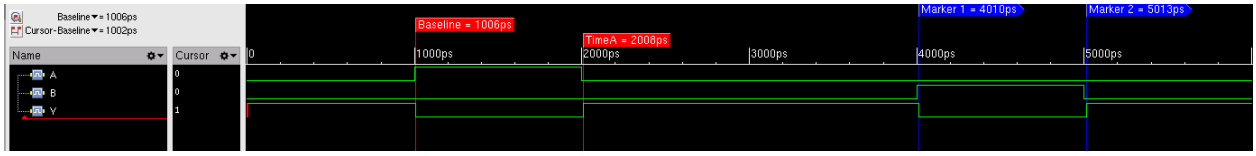
Hình 1.29: Mô phỏng cổng NOR2 tại corner ff_1v0_lvt



Hình 1.30: Mô phỏng cổng NOR2 tại corner ff_1v2_lvt

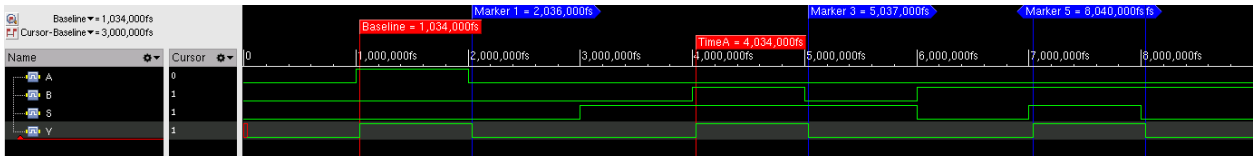


Hình 1.31: Mô phỏng cổng NOR2 tại corner sl_1v0_lvt



Hình 1.32: Mô phỏng cổng NOR2 tại corner sl_1v2_lvt

e. MUX 2:1



Hình 1.33: Mô phỏng cổng MUX21 tại corner ff_1v0_hvt

Trường hợp 1: Chọn ngõ ra A (S=0), tín hiệu A thay đổi

Quan sát:

Tại thời điểm 1,000,000fs (1ns): S=0 (chọn A), A chuyển từ 0 lên 1 → Y chuyển từ 0 lên 1. Marker Baseline xác định thời điểm Y hoàn tất là 1,034,000fs.

Tại thời điểm 2,000,000fs (2ns): S=0 (chọn A), A chuyển từ 1 xuống 0 → Y chuyển từ 1 xuống 0. Marker 1 xác định thời điểm Y hoàn tất là 2,036,000fs.

Phân tích:

Độ trễ đường dẫn A -> Y (t_rise): 1,034,000 - 1,000,000 = 34,000fs = 34ps.

Độ trễ đường dẫn A -> Y (t_fall): 2,036,000 - 2,000,000 = 36,000fs = 36ps.

Trường hợp 2: Chọn ngõ ra B (S=1), tín hiệu B thay đổi

Quan sát:

Tại thời điểm 5,000,000fs (5ns): S=1 (chọn B), B chuyển từ 1 xuống 0 → Y chuyển từ 1 xuống 0. Marker 3 xác định thời điểm Y hoàn tất là 5,037,000fs.

Phân tích:

Độ trễ đường dẫn B -> Y (t_fall): 5,037,000 - 5,000,000 = 37,000fs = 37ps.

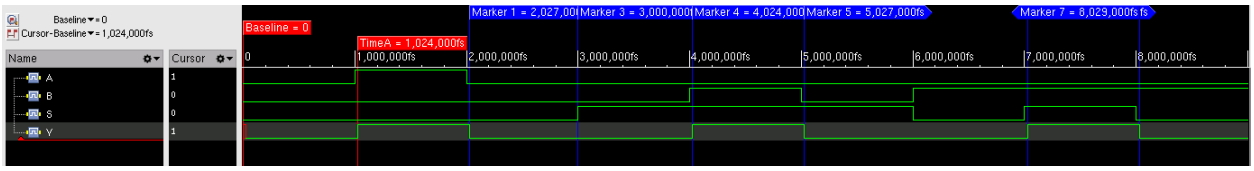
Trường hợp 3: Tín hiệu chọn S thay đổi (Chuyển kênh)

Quan sát:

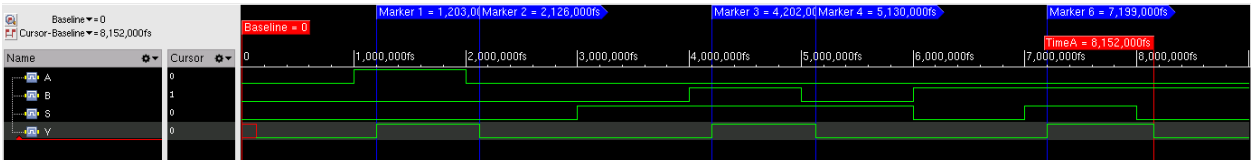
Tại thời điểm 8,000,000fs (8ns): A=0, B=1. S chuyển từ 1 xuống 0 (chuyển chọn B → A). Y chuyển từ 1 (giá trị B) về 0 (giá trị A). Marker 5 xác định thời điểm Y hoàn tất là 8,040,000fs.

Phân tích:

Độ trễ đường dẫn S -> Y (t_fall): 8,040,000 - 8,000,000 = 40,000fs = 40ps.



Hình 1.34: Mô phỏng cổng MUX21 tại corner ff_1v2_hvt



Hình 1.35: Mô phỏng cổng MUX21 tại corner sl_1v0_hvt

Trường hợp 1: Chọn ngõ ra A (S=0), tín hiệu A thay đổi

Quan sát:

Tại thời điểm 1,000,000fs (1ns), S=0, A chuyển từ 0 lên 1. Marker 1 chỉ thời điểm Y hoàn tất là 1,203,000fs.

Tại thời điểm 2,000,000fs (2ns), S=0, A chuyển từ 1 xuống 0. Marker 2 chỉ thời điểm Y hoàn tất là 2,126,000fs.

Phân tích:

Độ trễ đường dẫn A→ Y (t_rise): 1,203,000 - 1,000,000 = 203,000fs = 203ps.

Độ trễ đường dẫn A → Y (t_fall): 2,126,000 - 2,000,000 = 126,000fs = 126ps.

Trường hợp 2: Chọn ngõ ra B (S=1), tín hiệu B thay đổi

Quan sát:

Tại thời điểm 4,000,000fs (4ns), S=1, B chuyển từ 0 lên 1. Marker 3 chỉ thời điểm Y hoàn tất là 4,202,000fs.

Tại thời điểm 5,000,000fs (5ns), S=1, B chuyển từ 1 xuống 0. Marker 4 chỉ thời điểm Y hoàn tất là 5,130,000fs.

Phân tích:

Độ trễ đường dẫn B → Y (t_rise): 4,202,000 - 4,000,000 = 202ps.

Độ trễ đường dẫn B → Y (t_fall): 5,130,000 - 5,000,000 = 130ps.

Trường hợp 3: Tín hiệu chọn S thay đổi (Chuyển kênh)

Quan sát:

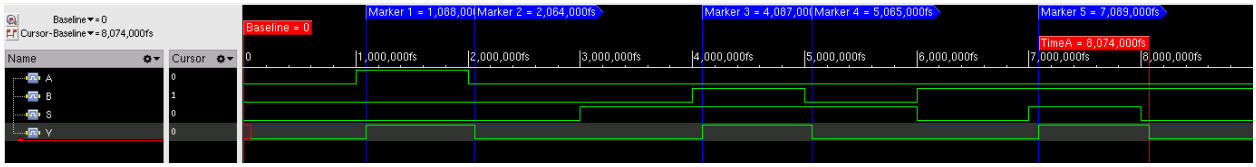
Tại thời điểm 7,000,000fs (7ns), A=0, B=1. S chuyển từ 0 lên 1 (chuyển chọn A → B). Y chuyển từ 0 lên 1. Marker 6 chỉ thời điểm Y hoàn tất là 7,199,000fs.

Tại thời điểm 8,000,000fs (8ns), A=0, B=1. S chuyển từ 1 xuống 0 (chuyển chọn B → A). Y chuyển từ 1 xuống 0. TimeA chỉ thời điểm Y hoàn tất là 8,152,000fs.

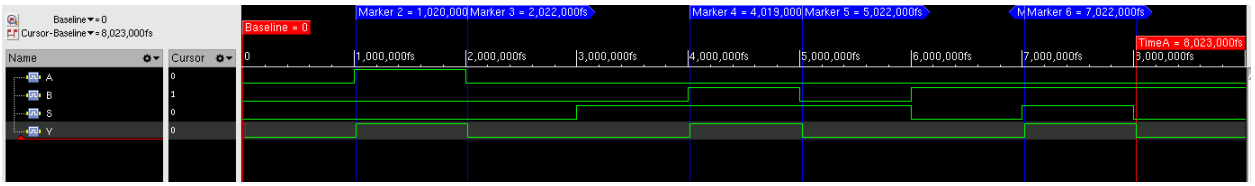
Phân tích:

Độ trễ đường dẫn S → Y (t_rise): 7,199,000 - 7,000,000 = 199ps.

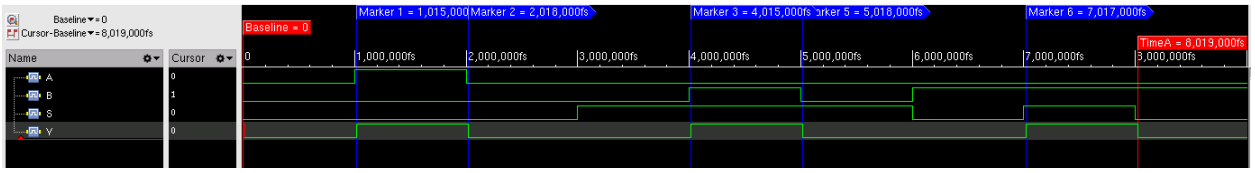
Độ trễ đường dẫn S → Y (t_fall): 8,152,000 - 8,000,000 = 152ps.



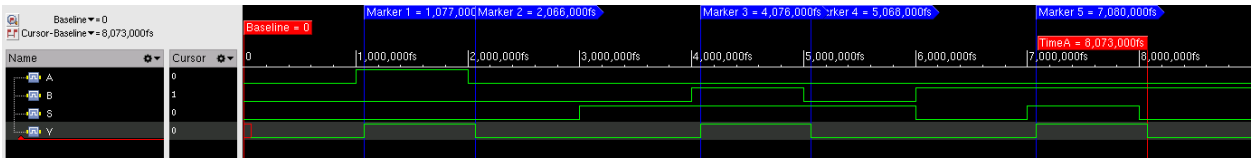
Hình 1.36: Mô phỏng cổng MUX21 tại corner sl_1v2_hvt



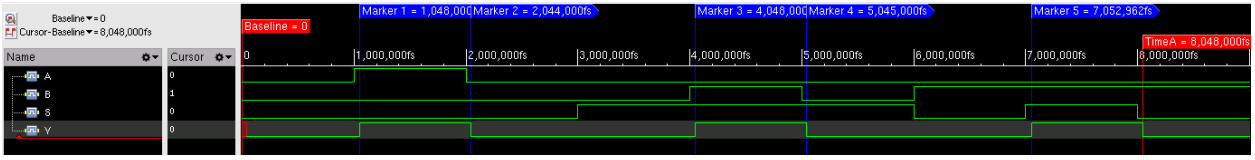
Hình 1.37: Mô phỏng cổng MUX21 tại corner ff_1v0_lvt



Hình 1.38: Mô phỏng cổng MUX21 tại corner ff_1v2_lvt

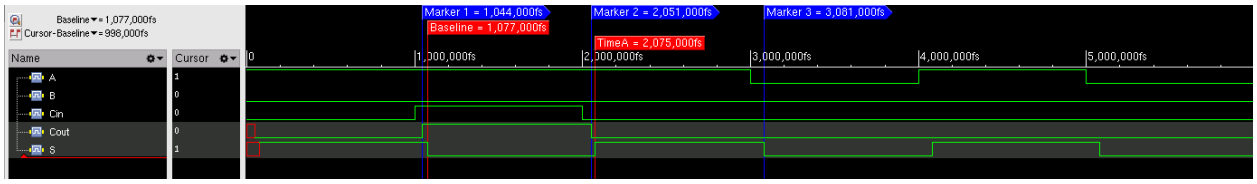


Hình 1.39: Mô phỏng cổng MUX21 tại corner sl_1v0_lvt



Hình 1.40: Mô phỏng cổng MUX21 tại corner sl_1v2_lvt

f. FULL ADDER



Hình 1.41: Mô phỏng cổng FULL_ADDER tại corner ff_1v0_hvt

Trường hợp 1: Tín hiệu A thay đổi (Giữ B=0, Cin=0)

Tại thời điểm 1,000,000fs (1ns): A chuyển từ 0 lên 1. Ngõ ra Sum (S) chuyển từ 0 lên 1. Marker Baseline chỉ thời điểm S hoàn tất là 1,077,000fs.

Tại thời điểm 2,000,000fs (2ns): A chuyển từ 1 xuống 0. Ngõ ra Sum (S) chuyển từ 1 xuống 0. Marker TimeA chỉ thời điểm S hoàn tất là 2,075,000fs.

Phân tích:

Độ trễ đường dẫn A→ S (t_rise): 1,077,000 - 1,000,000 = 77ps.

Độ trễ đường dẫn A → S (t_fall): 2,075,000 - 2,000,000 = 75ps.

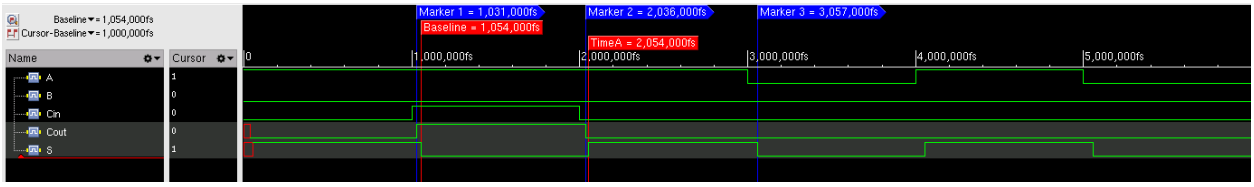
Ngõ ra Cout giữ nguyên ở mức 0

Trường hợp 2: Tín hiệu B thay đổi (Giữ A=0, Cin=0)

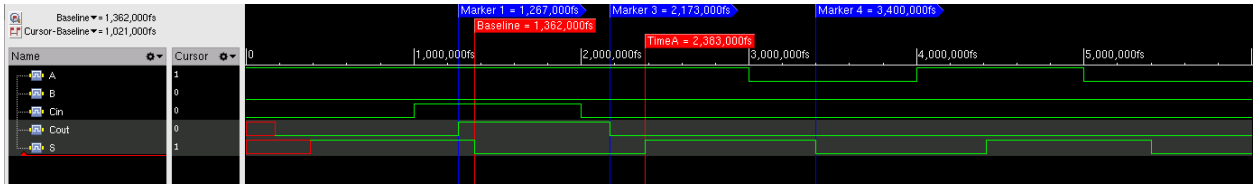
Tại thời điểm 3,000,000fs (3ns): B chuyển từ 0 lên 1. Ngõ ra Sum (S) chuyển từ 0 lên 1. Marker 3 chỉ thời điểm S hoàn tất là 3,081,000fs.

Phân tích:

Độ trễ đường dẫn B → S (t_rise): 3,081,000 - 3,000,000 = 81ps.



Hình 1.42: Mô phỏng cổng FULL_ADDER tại corner ff_1v2_hvt



Hình 1.43: Mô phỏng cổng FULL_ADDER tại corner sl_1v0_hvt

Trường hợp 1: Tín hiệu A thay đổi (Giữ B=0, Cin=0)

Quan sát:

Tại thời điểm 1,000,000fs (1ns): A chuyển từ 0 lên 1. Ngõ ra Sum (S) chuyển từ 0 lên 1. Marker Baseline chỉ thời điểm S hoàn tất là 1,362,000fs.

Tại thời điểm 2,000,000fs (2ns): A chuyển từ 1 xuống 0. Ngõ ra Sum (S) chuyển từ 1 xuống 0. Marker TimeA chỉ thời điểm S hoàn tất là 2,383,000fs.

Phân tích:

- Độ trễ đường dẫn A → S (t_rise): 1,362,000 - 1,000,000 = 362ps.
- Độ trễ đường dẫn A → S (t_fall): 2,383,000 - 2,000,000 = 383ps.

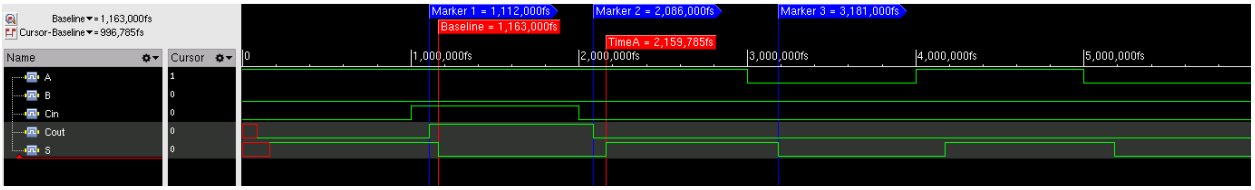
Trường hợp 2: Tín hiệu B thay đổi (Giữ A=0, Cin=0)

Quan sát:

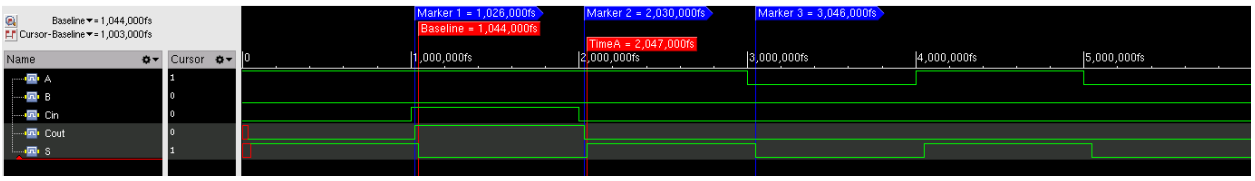
Tại thời điểm 3,000,000fs (3ns): B chuyển từ 0 lên 1. Ngõ ra Sum (S) chuyển từ 0 lên 1. Marker 4 chỉ thời điểm S hoàn tất là 3,400,000fs.

Phân tích:

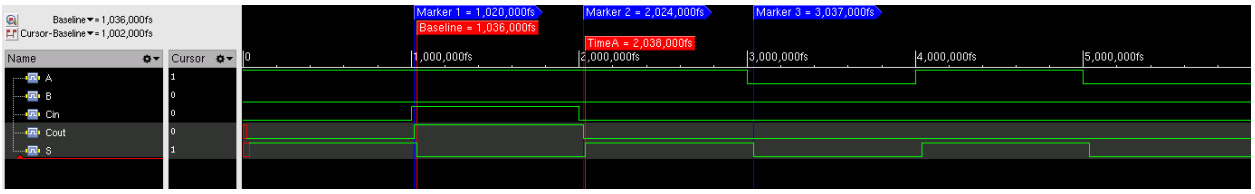
Độ trễ đường dẫn B -> S (t_rise): 3,400,000 - 3,000,000 = 400ps.



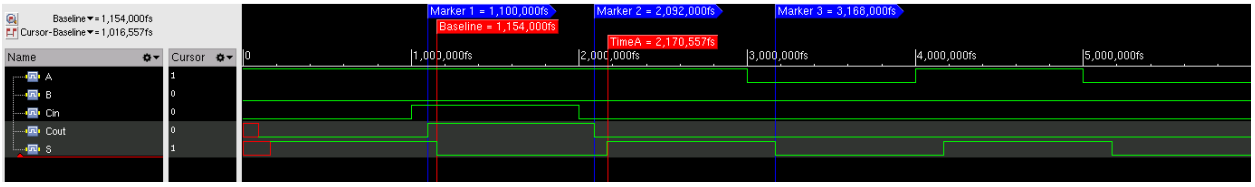
Hình 1.44: Mô phỏng cổng FULL_ADDER tại corner sl_1v2_hvt



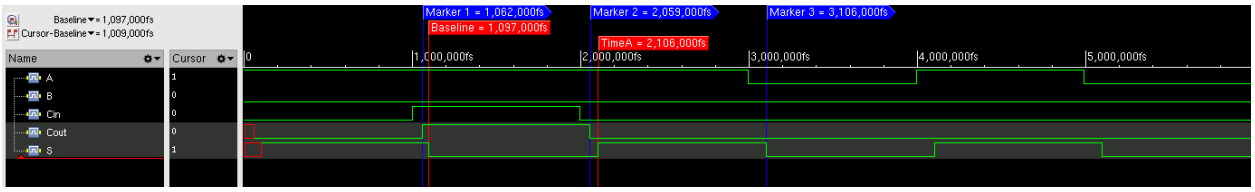
Hình 1.45: Mô phỏng cổng FULL_ADDER tại corner ff_1v0_lvt



Hình 1.46: Mô phỏng cổng FULL_ADDER tại corner ff_1v2_lvt



Hình 1.47: Mô phỏng cổng FULL_ADDER tại corner sl_1v0_lvt



Hình 1.48: Mô phỏng cổng FULL_ADDER tại corner sl_1v2_lvt

2. THÍ NGHIỆM 2

Objective: Perform gate-level simulation on the netlist after synthesis.

Description: In Experiment 1, we analyzed combinational logic cells. In this experiment, we will examine a more complex combinational circuit.

Đoạn code của bài thí nghiệm này:

```
module alu (
    input logic [1:0] A,
    input logic [1:0] B,
    input logic [1:0] S,
    output logic [1:0] Y
    );
    always_comb begin
        case (S)
        2'b00: Y=A+B;
        2'b01: Y=A-B;
        2'b10: Y=A&B;
        2'b11: Y=A|B;
        endcase
    end
endmodule
```

Thư viện sử dụng: *slow_vdd1v2_basicCells_hvt.lib*

2.1. Chuẩn bị các files cần thiết để chạy trên server:

2.1.1. TCL file (synth.tcl)

```
set_db init_lib_search_path {/home/yellow/ee3201_16/LAB2/gpdk045_lib}
set_db init_hdl_search_path {/home/yellow/ee3201_16/LAB2/Exp2/00_src}
read_libs slow_vdd1v2_basicCells_hvt.lib
read_hdl -sv alu.sv
elaborate
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
#syn_opt ---> We do not use this command in this experiment

report_timing   > 02_reports/timing.rpt
report_power    > 02_reports/power.rpt
report_area     > 02_reports/area.rpt
report_qor      > 02_reports/qor.rpt

write_hdl       > 03_outputs/alu_netlist.sv
```

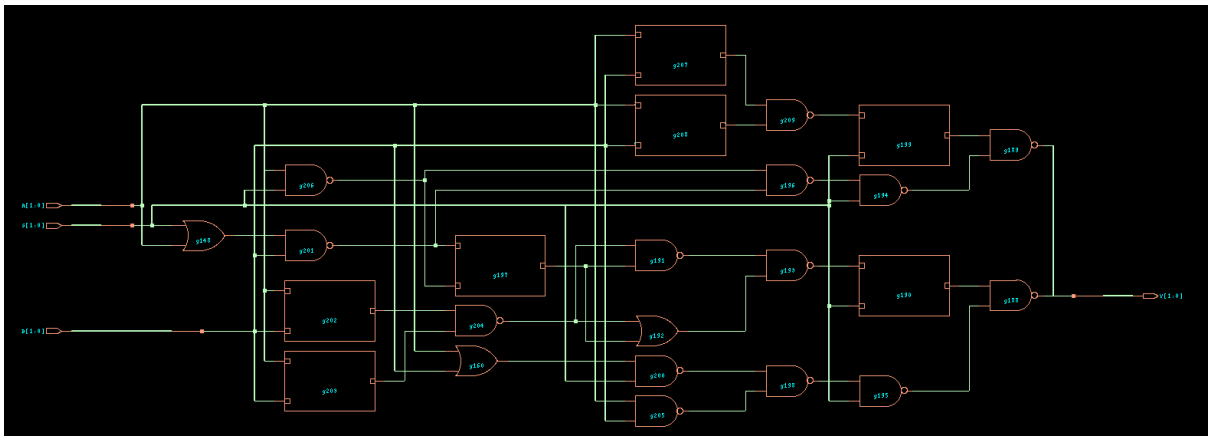
```
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -
setuphold split > 03_outputs/delay.sdf
```

2.1.2. *Testbench file (alu_tb.sv)*

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
    `include "./../03_outputs/alu_netlist.sv"
    `include "./../slow_vdd1v2_basicCells_hvt.v"
`else
    `include "./../00_src/alu.sv"
`endif

module alu_tb;
    parameter T = 20;
    logic [1:0] A, B, S;
    logic [1:0] Y;

    alu DUT (
        .A(A), .B(B), .S(S), .Y(Y)
    );

`ifdef ANNOTATION
    initial begin
        $sdf_annotate("./../Exp2/03_outputs/delay.sdf", DUT, , "annotate.log",
"MAXIMUM");
    end
`endif

    initial begin
        $dumpfile("alu_wave_T20.vcd");
        $dumpvars(0, alu_tb.DUT);

        #0;   S=2'b00; A=2'b01; B=2'b00;
        #(T); S=2'b00; A=2'b11; B=2'b01;
        #(T); S=2'b01; A=2'b11; B=2'b00;
        #(T); S=2'b01; A=2'b11; B=2'b11;
        #(T); S=2'b10; A=2'b11; B=2'b01;
        #(T); S=2'b10; A=2'b01; B=2'b10;
        #(T); S=2'b11; A=2'b11; B=2'b01;
        #(T); S=2'b11; A=2'b01; B=2'b10;
        #(T); $finish;
    end
endmodule
```

2.2. Kết quả sau khi chạy tool Genus trên server

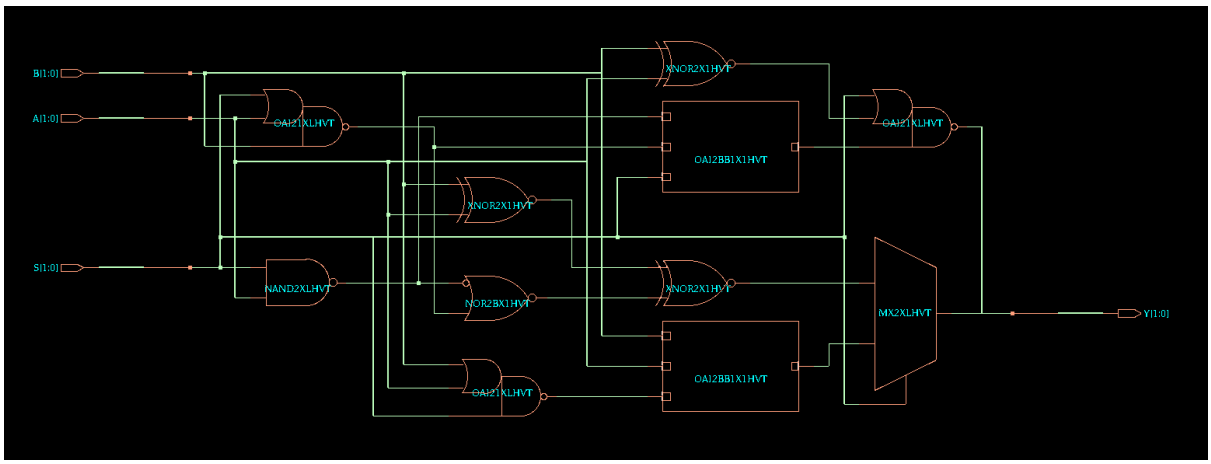
2.2.1. *Kết quả synthesis (trước khi mapping)*



Hình 2.1: Kết quả schematic sau khi synthesis (không mapping)

Khi bỏ đi lệnh syn_map, công cụ chuyển đổi code RTL sang các cổng logic tổng quát (generic gates), tuy dễ hiểu nhưng có cấu trúc chưa tối ưu về mặt công nghệ.

2.2.2. *Kết quả synthesis (sau khi mapping)*



Hình 2.2: Kết quả schematic sau khi synthesis (có mapping)

Sau khi thêm vào lệnh syn_map, thiết kế được ánh xạ vào thư viện công nghệ 45nm. Các cell đa năng như MUX, OAI2... được sử dụng giúp giảm số lượng cổng, tối ưu hóa diện tích và công suất.

2.2.3. *Đọc các file outputs*

a) Netlist file (alu_netlist.sv)

File netlist chứa toàn bộ các cells trong schematic, và cách kết nối giữa chúng thông qua dây nối (wires)

```
// Generated by Cadence Genus(TM) Synthesis Solution 21.19-s055_1
// Generated on: Nov 17 2025 10:25:17 +07 (Nov 17 2025 03:25:17 UTC)
```

```
// Verification Directory fv/alu

module alu(A, B, S, Y);
  input [1:0] A, B, S;
  output [1:0] Y;
  wire [1:0] A, B, S;
  wire [1:0] Y;
  wire n_0, n_1, n_2, n_3, n_4, n_5, n_6, n_7;
  wire n_9;
  MX2XLHVT g364__2398(.A (n_9), .B (n_3), .S0 (S[1]), .Y (Y[1]));
  XNOR2X1HVT g367__5107(.A (n_0), .B (n_6), .Y (n_9));
  OAI21XLHVT g365__6260(.A0 (S[1]), .A1 (n_2), .B0 (n_7), .Y (Y[0]));
  OAI2BB1X1HVT g368__4319(.A0N (n_5), .A1N (n_4), .B0 (S[1]), .Y (n_7));
  NOR2BX1HVT g370__8428(.AN (n_5), .B (n_4), .Y (n_6));
  OAI2BB1X1HVT g366__5526(.A0N (B[1]), .A1N (A[1]), .B0 (n_1), .Y (n_3));
  XNOR2X1HVT g372__6783(.A (B[0]), .B (A[0]), .Y (n_2));
  OAI21XLHVT g369__3680(.A0 (B[1]), .A1 (A[1]), .B0 (S[0]), .Y (n_1));
  XNOR2X1HVT g373__1617(.A (B[1]), .B (A[1]), .Y (n_0));
  OAI21XLHVT g371__2802(.A0 (S[0]), .A1 (A[0]), .B0 (B[0]), .Y (n_4));
  NAND2XLHVT g374__1705(.A (S[0]), .B (A[0]), .Y (n_5));
endmodule
```

b) SDF file (delay.sdf)

```
(DELAYFILE
  (SDFVERSION  "OVI 3.0")
  (DESIGN      "alu")
  (DATE        "Mon Nov 17 10:25:17 +07 2025")
  (VENDOR      "Cadence, Inc.")
  (PROGRAM     "Genus(TM) Synthesis Solution")
  (VERSION     "21.19-s055_1")
  (DIVIDER     .)
  (VOLTAGE     ::1.08)
  (PROCESS     "::1.0")
  (TEMPERATURE ::125.0)
  (TIMESCALE   1ns)
  (CELL
     (CELLTYPE "MX2XLHVT")
     (INSTANCE g364__2398)
     (DELAY
        (ABSOLUTE
          (PORT A (::0.000))
          (PORT B (::0.000))
          (PORT S0 (::0.000))
          (IOPATH A Y (::0.095) (::0.070))
          (IOPATH B Y (::0.097) (::0.080))
          (IOPATH S0 Y (::0.089) (::0.074))
        )
```

```
    )
  )
  (CELL
    (CELLTYPE "XNOR2X1HVT")
    (INSTANCE g367__5107)
    (DELAY
      (ABSOLUTE
        (PORT A (::0.000))
        (PORT B (::0.000))
        (IOPATH A Y (::0.131) (::0.110))
        (IOPATH B Y (::0.112) (::0.087))
      )
    )
  )
  (CELL
    (CELLTYPE "OAI21XLHVT")
    (INSTANCE g365__6260)
    (DELAY
      (ABSOLUTE
        (PORT A0 (::0.000))
        (PORT A1 (::0.000))
        (PORT B0 (::0.000))
        (IOPATH A0 Y (::0.028) (::0.045))
        (IOPATH B0 Y (::0.026) (::0.044))
        (IOPATH A1 Y (::0.030) (::0.041))
      )
    )
  )
  (CELL
    (CELLTYPE "OAI2BB1X1HVT")
    (INSTANCE g368__4319)
    (DELAY
      (ABSOLUTE
        (PORT A0N (::0.000))
        (PORT A1N (::0.000))
        (PORT B0 (::0.000))
        (IOPATH A1N Y (::0.098) (::0.077))
        (IOPATH B0 Y (::0.012) (::0.035))
        (IOPATH A0N Y (::0.092) (::0.074))
      )
    )
  )
  (CELL
    (CELLTYPE "NOR2BX1HVT")
    (INSTANCE g370__8428)
    (DELAY
      (ABSOLUTE
```

```
          (PORT AN (::0.000))
          (PORT B (::0.000))
          (IOPATH B Y (::0.057) (::0.045))
          (IOPATH AN Y (::0.063) (::0.062))
        )
    )
)
(CELL
    (CELLTYPE "OAI2BB1X1HVT")
    (INSTANCE g366__5526)
    (DELAY
        (ABSOLUTE
          (PORT A0N (::0.000))
          (PORT A1N (::0.000))
          (PORT B0 (::0.000))
          (IOPATH A1N Y (::0.075) (::0.049))
          (IOPATH B0 Y (::0.042) (::0.057))
          (IOPATH A0N Y (::0.082) (::0.052))
        )
    )
)
(CELL
    (CELLTYPE "XNOR2X1HVT")
    (INSTANCE g372__6783)
    (DELAY
        (ABSOLUTE
          (PORT A (::0.000))
          (PORT B (::0.000))
          (IOPATH A Y (::0.125) (::0.104))
          (IOPATH B Y (::0.103) (::0.072))
        )
    )
)
(CELL
    (CELLTYPE "OAI21XLHVT")
    (INSTANCE g369__3680)
    (DELAY
        (ABSOLUTE
          (PORT A0 (::0.000))
          (PORT A1 (::0.000))
          (PORT B0 (::0.000))
          (IOPATH A0 Y (::0.040) (::0.058))
          (IOPATH B0 Y (::0.014) (::0.048))
          (IOPATH A1 Y (::0.033) (::0.046))
        )
    )
)
```

```
  (CELL
     (CELLTYPE "XNOR2X1HVT")
     (INSTANCE g373__1617)
     (DELAY
        (ABSOLUTE
          (PORT A (::0.000))
          (PORT B (::0.000))
          (IOPATH A Y (::0.125) (::0.104))
          (IOPATH B Y (::0.103) (::0.072))
        )
     )
  )
  (CELL
     (CELLTYPE "OAI21XLHVT")
     (INSTANCE g371__2802)
     (DELAY
        (ABSOLUTE
          (PORT A0 (::0.000))
          (PORT A1 (::0.000))
          (PORT B0 (::0.000))
          (IOPATH A0 Y (::0.046) (::0.067))
          (IOPATH B0 Y (::0.017) (::0.057))
          (IOPATH A1 Y (::0.039) (::0.055))
        )
     )
  )
  (CELL
     (CELLTYPE "NAND2XLHVT")
     (INSTANCE g374__1705)
     (DELAY
        (ABSOLUTE
          (PORT A (::0.000))
          (PORT B (::0.000))
          (IOPATH A Y (::0.016) (::0.047))
          (IOPATH B Y (::0.014) (::0.039))
        )
     )
  )
)
```

File này chứa các thông số liên quan đến hiện tượng trễ tín hiệu (delay) của từng standard cell theo thư viện đã cho. Cụ thể:

(TIMESCALE 1ns): Đơn vị thời gian được sử dụng trong file này là nano giây

(CELLTYPE "NAND2X LHVT"): Tên của cell trong thư viện standard cells (cổng NAND 2 ngõ vào, loại High-Vt).

(INSTANCE g374__1705): Tên instance của cell trong mạch — được gán bởi công cụ synthesis.

ABSOLUTE: Các thông số tuyệt đối về độ trễ, là kết quả đo được của cell ở một số điều kiện

(IOPATH A Y (::0.016) (::0.047)):

- ***IOPATH A Y:*** Đường đi của tín hiệu từ ngõ vào A đến ngõ ra Y.
- ***(::0.016) (::0.047):***
  - Giá trị đầu tiên (0.016 ns) độ trễ của cạnh lên (0 → 1).
  - Giá trị thứ hai (0.047 ns) độ trễ của cạnh xuống (1 → 0).

Tương tự, **IOPATH B Y** là đường đi của tín hiệu từ ngõ vào B đến ngõ ra Y

2.3. Kiểm thử và kết luận

2.3.1. Các trường hợp kiểm thử

Bài kiểm thử bao gồm 3 trạng thái chu kỳ, mỗi chu kỳ gồm 8 trường hợp, mục đích là kiểm thử tối đa các kết quả có thể có của ngõ ra Y theo các ngõ vào S, A và B theo mức độ trễ của các standard cells. Cụ thể như sau:

Bảng 2.1: Các trường hợp kiểm thử khối ALU

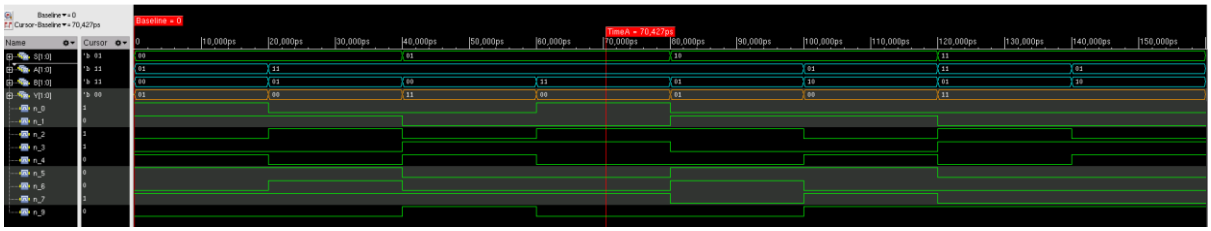
Trường hợp	Thời gian theo chu kỳ (T)			S	A	B	Y (Dự kiến)
	T = 20 ns	T = 100 ns	T = 200 ns				
1	0 ns	0 ns	0 ns	00	01	00	01
2	20 ns	100 ns	200 ns	00	11	01	00
3	40 ns	200 ns	400 ns	01	11	00	11
4	60 ns	300 ns	600 ns	01	11	11	00
5	80 ns	400 ns	800 ns	10	11	01	01
6	100 ns	500 ns	1000 ns	10	01	10	00
7	120 ns	600 ns	1200 ns	11	11	01	11
8	140 ns	700 ns	1400 ns	11	01	10	11

2.3.2. Chạy trên Xcelium và phân tích dạng sóng trên Simvision

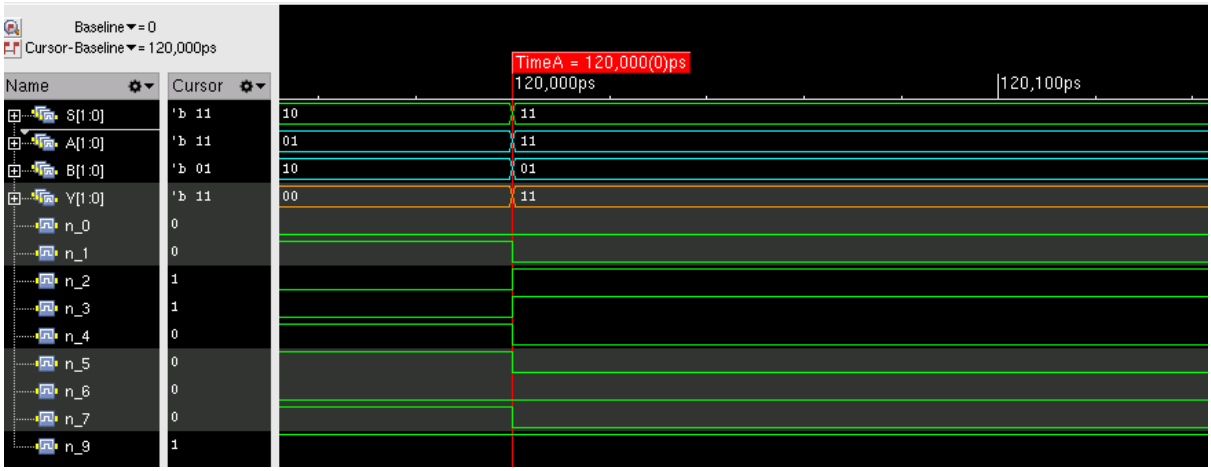
Tại mỗi chu kỳ T, quá trình mô phỏng được chia thành 8 trường hợp (cases) tương ứng với 8 tổ hợp ngõ vào của dạng sóng, từ đó phân tích sự thay đổi của một số

đường dây tại những mốc thời gian chuyển tiếp tín hiệu và phân tích nguyên do gây trễ ở ngõ ra

a) Trường hợp không có delay (với T = 20 ns)



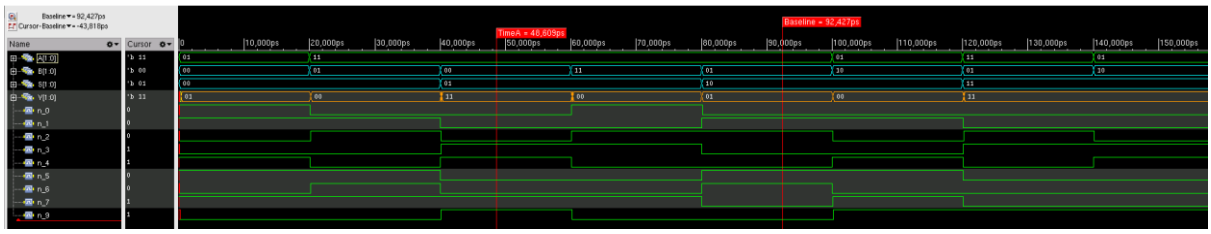
Hình 2.3: dạng sóng khi không áp dụng delay vào testbench



Hình 2.4: Dạng sóng phóng to tại trường hợp 120 ns

Khi không áp dụng delay, trạng thái tín hiệu ngõ ra Y và các dây nối thay đổi cùng thời điểm với ngõ vào, chính xác tại mốc thời gian đang xét

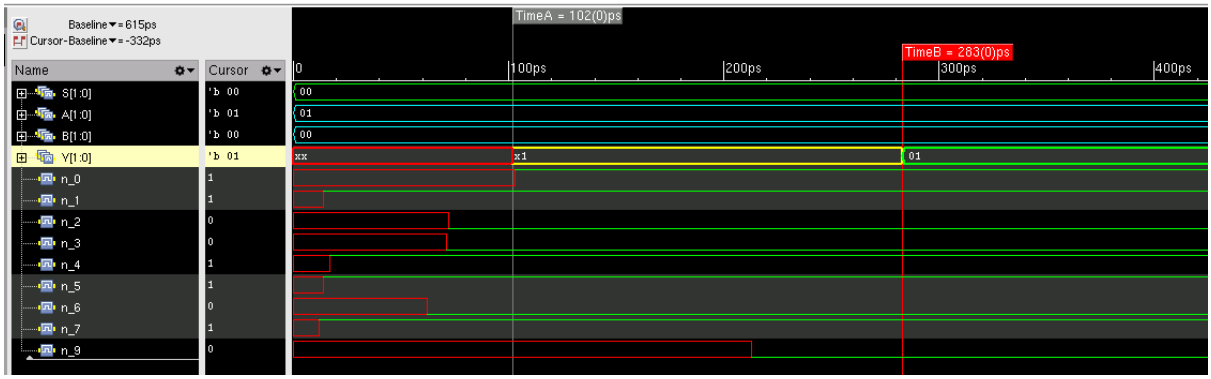
b) Trường hợp T = 20 ns



Hình 2.5: Dạng sóng của toàn bộ bài kiểm thử với T = 20 ns

Từ dạng sóng, ta dễ thấy ngõ ra Y thay đổi và ổn định muộn hơn các ngõ vào (S, A, B) tại các mốc thời gian đang xét. Ngõ ra còn xuất hiện một vài giá trị khác với giá trị mong muốn, dù chúng tồn tại trong một khoảng thời gian rất nhỏ. Sau đây, ta sẽ phân tích cụ thể từng trường hợp để hiểu rõ tại sao lại có hiện tượng này.

- **Tại t = 0 ns**



Hình 2.6: Trạng thái các ngõ vào/ra và các đường dây tại giai đoạn khởi tạo

Tại thời điểm ban đầu t = 0, các tín hiệu đầu vào được khởi tạo: A = 01, B = 00 S = 00 (phép cộng). Ngõ ra Y mong muốn:

$$Y = A + B = 1_D(01)_B + 0_D(00)_B = 1_D(01)_B$$

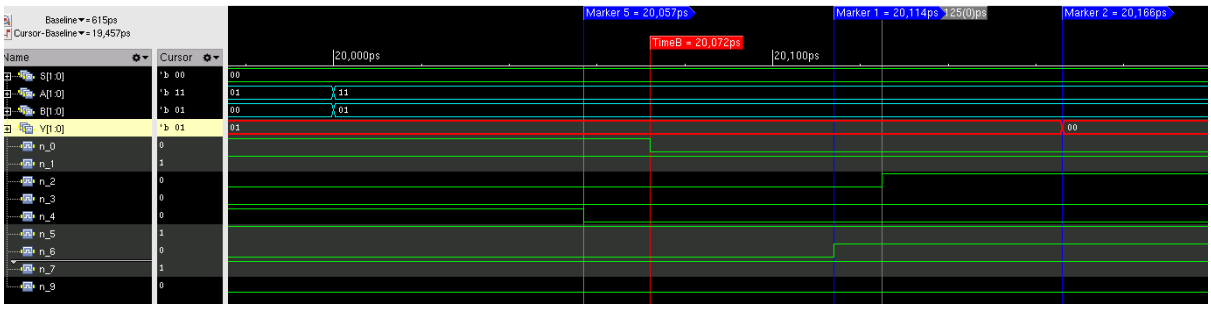
Trên thực tế, ngõ ra Y không đổi giá trị ngay lập tức. Theo dạng sóng, Y thay đổi như sau:

- Từ 0 đến 102 ps: Y vẫn chưa được khởi tạo, có giá trị tùy định (xx)
- Từ 102 ps đến 283 ps: Y mang giá trị trung gian x1
- Sau 283 ps: Y chuyển sang giá trị 01, là giá trị mà ta mong muốn đạt được.

Tín hiệu ngõ ra được xác định và ổn định sau một khoảng trễ lan truyền sau khi ngõ vào thay đổi. Y không chuyển sang giá trị 01 ngay lập tức mà bị trễ một khoảng $\Delta t = 283\ ps$ (đo được trên sóng). Trong khoảng này xuất hiện các giá trị tùy định (xx) và trung gian (x1)

Việc xuất hiện giá trị trung gian x1 là do các bit của ngõ ra Y được xử lý trong các khoảng thời gian khác nhau. Ví dụ, bit Y[0] được quy định bởi khối OAI21XLHVT, có thời gian trễ ngắn hơn nhiều so với bit Y[1], được quy định bởi khối MX2XLHVT, khối này có thời gian trễ rất cao. Hiện tượng 2 bit khác trọng số chuyển đổi không đồng thời đã tạo ra giá trị trung gian (x1) hiển thị trên dạng sóng

- **Tại t = 20 ns**



Hình 2.7: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 20 ns

Tại t = 20 ns = 20.000 ps, các tín hiệu đầu vào thay đổi: A chuyển từ 01 sang 11, B chuyển từ 00 sang 01, S vẫn giữ nguyên 00 (phép cộng). Ngõ ra Y mong muốn:

$$Y = A + B = 3_D(11)_B + 1_D(01)_B = 4_D(100)_B$$

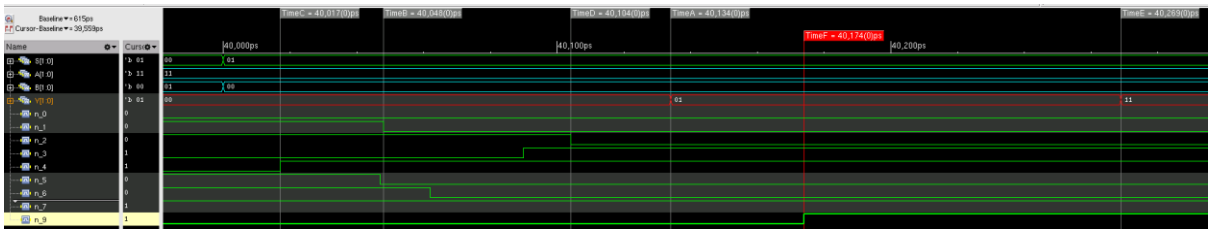
Phạm vi biểu diễn của Y chỉ có 2 bit, nên Y sẽ mang giá trị 00. Trên thực tế, ngõ ra Y không đổi giá trị ngay lập tức. Theo dạng sóng, Y thay đổi như sau:

- Từ 20,000 ps đến 20,166 ps: Y vẫn giữ nguyên giá trị cũ từ phép tính trước là 01
- Sau 20,166 ps: Y chuyển sang giá trị 00, là giá trị mà ta mong muốn đạt được.

Y không chuyển sang giá trị 00 ngay lập tức mà bị trễ một khoảng $\Delta t = 166\ ps$ (đo được trên sóng). Trong khoảng này không có giá trị trung gian

Giá trị trung gian không xuất hiện vì theo như lý thuyết, khi chuyển từ trường hợp 2 sang trường hợp 3, Y thay đổi từ 01 thành 00, chỉ có Y[0] thay đổi từ 1 thành 0, Y[1] vẫn giữ nguyên giá trị 0. Vì vậy ngõ ra Y chỉ thay đổi 1 lần cho biến Y[0], và từ đó ổn định giá trị đến trường hợp tiếp theo

- **Tại t = 40 ns**



Hình 2.8: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 40 ns

Tại t = 40 ns = 40.000 ps, các tín hiệu đầu vào thay đổi: B chuyển từ 01 sang 00, S từ 00 (phép cộng) sang 01 (phép trừ), A vẫn giữ nguyên giá trị 11. Kết quả ngõ ra Y mong muốn:

$$Y = A - B = 3_D(11)_B - 0_D(00)_B = 3_D(11)_B$$

Trên thực tế, ngõ ra Y không đổi giá trị ngay lập tức. Theo dạng sóng, Y thay đổi như sau:

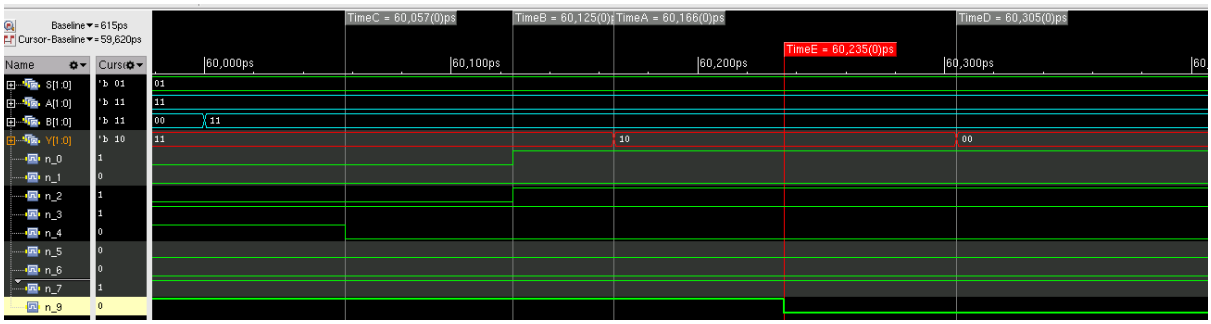
- Từ 40,000 ps đến 40,134 ps: Y vẫn giữ nguyên giá trị cũ từ phép tính trước là 00
- Từ 40,134 ps đến 40,269 ps: Y chuyển sang giá trị 01 (giá trị trung gian)
- Sau 40,269 ps: Y chuyển sang giá trị 11, là giá trị mà ta mong muốn đạt được.

Tương tự với trường hợp trên, ngõ ra Y xuất hiện giá trị trung gian trước khi chuyển sang kết quả đúng và ổn định.

Trong phép cộng/trừ, bit LSB Y[0] có thời gian tính toán nhanh hơn do không phụ thuộc vào bit nhớ hoặc bit mượn. Ngược lại, bit MSB Y[1] phải trải qua thêm một tầng logic tính toán bit nhớ/mượn nên thay đổi trạng thái chậm hơn.

Kết luận: Để thực hiện phép trừ này, ta cần tối thiểu 269 ps để ngõ ra Y có thể chuyển sang đúng giá trị và ổn định trước khi tiếp tục phép toán mới

- **Tại t = 60 ns**



Hình 2.9: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 60 ns

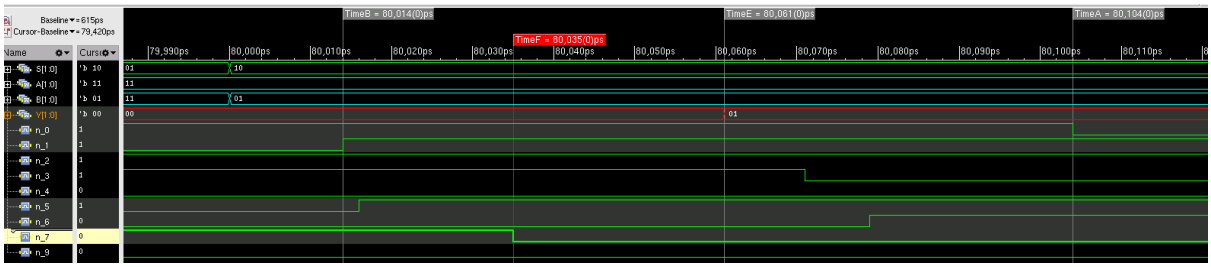
Tại t = 60 ns = 60,000 ps, chỉ có ngõ vào B thay đổi giá trị từ 00 thành 11, các ngõ vào khác vẫn giữ nguyên (S = 01 – phép trừ, A = 11). Kết quả Y mong muốn:

$$Y = A - B = 3_D(11)_B - 3_D(11)_B = 0_D(00)_B$$

Tương tự như trường hợp t = 40 ns, ngõ ra Y chỉ đúng và ổn định sau mốc thời gian 60,305 ps, trước đó Y mang giá trị trung gian (10, từ 60,160 đến 60,305 ps) và giá trị từ phép tính trước (11, trước mốc 60,160 ps). Như trường hợp trước, vì hiện tượng 2 bit khác trọng số chuyển đổi không đồng thời đã tạo ra giá trị trung gian

Kết luận: Để thực hiện phép trừ này, ta cần tối thiểu 305 ps để ngõ ra Y có thể trở về đúng giá trị và ổn định trước khi tiếp tục phép toán mới

- **Tại t = 80 ns**



Hình 2.10: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 80 ns

Tại t = 80 ns = 80,000 ps, các ngõ vào thay đổi: S chuyển từ phép trừ (01) sang phép AND (10), A vẫn giữ nguyên giá trị 11 và B thay đổi từ 11 sang 01. Kết quả Y mong muốn trong phép toán này:

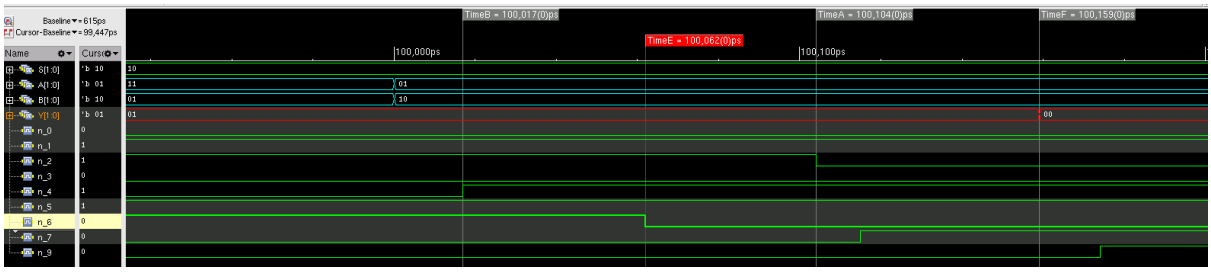
$$Y = A \,\&\, B = 11 \,\&\, 01 = 01$$

Thực tế theo dạng sóng, ngõ ra Y thay đổi như sau:

- Từ 80,000 ps đến 80,061 ps: Ngõ ra Y vẫn giữ giá trị từ phép toán trước là 00
- Sau 80,061 ps: Ngõ ra Y chuyển từ 00 thành 01, đúng theo mong muốn

Vì ngõ ra Y chỉ thay đổi tại biến Y[0], ngõ ra đã có thể đạt giá trị mong muốn và ổn định sau mốc 80,061 ps. Thực tế còn bước quy định Y[1], nhưng giá trị Y[1] vẫn giữ nguyên từ phép toán trước nên nhìn dạng sóng ta không thấy được bước này

- **Tại t = 100 ns**



Hình 2.11: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 100 ns

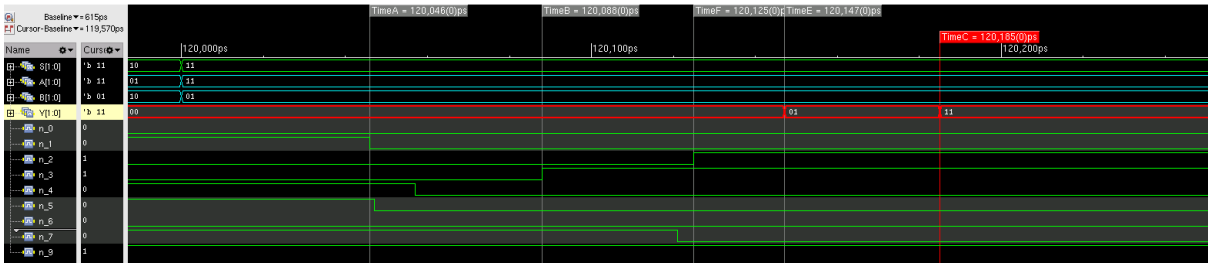
Trường hợp này có cả hai ngõ vào A và B cùng thay đổi tại mốc t = 100 ns. Cụ thể, A chuyển từ 11 thành 01, và B chuyển từ 01 thành 10. Ngõ ra Y mong muốn:

$$Y = A \,\&\, B = 01 \,\&\, 10 = 00$$

Thực tế, Y chỉ chuyển sang giá trị mong muốn sau mốc thời gian 100,159 ps. Trước mốc này, Y vẫn giữ giá trị từ phép tính trước (01)

Vẫn như trường hợp trước, Y[0] được quy định trước, Y[1] quy định sau, nhưng Y[1] vẫn giữ nguyên giá trị từ trường hợp trước nên ngõ ra đã có thể ổn định ngay khi Y[0] ổn định

- **Tại t = 120 ns**



Hình 2.12: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 120 ns

Tại t = 120 ns = 120,000 ps, cả 3 ngõ vào đều thay đổi: S chuyển từ 10 (phép AND) sang 11 (phép OR), A chuyển giá trị từ 01 thành 11, B chuyển từ 10 sang 01. Kết quả Y mong muốn:

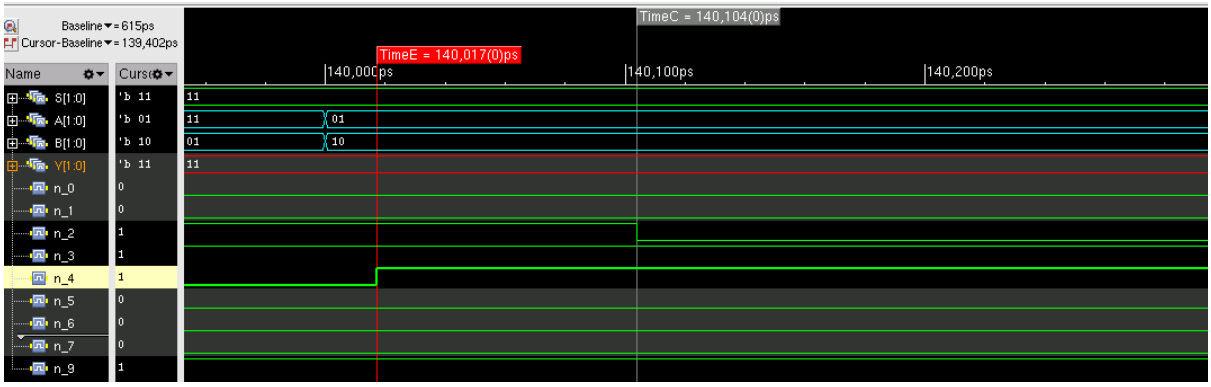
$$Y = A \mid B = 11 \mid 01 = 11$$

Theo dạng sóng, ngõ ra Y không thay đổi thành giá trị mong muốn ngay lập tức mà chuyển qua nhiều giá trị trước khi ổn định

- Từ 120,000 ps đến 120, 147 ps: Y vẫn giữ giá trị 00 từ phép tính trước
- Từ 120,147 ps đến 120,185 ps: Y chuyển sang giá trị trung gian 01
- Sau mốc 120,185 ps: Y chuyển sang giá trị 11, giá trị dự kiến, và ổn định

Cả hai bit Y[0] và Y[1] của ngõ ra đều thay đổi, vì vậy khoảng thời gian 185 ps là tối thiểu để ngõ ra chuyển thành giá trị mong muốn và ổn định.

- **Tại t = 140 ns**



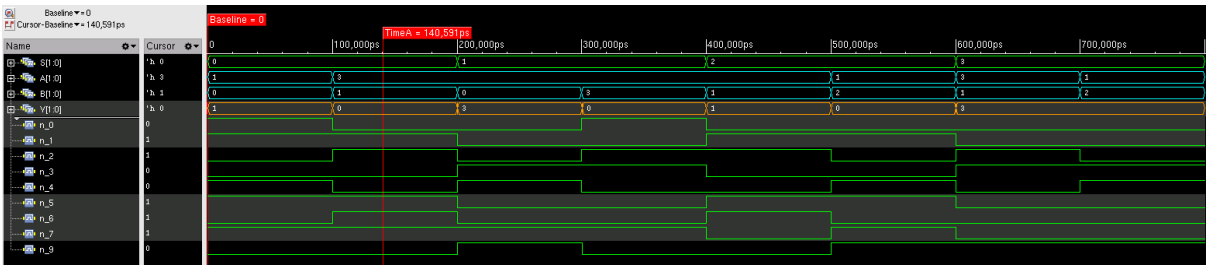
Hình 2.13: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 140 ns

Tại t = 140 ns = 140,000 ps, phép toán OR vẫn giữ nguyên, chỉ thay đổi 2 ngõ vào: A thành 01, và B thành 10. Ngõ ra Y mong muốn:

$$Y = A \mid B = 01 \mid 10 = 11$$

Vì kết quả Y mong muốn trùng với kết quả trước đó, nên quan sát dạng sóng vẫn thấy giá trị Y giữ kết quả là 11 trong suốt quá trình thực hiện phép toán.

c) Trường hợp T = 100 ns

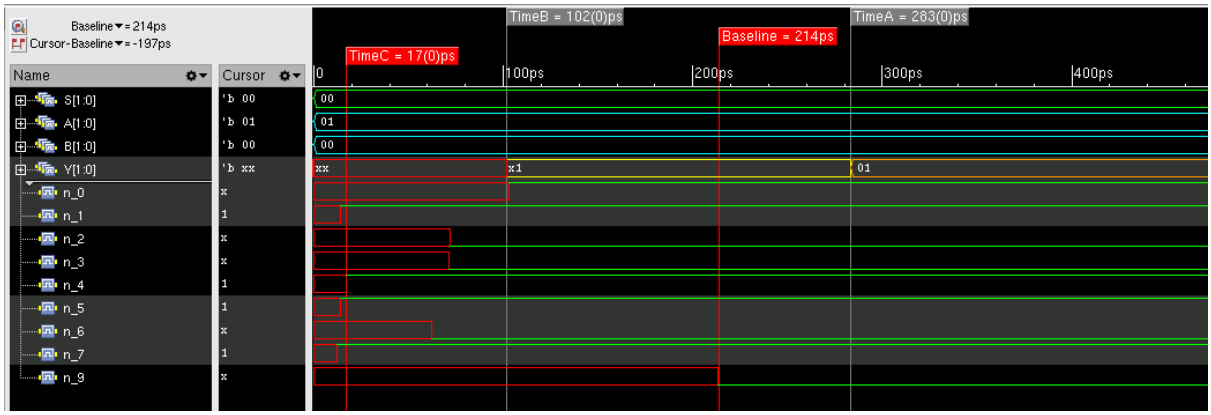


Hình 2.14: Tổng quan dạng sóng khi đặt T = 100 ns

Khi tăng T đến 100 ns, dạng sóng đã gần đạt đến dạng lý thuyết (không có delay), nhưng các giá trị trung gian vẫn còn hiện hữu nếu nhìn kỹ

Khi phân tích sâu vào từng trường hợp, ta thấy ngõ ra vẫn đạt giá trị mong muốn trễ hơn so với mốc thời gian thay đổi các ngõ vào

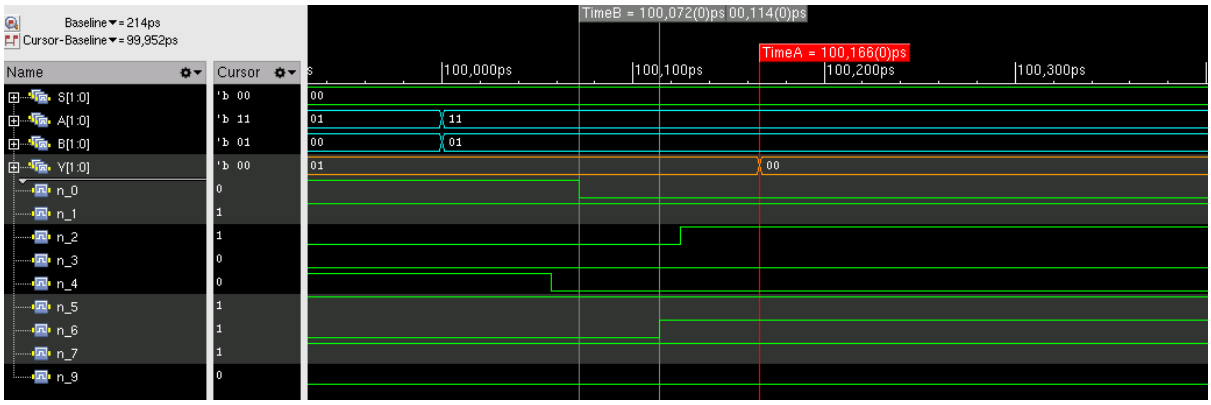
- **Tại t = 0 ns**



Hình 2.15: Dạng sóng tại giai đoạn khởi tạo.

Giai đoạn khởi tạo với T = 100 ns vẫn giống với giai đoạn khởi tạo với T = 20 ns, ngõ ra Y vẫn đạt giá trị mong muốn (01) sau $\Delta t = 283\ ps$ kể từ khi các ngõ vào được xác định

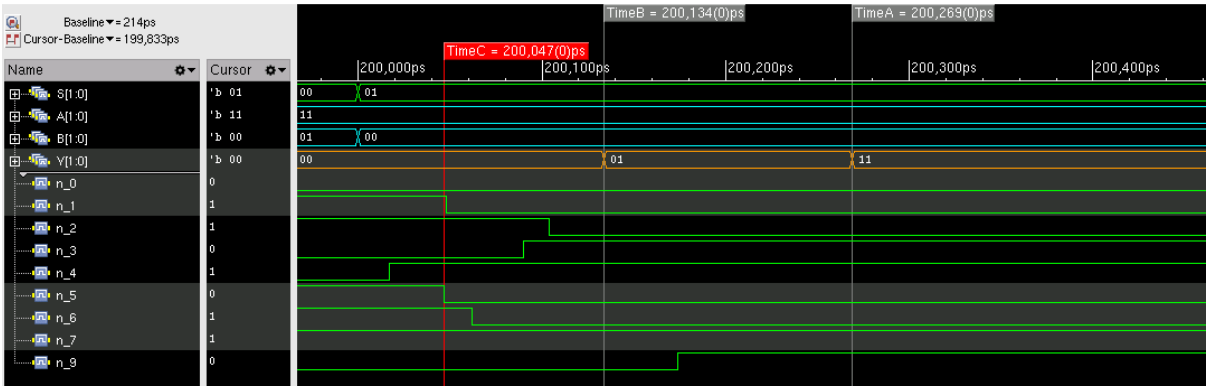
- **Tại t = 100 ns**



Hình 2.16: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 100 ns

Tương tự như trường hợp t = 20 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 166 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

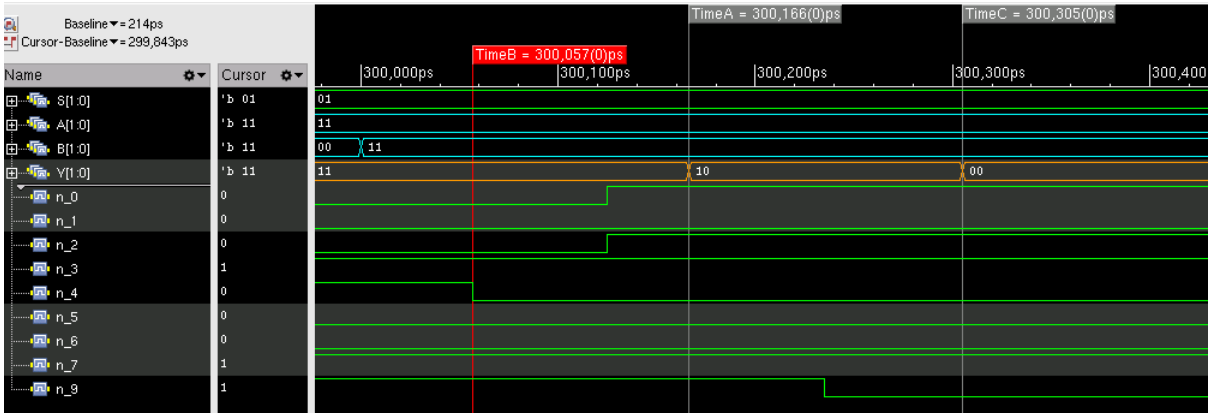
- **Tại t = 200 ns**



Hình 2.17: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 200 ns

Tương tự như trường hợp t = 40 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 269 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

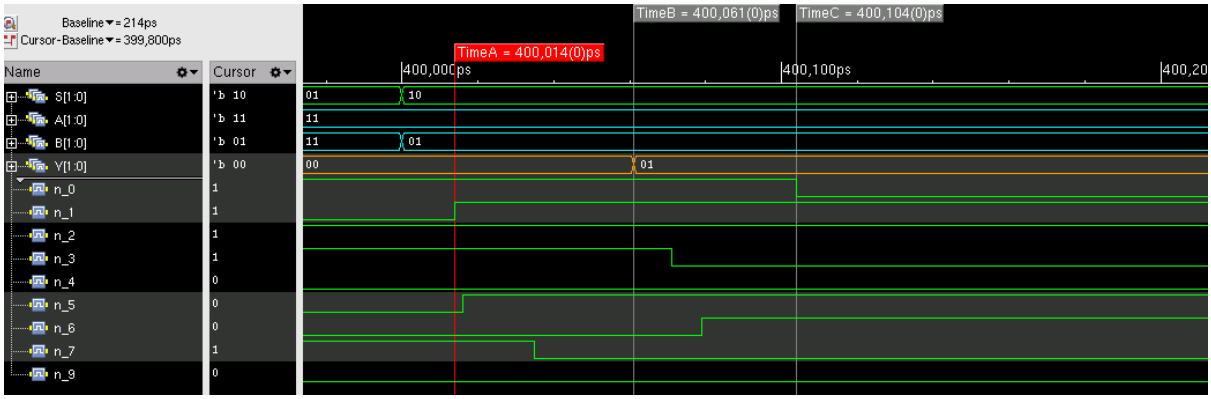
- **Tại t = 300 ns**



Hình 2.18: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 300 ns

Tương tự như trường hợp t = 60 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 305 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

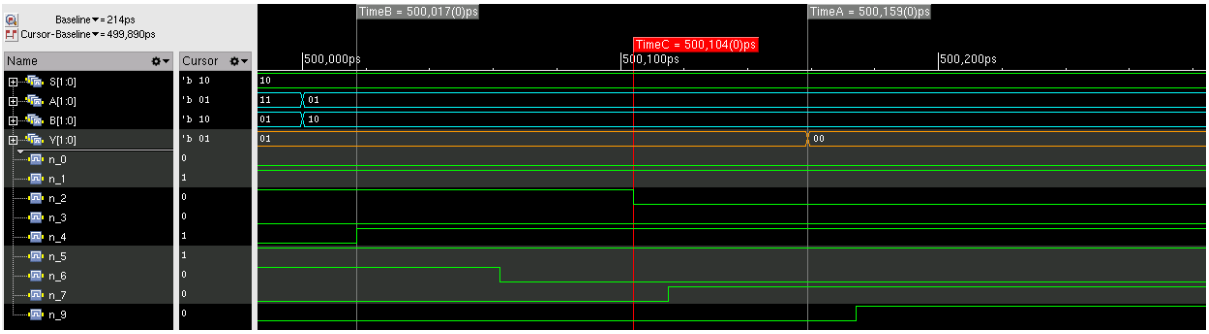
- **Tại t = 400 ns**



Hình 2.19: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 400 ns

Tương tự như trường hợp t = 80 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 61 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

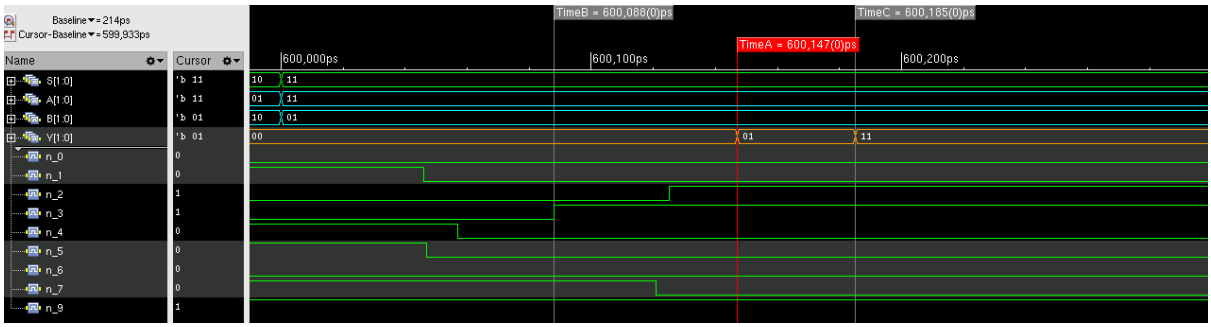
- **Tại t = 500 ns**



Hình 2.20: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 500 ns

Tương tự như trường hợp t = 100 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 159 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

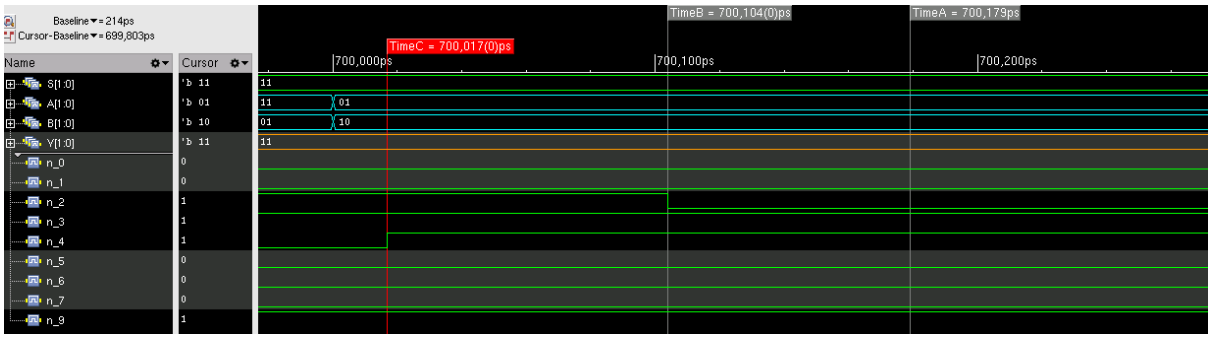
- **Tại t = 600 ns**



Hình 2.21: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 600 ns

Tương tự như trường hợp t = 120 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 185 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

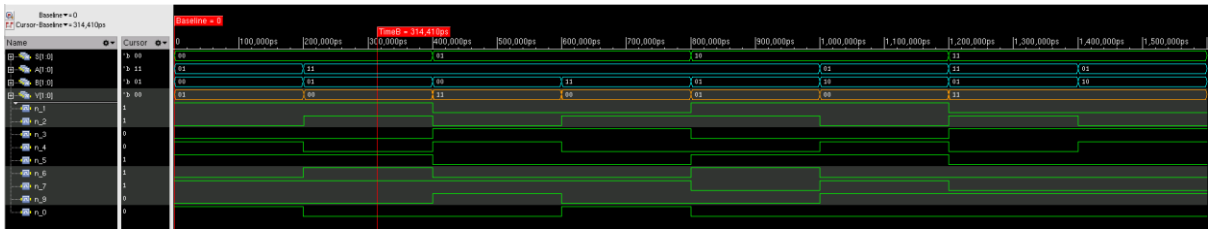
- **Tại t = 700 ns**



Hình 2.22: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 700 ns

Tương tự như trường hợp t = 140 ns khi T = 20 ns, ngõ ra Y vẫn giữ nguyên giá trị từ trường hợp trước đó. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

d) Trường hợp T = 200 ns

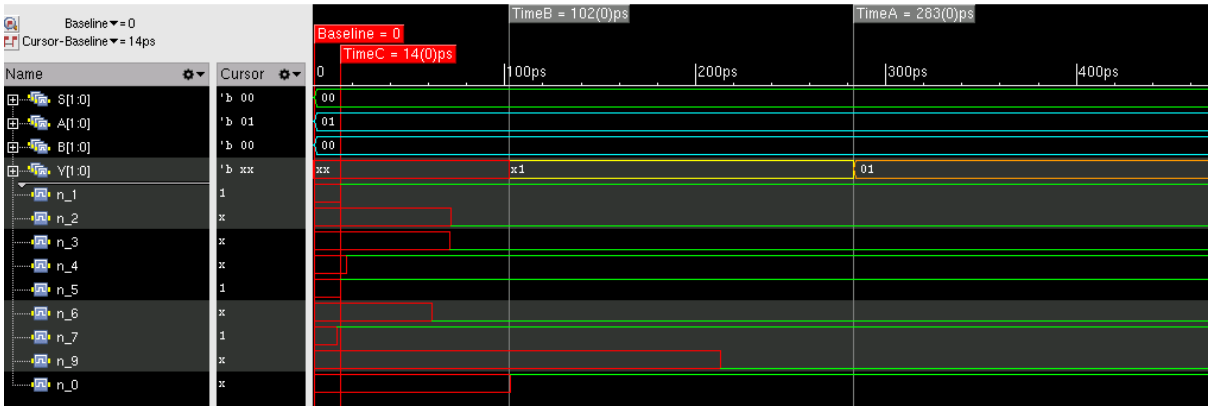


Hình 2.23: Tổng quan dạng sóng khi đặt T = 100 ns

Khi tăng T đến 200 ns, dạng sóng đã gần như lý thuyết (không có delay). Trên dạng sóng, thời gian cho các giá trị trung gian đã ngắn hơn rất nhiều, chỉ còn là một chấm nhỏ để thể hiện tại mốc thời gian đó có giá trị trung gian

Khi phân tích sâu vào từng trường hợp, ta thấy ngõ ra vẫn đạt giá trị mong muốn trễ hơn so với mốc thời gian thay đổi các ngõ vào

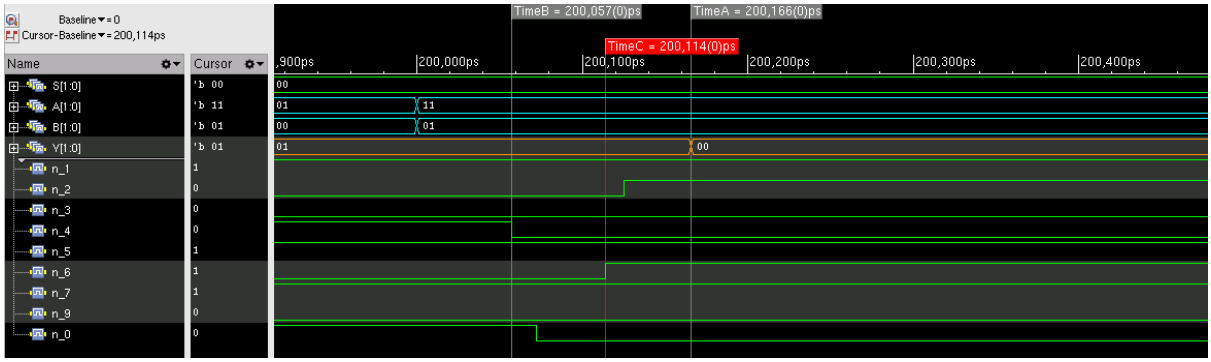
- **Tại t = 0 ns**



Hình 2.24: Dạng sóng tại giai đoạn khởi tạo.

Giai đoạn khởi tạo với T = 200 ns vẫn giống với giai đoạn khởi tạo với T = 20 ns, ngõ ra Y vẫn đạt giá trị mong muốn (01) sau $\Delta t = 283\ ps$ kể từ khi các ngõ vào được xác định

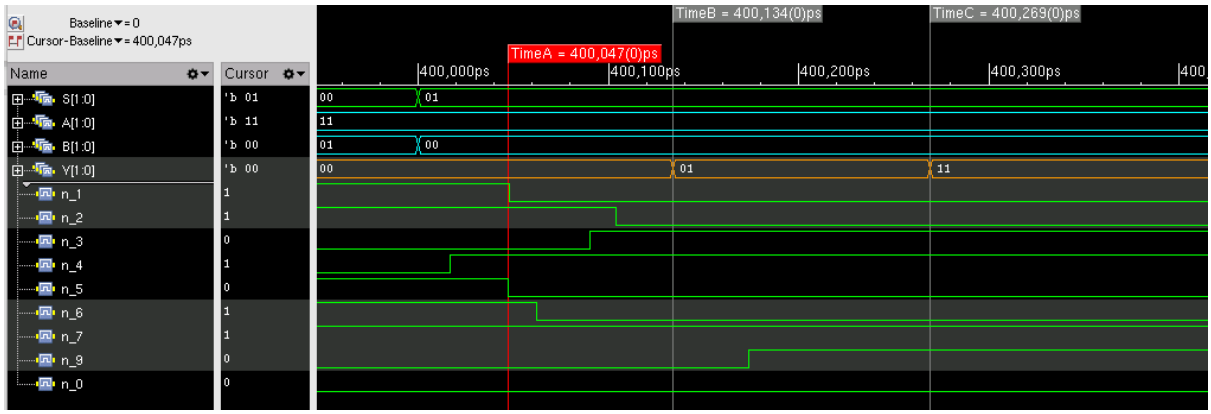
- **Tại t = 200 ns**



Hình 2.25: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 200 ns

Tương tự như trường hợp t = 20 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 166 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

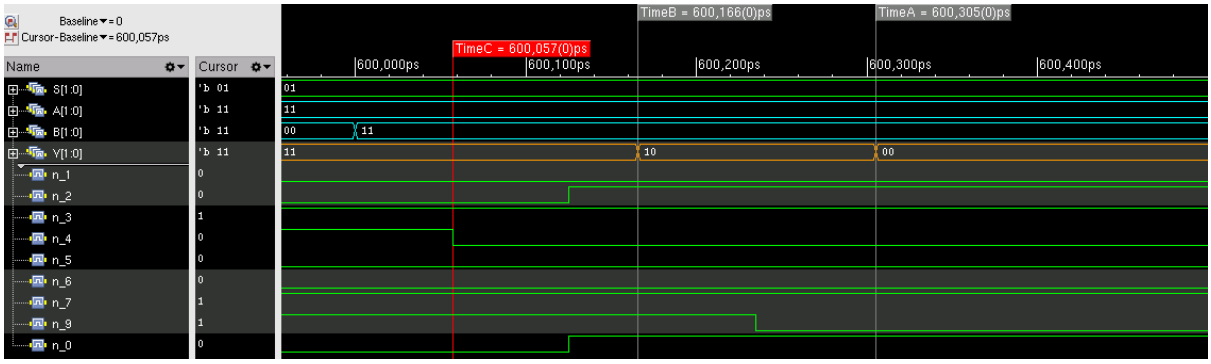
- **Tại t = 400 ns**



Hình 2.26: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 400 ns

Tương tự như trường hợp t = 40 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 269 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

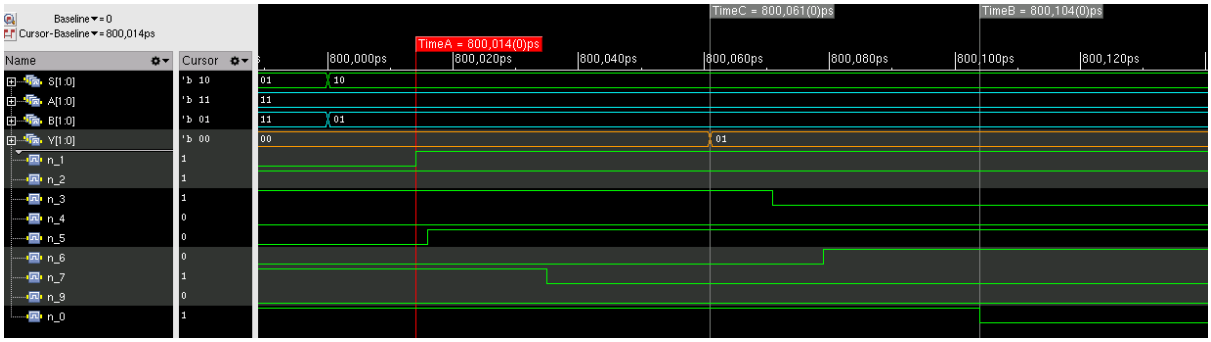
- **Tại t = 600 ns**



Hình 2.27: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 600 ns

Tương tự như trường hợp t = 60 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 305 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

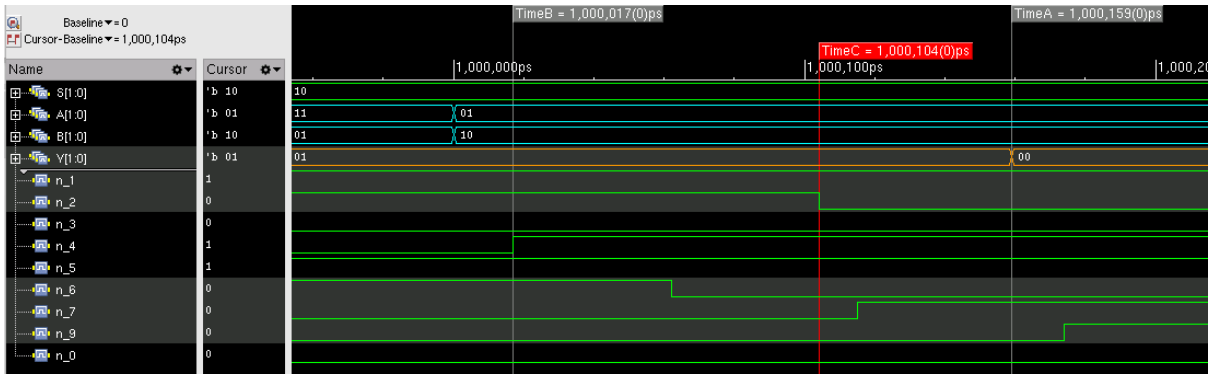
- **Tại t = 800 ns**



Hình 2.28: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 800 ns

Tương tự như trường hợp t = 80 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 61 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

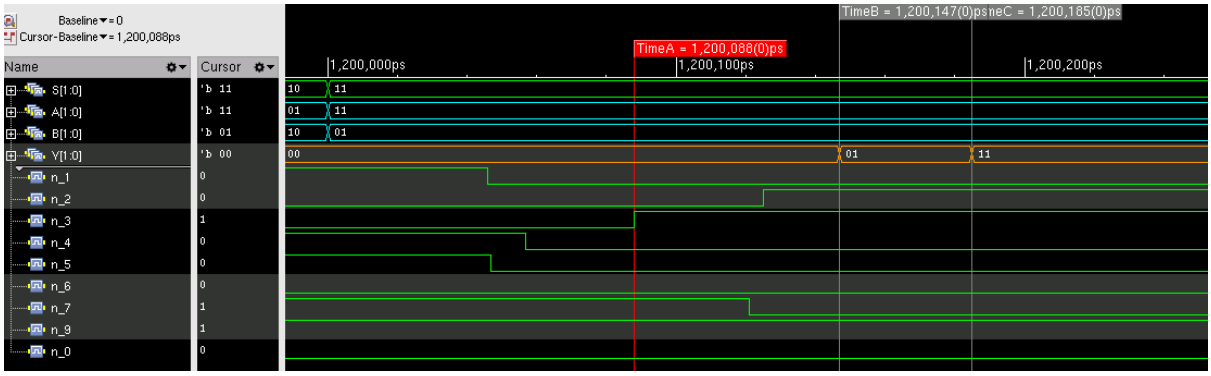
- **Tại t = 1000 ns**



Hình 2.29: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 1000 ns

Tương tự như trường hợp t = 100 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 159 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

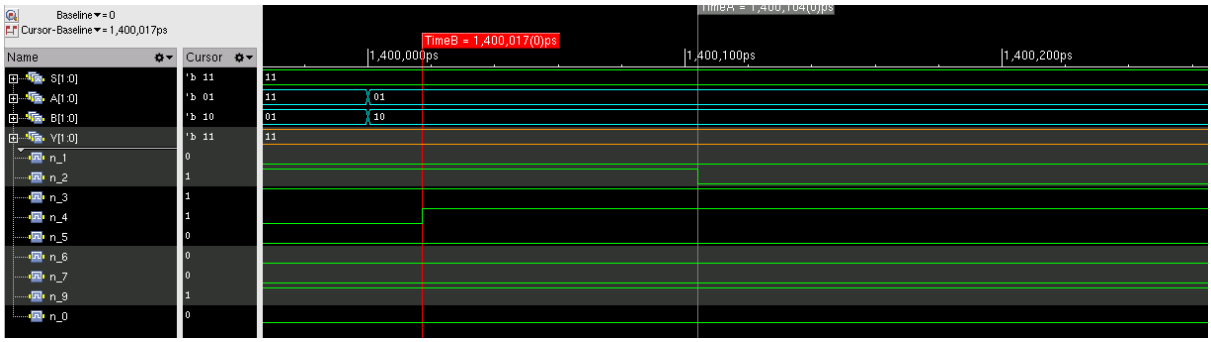
- **Tại t = 1200 ns**



Hình 2.30: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 1200 ns

Tương tự như trường hợp t = 120 ns khi T = 20 ns, ngõ ra Y ổn định sau mốc thời gian 185 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

- **Tại t = 1400 ns**



Hình 2.31: Trạng thái các ngõ vào/ra và các đường dây tại khoảng t = 1400 ns

Tương tự như trường hợp t = 140 ns khi T = 20 ns, ngõ ra Y vẫn giữ nguyên giá trị từ trường hợp trước đó. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

2.3.3. Tổng kết

a) Tổng hợp kết quả của bài kiểm thử

Bảng 2.2: Tổng hợp các kết quả kiểm thử

Trường hợp	Thời gian theo chu kỳ (ns)			S	A	B	Y (Dự kiến)	Thời gian trễ
	T = 20	T = 100	T = 200					
1	0 ns	0 ns	0 ns	00	01	00	01	283 ps
2	20 ns	100 ns	200 ns	00	11	01	00	166 ps
3	40 ns	200 ns	400 ns	01	11	00	11	269 ps
4	60 ns	300 ns	600 ns	01	11	11	00	305 ps
5	80 ns	400 ns	800 ns	10	11	01	01	61 ps
6	100 ns	500 ns	1000 ns	10	01	10	00	159 ps
7	120 ns	600 ns	1200 ns	11	11	01	11	185 ps
8	140 ns	700 ns	1400 ns	11	01	10	11	x

b) Độ trễ lan truyền (Propagation Delay):

Mạch ALU sau quá trình Synthesis được cấu thành từ mạng lưới các cổng logic mắc nối tiếp nhau, tạo nên nhiều tầng logic. Mỗi cổng logic cơ bản đều có độ trễ nhất định, khi tín hiệu lan truyền từ ngõ vào (A, B, S) đến ngõ ra, các độ trễ này tích lũy dọc theo đường truyền tín hiệu. Do đó, độ trễ tổng quan sát được tại ngõ ra chính là tổng thời gian trễ của các cổng logic nằm trên đường dẫn tới ngõ ra

c) Hiện tượng xuất hiện giá trị trung gian:

Tại một vài trường hợp, ngõ ra xuất hiện các giá trị sai tạm thời trước khi chuyển sang giá trị mong muốn. Nguyên nhân là do sự mất đồng bộ giữa các đường tín hiệu của các bit trong ngõ ra Y: Tín hiệu Y[0] chỉ được xác định thông qua 3 tầng logic, trong khi tín hiệu Y[1] đôi khi phải trải qua 4 tầng mới xác định được giá trị, do đó tốn nhiều thời gian hơn.

Hiện tượng các bit của ngõ ra không được xác định cùng lúc khiến mạch rơi vào trạng thái trung gian trong một khoảng thời gian ngắn trước khi tất cả tín hiệu được cập

nhật và ổn định. Đây cũng là lý do khi chỉ có Y[0] thay đổi, Y[1] giữ nguyên giá trị thì thời gian để ngõ ra ổn định sẽ ngắn hơn so với khi Y[1] thay đổi

d) Kết luận về tham số T:

Kết quả mô phỏng với 3 giá trị T (20 ns, 100 ns, 200 ns) cho thấy độ trễ lan truyền là một hằng số phụ thuộc vào công nghệ và schematic, không phụ thuộc vào chu kỳ T.

3. THÍ NGHIỆM 3

Ở thí nghiệm này chúng ta sẽ synthesis standard cell là D-FF để khảo sát các thông số "Area", "Power", "Delay"

1.1. Chuẩn bị các files cần thiết để chạy trên server

1.1.1. HDL file

```
module dff (
    input  logic clk,
    input  logic d,
    output logic q
);

always_ff @(posedge clk) begin
    q <= d;
end

endmodule
```

1.1.2. TCL files

Với mục tiêu bài thí nghiệm là khảo sát hết 8 library corner, nên nhóm sẽ chuẩn bị 8 file TCL tương tự nhau, chỉ thay đổi tên thư viện, đường dẫn thư mục chứa các file reports và outputs tương ứng mỗi library.

Dưới đây là file TCL của *fast_vdd1v0_basicCells_hvt.lib*, các thư viện còn lại làm tương tự.

```
# synth_ff_hvt_1v0.tcl
# Synthesis script for Fast corner, VDD=1.0V, HVT

set_db init_lib_search_path {/home/yellow/ee3201_16/LAB2/gpdk045_lib}
set_db init_hdl_search_path {/home/yellow/ee3201_16/LAB2/Exp3/00_src}

read_libs fast_vdd1v0_basicCells_hvt.lib
read_hdl -sv dff.sv

elaborate

set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium

syn_generic
syn_map
```

```
report_timing > ./02_reports/ff_hvt_1v0/timing.rpt
report_power  > ./02_reports/ff_hvt_1v0/power.rpt
report_area   > ./02_reports/ff_hvt_1v0/area.rpt
report_qor    > ./02_reports/ff_hvt_1v0/qor.rpt

write_hdl > ./03_outputs/ff_hvt_1v0/dff_netlist.sv

write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -
setuphold split > ./03_outputs/ff_hvt_1v0/delay.sdf
```

Lưu ý:

Vì cấu trúc thư mục project của nhóm, lệnh chạy file từng file TCL trên Genus sẽ như sau:

```
genus -f 04_scripts/<lib corner.tcl> -log log/<lib_folder>/genus.log
```

Trong đó:

- <lib corner.tcl>: Tên file TCL của từng corner
- <lib_folder>: Tên folder chứa file log của từng corner

1.1.3. Testbench file

```
`timescale 1ns/1ps

//=========================================
// Select corner using compiler define
//=========================================
`ifdef FF_HVT_1V0
    `define CORNER_NAME "FF HVT 1.0V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/fast_vdd1v0_basicCells_hvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_hvt_1v0/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_hvt_1v0/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_hvt_1v0/dff_wave.vcd"
    `define T_SETUP_RISE 0.025
    `define T_SETUP_FALL 0.019
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.000
    `define T_CQ_RISE    0.051
    `define T_CQ_FALL    0.058

`elsif FF_HVT_1V2
    `define CORNER_NAME "FF HVT 1.2V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/fast_vdd1v2_basicCells_hvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_hvt_1v2/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_hvt_1v2/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_hvt_1v2/dff_wave.vcd"
    `define T_SETUP_RISE 0.016
    `define T_SETUP_FALL 0.012
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.000
    `define T_CQ_RISE    0.037
    `define T_CQ_FALL    0.042

`elsif SS_HVT_1V0
    `define CORNER_NAME "SS HVT 1.0V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/slow_vdd1v0_basicCells_hvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_hvt_1v0/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_hvt_1v0/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_hvt_1v0/dff_wave.vcd"
    `define T_SETUP_RISE 0.177
    `define T_SETUP_FALL 0.058
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.051
    `define T_CQ_RISE    0.243
    `define T_CQ_FALL    0.292

`elsif SS_HVT_1V2
    `define CORNER_NAME "SS HVT 1.2V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/slow_vdd1v2_basicCells_hvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_hvt_1v2/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_hvt_1v2/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_hvt_1v2/dff_wave.vcd"
    `define T_SETUP_RISE 0.072
    `define T_SETUP_FALL 0.026
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.018
    `define T_CQ_RISE    0.115
    `define T_CQ_FALL    0.135
```

```
`elsif FF_LVT_1V0
    `define CORNER_NAME "FF LVT 1.0V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/fast_vdd1v0_basicCells_lvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_lvt_1v0/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_lvt_1v0/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_lvt_1v0/dff_wave.vcd"
    `define T_SETUP_RISE 0.013
    `define T_SETUP_FALL 0.010
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.000
    `define T_CQ_RISE    0.029
    `define T_CQ_FALL    0.033

`elsif FF_LVT_1V2
    `define CORNER_NAME "FF LVT 1.2V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/fast_vdd1v2_basicCells_lvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_lvt_1v2/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_lvt_1v2/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ff_lvt_1v2/dff_wave.vcd"
    `define T_SETUP_RISE 0.010
    `define T_SETUP_FALL 0.007
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.000
    `define T_CQ_RISE    0.024
    `define T_CQ_FALL    0.027

`elsif SS_LVT_1V0
    `define CORNER_NAME "SS LVT 1.0V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/slow_vdd1v0_basicCells_lvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_lvt_1v0/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_lvt_1v0/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_lvt_1v0/dff_wave.vcd"
    `define T_SETUP_RISE 0.061
    `define T_SETUP_FALL 0.029
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.014
    `define T_CQ_RISE    0.106
    `define T_CQ_FALL    0.121

`elsif SS_LVT_1V2
    `define CORNER_NAME "SS LVT 1.2V"
    `define LIB_PATH "/home/yellow/ee3201_16/LAB2/gpdk045_verilog/slow_vdd1v2_basicCells_lvt.v"
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_lvt_1v2/dff_netlist.sv"
    `define SDF_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_lvt_1v2/delay.sdf"
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/ss_lvt_1v2/dff_wave.vcd"
    `define T_SETUP_RISE 0.036
    `define T_SETUP_FALL 0.016
    `define T_HOLD_RISE  0.000
    `define T_HOLD_FALL  0.005
    `define T_CQ_RISE    0.068
    `define T_CQ_FALL    0.078
```

```
`else
    `define CORNER_NAME "RTL (No Corner Selected)"
    `define LIB_PATH ""
    `define NETLIST_PATH "/home/yellow/ee3201_16/LAB2/Exp3/00_src/dff.sv"
    `define SDF_PATH ""
    `define VCD_PATH "/home/yellow/ee3201_16/LAB2/Exp3/03_outputs/rtl_sim.vcd"
    `define T_SETUP_RISE 0.0
    `define T_SETUP_FALL 0.0
    `define T_HOLD_RISE  0.0
    `define T_HOLD_FALL  0.0
    `define T_CQ_RISE    0.0
    `define T_CQ_FALL    0.0
`endif

//=========================================
// Include files based on corner
//=========================================
`ifdef GATE_LEVEL
    `include `NETLIST_PATH
    `include `LIB_PATH
`else
    `include "/home/yellow/ee3201_16/LAB2/Exp3/00_src/dff.sv"
`endif
```

```
module tb_dff;

    //=========================================
    // Signal Declaration
    //=========================================
    logic clk;
    logic d;
    logic q;

    //=========================================
    // Convert defines to parameters
    //=========================================
    parameter real CLK_PERIOD    = 1.0;   // T=1ns => f=1GHz
    parameter real T_SETUP_RISE  = `T_SETUP_RISE;
    parameter real T_SETUP_FALL  = `T_SETUP_FALL;
    parameter real T_HOLD_RISE   = `T_HOLD_RISE;
    parameter real T_HOLD_FALL   = `T_HOLD_FALL;
    parameter real T_CQ_RISE     = `T_CQ_RISE;
    parameter real T_CQ_FALL     = `T_CQ_FALL;

    // Calculate test margins automatically based on timing
    parameter real MARGIN_SAFE      = 0.300;            //Cần chỉnh sửa chỗ này
    parameter real MARGIN_VIOLATION = 0.003;

    //=========================================
    // DUT Instantiation
    //=========================================
    dff DUT (
        .clk(clk),
        .d(d),
        .q(q)
    );
```

```
//=========================================
// SDF Annotation
//=========================================
`ifdef ANNOTATION
initial begin
    $sdf_annotate(`SDF_PATH, DUT, , "annotate.log", "MAXIMUM");
    $display("========================================");
    $display("SDF Annotation Enabled");
    $display("File: %s", `SDF_PATH);
    $display("========================================");
end
`endif

//=========================================
// Clock Generation
//=========================================
initial begin
    clk = 0;
    forever #(CLK_PERIOD/2) clk = ~clk;
end

//=========================================
// Waveform Dump
//=========================================
initial begin
    $dumpfile(`VCD_PATH);
    $dumpvars(0, tb_dff.DUT);
end

//=========================================
// Test Stimulus
//=========================================
initial begin
    $display("\n========================================");
    $display("D-FF SETUP/HOLD TIME VERIFICATION");
    $display("Corner: %s", `CORNER_NAME);
    $display("========================================");
    $display("Clock Period     : %.3f ns", CLK_PERIOD);
    $display("Setup Time (D↑)  : %.3f ns", T_SETUP_RISE);
    $display("Setup Time (D↓)  : %.3f ns", T_SETUP_FALL);
    $display("Hold Time (D↑)   : %.3f ns", T_HOLD_RISE);
    $display("Hold Time (D↓)   : %.3f ns", T_HOLD_FALL);
    $display("Clock-to-Q (Q↑)  : %.3f ns", T_CQ_RISE);
    $display("Clock-to-Q (Q↓)  : %.3f ns", T_CQ_FALL);
    $display("========================================\n");

    d = 0;
    #(CLK_PERIOD * 3);
```

```
//=========================================
// TEST 1: NORMAL OPERATION
//=========================================
$display(">>> TEST 1: NORMAL OPERATION");
$display("    D arrives %.3fns before clock edge", MARGIN_SAFE);

@(posedge clk);
#(CLK_PERIOD - MARGIN_SAFE);
d = 1;
#(CLK_PERIOD * 1.5);

@(posedge clk);
#(CLK_PERIOD - MARGIN_SAFE);
d = 0;
#(CLK_PERIOD * 1.5);
```

```
//=========================================
// TEST 2: SETUP VIOLATION
//=========================================
$display("\n>>> TEST 2: SETUP VIOLATION");

$display("    D↑ arrives %.3fns before clock (required: %.3fns)",
         T_SETUP_RISE - MARGIN_VIOLATION, T_SETUP_RISE);

@(posedge clk);
#(CLK_PERIOD - T_SETUP_RISE + MARGIN_VIOLATION);
d = 1;
#(CLK_PERIOD * 2);

$display("    D↓ arrives %.3fns before clock (required: %.3fns)",
         T_SETUP_FALL - MARGIN_VIOLATION, T_SETUP_FALL);

@(posedge clk);
#(CLK_PERIOD - T_SETUP_FALL + MARGIN_VIOLATION);
d = 0;
#(CLK_PERIOD * 2);
```

```
//==========================================
// TEST 3: HOLD VIOLATION (if applicable)
//==========================================
if (T_HOLD_RISE > 0.000 || T_HOLD_FALL > 0.000) begin
    $display("\n>>> TEST 3: HOLD VIOLATION");
    $display("    D changes %.3fns after clock (required: %.3fns stable)",
             MARGIN_VIOLATION, T_HOLD_FALL);

    @(posedge clk);
    #(CLK_PERIOD - MARGIN_SAFE);        //Cần coi lại mấy chỗ này
    d = 1;

    @(posedge clk);
    #(MARGIN_VIOLATION);  // Change before hold time expires
    d = 0;
    #(CLK_PERIOD * 2);

end else begin
    $display("\n>>> TEST 3: HOLD VIOLATION - SKIPPED");
    $display("    Hold time = 0ns for this corner");
end
```

```
//==========================================
// TEST 4: EDGE CASES
//==========================================
    $display("\n>>> TEST 4: BOUNDARY CONDITIONS");
    $display("    Testing D change EXACTLY at setup time limit");

// D↑ exactly at setup time
    @(posedge clk);
    #(CLK_PERIOD - T_SETUP_RISE);
    d = 1;
    #(CLK_PERIOD * 2);

// D↓ exactly at setup time
    @(posedge clk);
    #(CLK_PERIOD - T_SETUP_FALL);
    d = 0;
    #(CLK_PERIOD * 2);

    //==========================================
    // End
    //==========================================
    #(CLK_PERIOD * 2);
    $display("\n==========================================");
    $display("SIMULATION COMPLETE");
    $display("==========================================\n");
    $finish;
end

initial begin
    $monitor("Time=%0t ns | clk=%b | d=%b | q=%b", $realtime, clk, d, q);
end
```

Tại đây nhóm sẽ không viết 8 testbench cho từng corner, thay vào đó nhóm sẽ viết 1 testbench duy nhất kết hợp với ‘define macro để trỏ tới đúng corner sẽ test. Vì vậy, câu lệnh chạy testbench trên Xcelium và xem waveform trên Simvision sẽ có cấu trúc:

```
xrun -define GATE_LEVEL -define ANNOTATION -define <CORNER_NAME> +accesss+rw
01_tb/tb_dff.sv
```

```
simvision 03_outputs/<lib_folder>/dff_wave.vcd &
```

Trong đó:

- <CORNER_NAME>: Tên corner sẽ test
- <lib_folder>: Tên folder chứa file dff_wave.vcd của từng corner

1.2. Kết quả khi chạy tool Genus trên server

1.2.1. Kết quả Area và Power

Bảng 3.1: Kết quả Area và Power của DFF ở từng corner

Corner	Process	VDD	Type Vt	Area	Power (W)
fast_vdd1v0_hvt	FF	1.0V	HVT	5.472	5.72696e-08
fast_vdd1v2_hvt	FF	1.2V	HVT	5.472	8.44054e-08
slow_vdd1v0_hvt	SS	1.0V	HVT	5.472	4.93175e-08
slow_vdd1v2_hvt	SS	1.2V	HVT	5.472	7.49199e-08
fast_vdd1v0_lvt	FF	1.0V	LVT	5.472	7.23510e-08
fast_vdd1v2_lvt	FF	1.2V	LVT	5.472	1.04679e-07
slow_vdd1v0_lvt	SS	1.0V	LVT	5.472	5.75687e-08

slow_vdd1v2_lvt	SS	1.2V	LVT	5.472	8.60509e-08

Từ các giá trị thu được trong bảng, ta có vài nhận xét sau:

- Area giống nhau ở tất cả 8 corner (5.472), không phụ thuộc vào process, v_{DD}, hay threshold voltage. Có thể Area phụ thuộc vào physical layout (kích thước transistor, routing, metal layers) trong khi các library khác nhau về timing, delay, power.

- Power phụ thuộc mạnh vào VDD vào Vt:

- **Ảnh hưởng của VDD**

Bảng 3.2: Ảnh hưởng của VDD lên Power từng corner

Process-Vt	1.0V Power (nW)	1.2V Power (nW)	Tăng (%)
FF-HVT	57.3	84.4	+47.3%
SS-HVT	49.3	74.9	+52.0%
FF-LVT	72.4	104.7	+44.6%
SS-LVT	57.6	86.1	+49.5%
Trung bình	-	-	~48.4%

=> VDD tăng khoảng 20% thì Power tăng khoảng 48%

- **Ảnh hưởng của Vt**

Bảng 3.3: Ảnh hưởng của Vt lên Power từng corner

Process-VDD	HVT Power (nW)	LVT Power (nW)	Tăng (%)
FF-1.0V	57.3	72.4	+26.4%
FF-1.2V	84.4	104.7	+24.1%
SS-1.0V	49.3	57.6	+16.8%
SS-1.2V	74.9	86.1	+15.0%
Trung bình	-	-	~20.6%

=> Power của LVT tăng khoảng 21% so với HVT

- **Ảnh hưởng của Process**

Bảng 3.4: Ảnh hưởng của Process lên Power từng corner

Vt-VDD	FF Power (nW)	SS Power (nW)	Chênh lệch (%)
HVT-1.0V	57.3	49.3	-14.0%

HVT-1.2V	84.4	74.9	-11.3%
LVT-1.0V	72.4	57.6	-20.4%
LVT-1.2V	104.7	86.1	-17.8%
Trung bình	-	-	~ -15.9%

=> Fast corner (FF) lại có Power lớn hơn khoảng 16% so với Slow corner (SS)

1.2.2. Kết quả Delay

Bảng 3.5: Kết quả Delay của DFF ở từng corner

	SETUP (D↑) ns	SETUP (D↓) ns	HOLD (D↑) ns	HOLD (D↓) ns	C-Q (Q↑) ns	C-Q (Q↓) ns
fast_vdd1v0_basicCells_hvt.lib	0.025	0.019	0.000	0.000	0.051	0.058
fast_vdd1v2_basicCells_hvt.lib	0.016	0.012	0.000	0.000	0.037	0.042
slow_vdd1v0_basicCells_hvt.lib	0.177	0.058	0.000	0.051	0.243	0.292
slow_vdd1v2_basicCells_hvt.lib	0.072	0.026	0.000	0.018	0.115	0.135
fast_vdd1v0_basicCells_lvt.lib	0.013	0.010	0.000	0.000	0.029	0.033
fast_vdd1v2_basicCells_lvt.lib	0.010	0.007	0.000	0.000	0.024	0.027
slow_vdd1v0_basicCells_lvt.lib	0.061	0.029	0.000	0.014	0.106	0.121
slow_vdd1v2_basicCells_lvt.lib	0.036	0.016	0.000	0.005	0.068	0.078

Dựa vào bảng trên ta có vài nhận xét:

- Về t_{c-q}: Process ảnh hưởng lớn, Slow luôn có thời gian delay lớn hơn Fast (Xét cùng VDD, Vt).

- Về t_{setup}: Set up time low-to-high luôn lớn hơn high-to-low.

- Về t_{hold}: Fast corner không có hold violation (t_{hold}=0ns). Slow corner chỉ có violation khi high-to-low => Hold violations chỉ cần check ở slow corners và cạnh xuống của D.

1.3. Chạy testbench và kết luận

1.3.1. Testcases

Với yêu cầu kiểm định các giá trị set up time và hold time của D-FF trong từng library corner, nhóm chọn chiến lược viết một testbench duy nhất là *tb_dff.sv* để chạy

cho cả 8 corner. Để làm được được như vậy, nhóm thêm định nghĩa các đường dẫn và giá trị t_{setup}, t_{hold} của từng corner, sau đó macro đến từng corner bằng lệnh sau:

```
xrun -define GATE_LEVEL -define ANNOTATION -define <FF_HVT_1V0> +accesss+rw
01_tb/tb_dff.sv
```

Trong đó: `<FF_HVT_1V0>` là tên thư viện sẽ macro đến

Quan sát giá trị t_{setup}, t_{hold} thu được ở trên, nhóm nhận thấy giá trị nhỏ nhất (khác 0ns) của chúng là 5ps và giá trị lớn nhất là 177ps. Vì vậy nhóm sẽ đặt MARGIN_SAFE = 300ps và MARGIN_VIOLATION = 3ps là các biến an toàn, vi phạm nhóm dùng trong các testcase dưới đây

- **TESTCASE 1: NORMAL OPERATION**

Ở đây ta sẽ quan sát giá trị Q của D-FF trong trường hợp ngõ vào D được giữ ổn định trong 300ps (nhỏ hơn tất cả t_{setup}) trước clock cạnh lên. Đồng thời kiểm tra các giá trị t_{c-q} có đúng với lý thuyết.

Kết quả kì vọng của testcase trên là Q sẽ có giá trị thay đổi theo D sau một khoảng thời gian là t_{c-q} sau clock cạnh lên.

- **TESTCASE 2: SETUP VIOLATION**

Ở đây ta sẽ khảo sát set up time của D-FF trong cả 2 trường hợp khi D thay đổi từ mức cao lên mức thấp và ngược lại, bằng cách thay đổi giá trị D trước khi có clock cạnh lên một khoảng thời gian t_{setup} – 3ps (D không giữ ổn định trong set up time).

Kết quả kì vọng ngõ ra Q sẽ không xét định (X) vì sampling sai.

- **TESTCASE 3: HOLD VIOLATION**

Vì hold vioaltion chỉ xảy ra khi D chuyển từ mức cao xuống mức thấp đối với những Slow corner, nên nhóm sẽ kẹp điều kiện (T_HOLD_RISE > 0.000 || T_HOLD_FALL > 0.000), nếu không thỏa sẽ không hiện wave của hold violation.

Tương tự setup violation, ta sẽ thay đổi giá trị D sau clock cạnh lên sau 3ps (D không ổn định trước khi hết hold time).

Kết quả kì vọng tương tự setup violation.

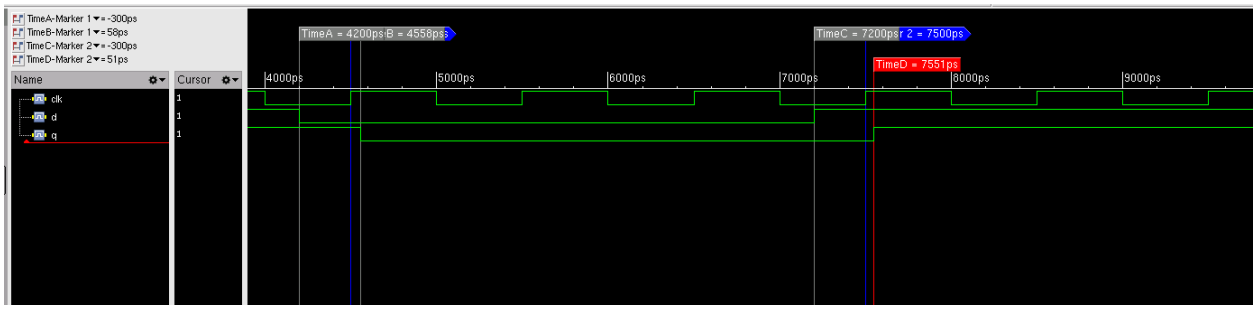
❖ TESTCASE 4: EDGE CASES

Testcase này tương tự testcase 1, chỉ khác rằng ta sẽ thay đổi D trước clock cạnh lên một khoảng thời gian bằng với t_{setup}.

1.3.2. Chạy trên Xcelium và phân tích dạng sóng trên Simvision

a. fast_vdd1v0_basicCells_hvt.lib

- ***Case 1: Normal operation***

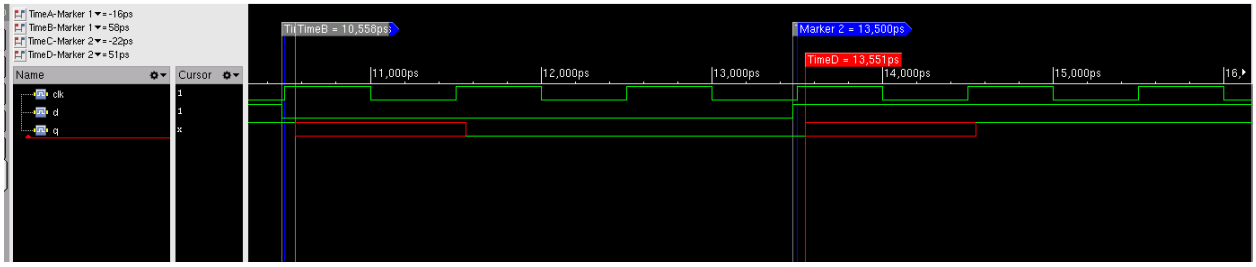


Hình 3.1: Case 1 Normal operation

D thay đổi trước clock cạnh lên 300ps (>t_{setup}) và được giữ ổn định hết t_{hold}, sau đó Q có giá trị bằng D sau lần lượt 58ps (high-to-low) và 51ps (low-to-high).

=> Có giá trị Q bình thường sau khi bị delay

- ***Case 2: Setup violation***

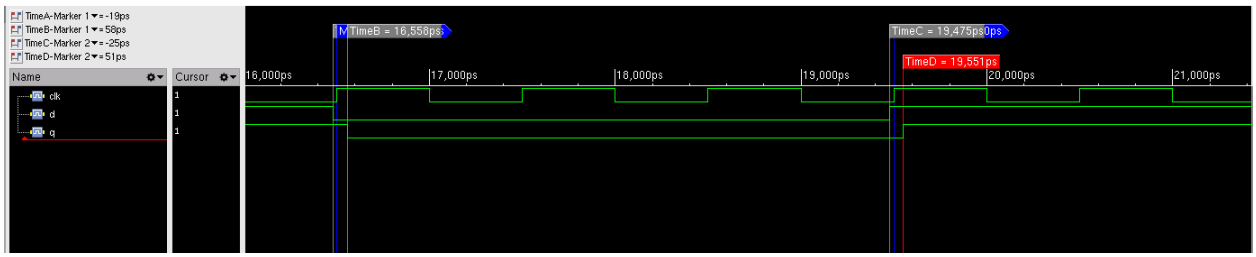


Hình 3.2: Case 2 Setup Violation

D thay đổi từ mức cao xuống mức thấp trước 16ps (<t_{setup}↓=19ps) và từ mức thấp lên mức cao trước 22ps (<t_{setup}↑=25ps)

=> Không xác định được ngõ ra Q

- ***Case 4: Boundary condition***



Hình 3.3: Case 4 Boundary condition

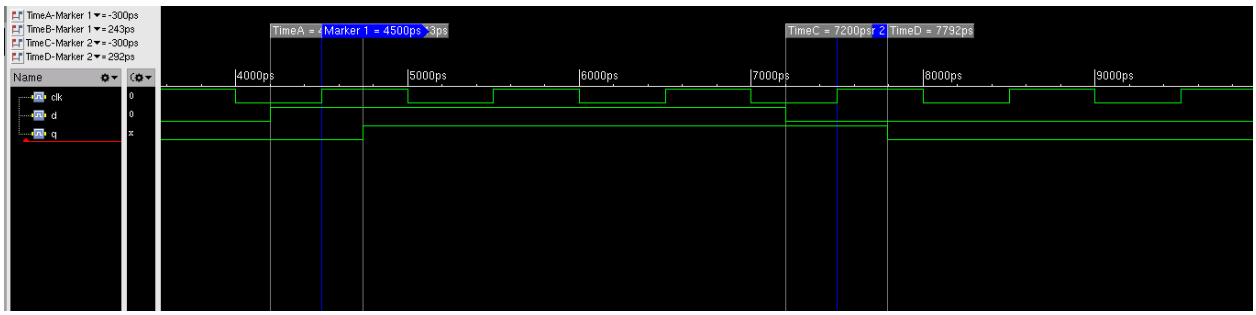
D thay đổi trước clock cạnh lên ngay khoảng thời gian bằng $t_{setup}\downarrow$=19ps và $t_{setup}\uparrow$=25ps

=> Kết quả thu được giống với case 1

b. slow_vdd1v0_basicCells_hvt.lib

Đặc điểm của Process Slow (SS) có t hold khi D thay đổi từ mức cao xuống mức thấp sau clock cạnh lên

- ***Case 1: Normal operation***

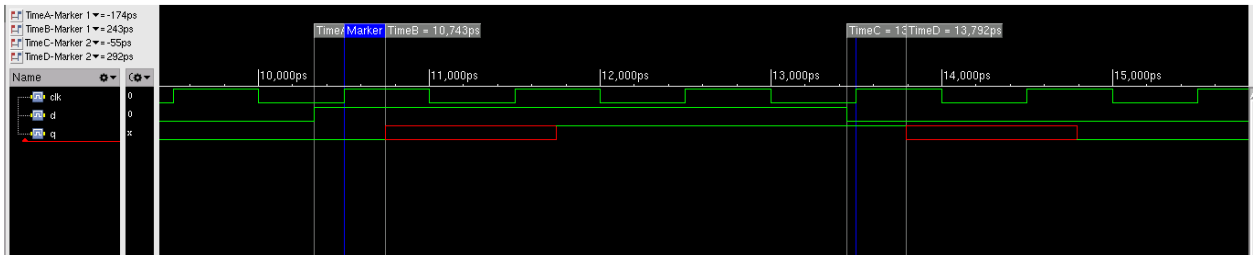


Hình 3.4: Case 1 Normal operation

D thay đổi trước clock cạnh lên 300ps (>t_{setup}) và được giữ ổn định hết t_{hold}, sau đó Q có giá trị bằng D sau lần lượt 292ps (high-to-low) và 243ps (low-to-high).

=> Có giá trị Q bình thường sau khi bị delay

- ***Case 2: Set up violation***

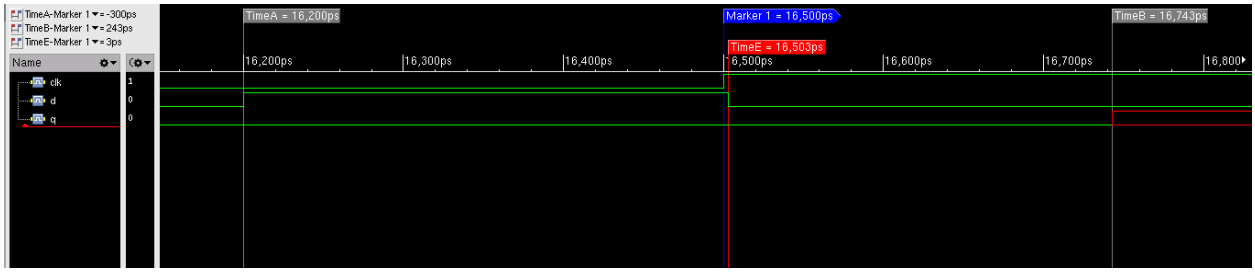


Hình 3.5: Case 2 Set up violation

D thay đổi từ mức cao xuống mức thấp trước 55ps (<t_{setup}↓=58ps) và từ mức thấp lên mức cao trước 174ps (<t_{setup}↑=177ps)

=> Không xác định được ngõ ra Q

- ***Case 3: Hold violation***

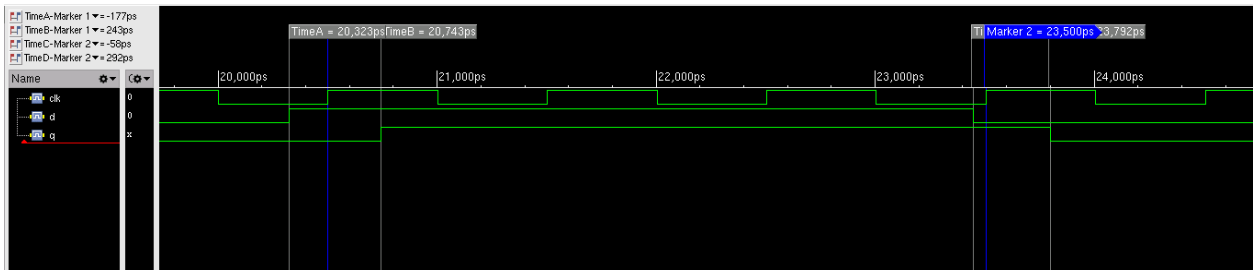


Hình 3.6: Case 3 Hold violation

D thay đổi từ mức cao xuống mức thấp sau 3ps so với clock (<t_{hold}↓=51ps)

=> Tương tự setup violation, không xác định được giá trị Q

- ***Case 4: Boundary condition***



Hình 3.7: Case 4 Boundary condition

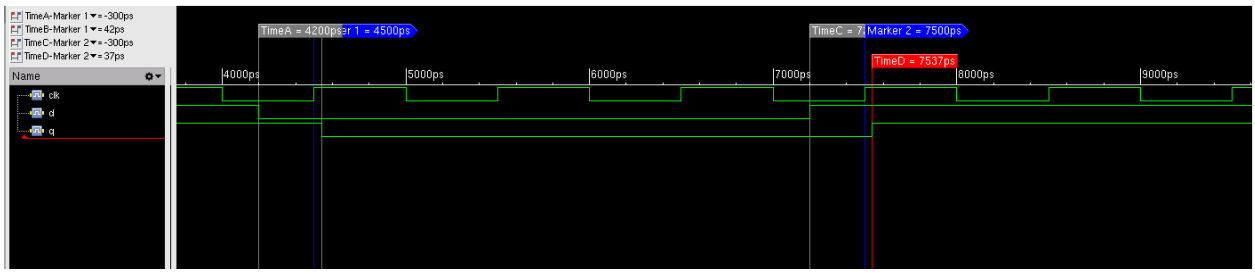
D thay đổi trước clock cạnh lên ngay khoảng thời gian bằng t_{setup}↓=58ps và t_{setup}↑=177ps

=> Kết quả thu được giống với case 1

Với các corner còn lại, nhóm sẽ thực hiện tương tự 2 corner điển hình trên và quan sát các testcase trên cùng một waveform cho dưới đây:

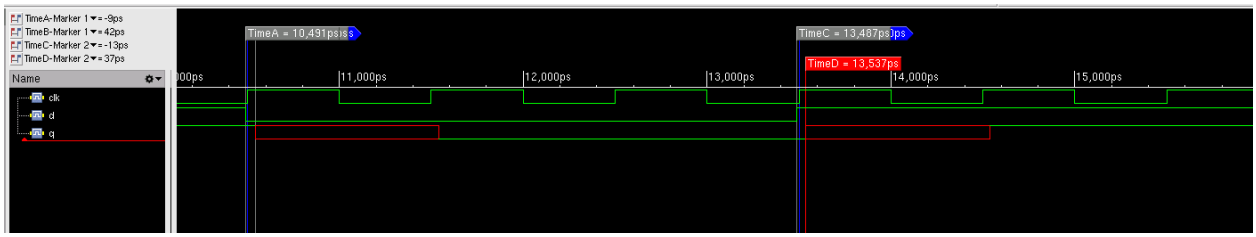
c. fast_vdd1v2_basicCells_hvt.lib

- ***Case 1: Normal operation***



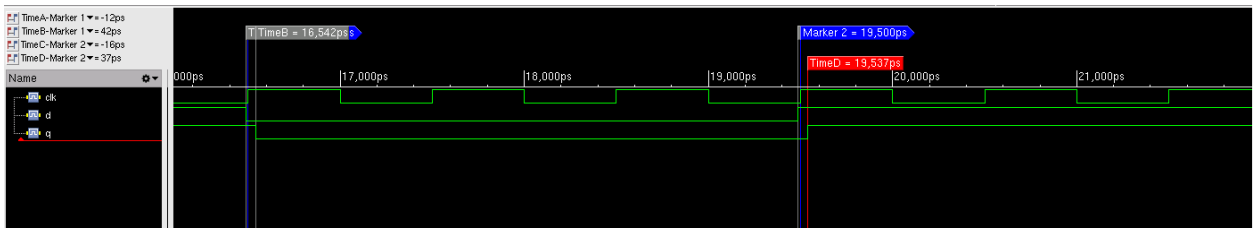
Hình 3.8: Case 1 Normal operation

- ***Case 2: Set up violation***



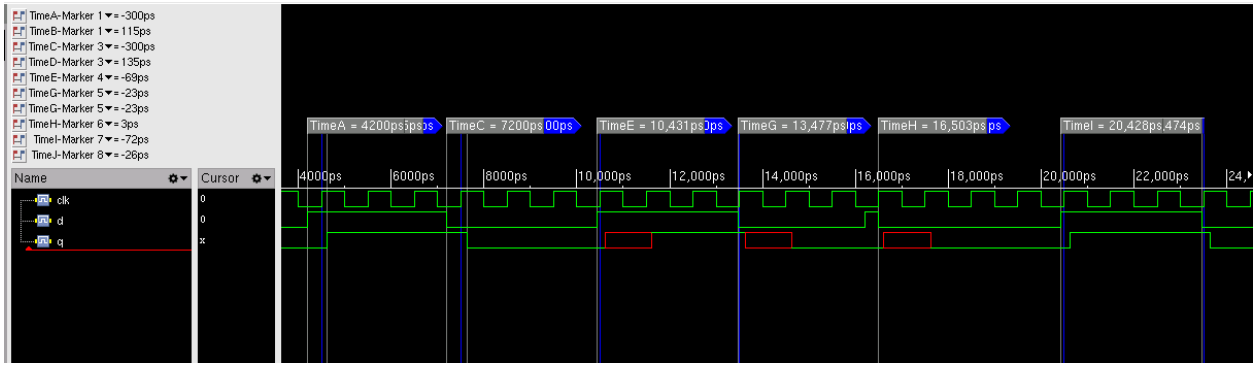
Hình 3.9: Case 2 Set up violation

- ***Case 4: Boundary condition***



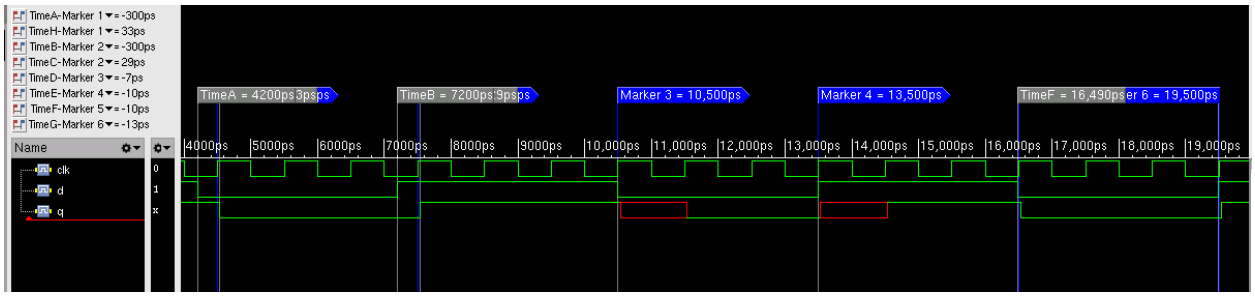
Hình 3.10: Case 4 Boundary condition

d. slow_vdd1v2_basicCells_hvt.lib



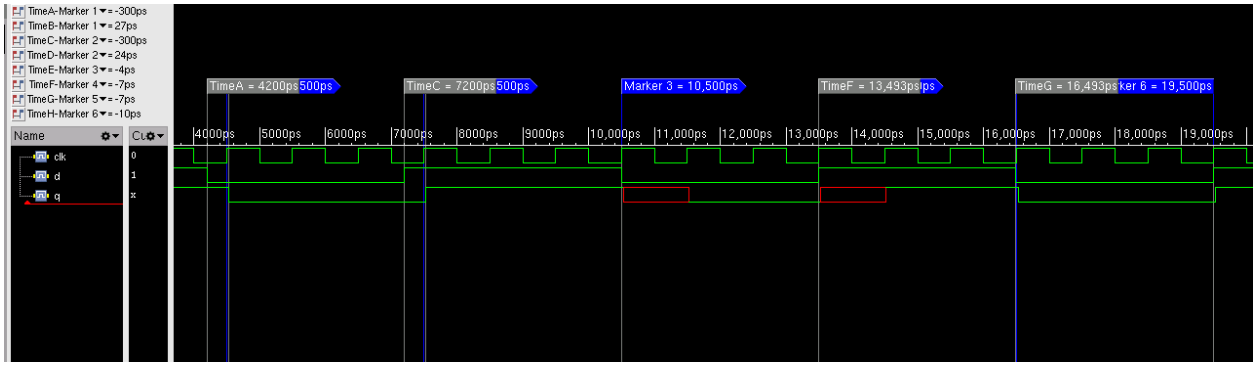
Hình 3.11: slow_vdd1v2_basicCells_hvt.lib

e. fast_vdd1v0_basicCells_lvt.lib



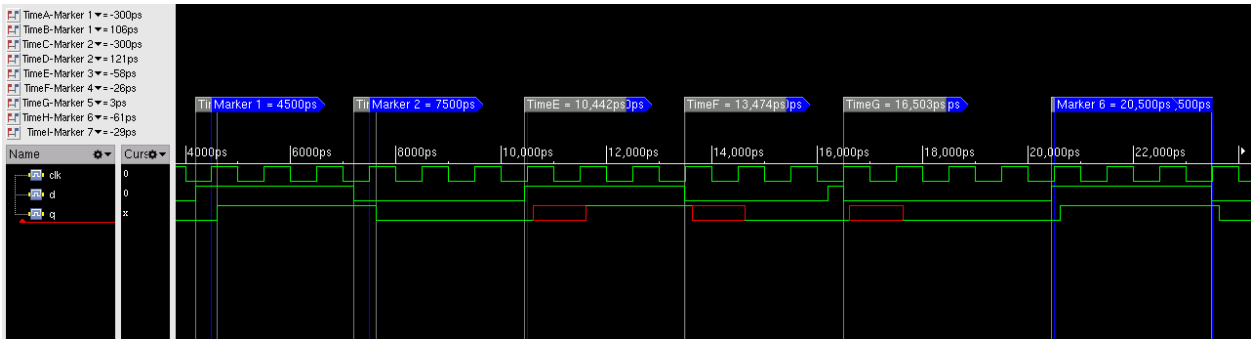
Hình 3.12: fast_vdd1v0_basicCells_lvt.lib

f. *fast_vdd1v2_basicCells_lvt.lib*



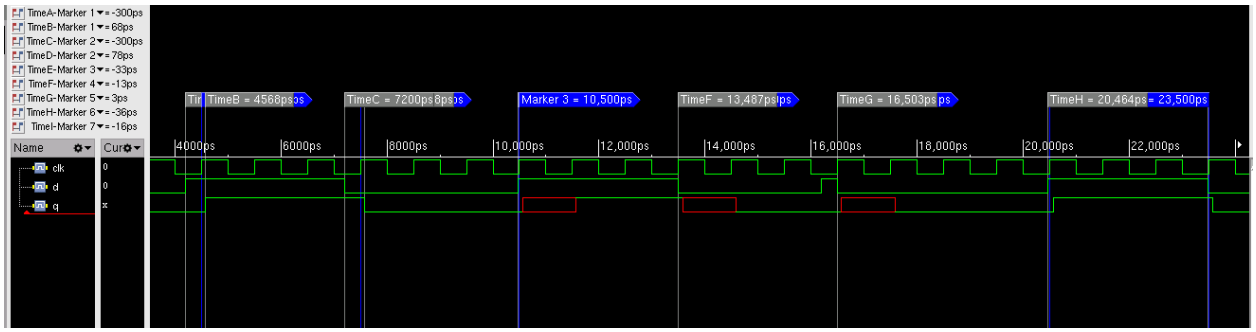
Hình 3.13: fast_vdd1v2_basicCells_lvt.lib

g. *slow_vdd1v0_basicCells_lvt.lib*



Hình 3.14: slow_vdd1v0_basicCells_lvt.lib

h. *slow_vdd1v2_basicCells_lvt.lib*



Hình 3.15: slow_vdd1v2_basicCells_lvt.lib

4. THÍ NGHIỆM 4

Objective: Perform gate-level simulation on the netlist after synthesis.

Description: In Experiment 1 and 3, we analyzed combinational logic cells and a DFF cell. In this experiment, we will examine a simple squential circuit

Đoạn code của bài thí nghiệm này:

```
module counter (
    input logic clk,
    input logic rst,
    output logic [1:0] Q
);

    always_ff @ (posedge clk) begin
        if (rst)
            Q <= 2'b00;
        else
            Q <= Q + 2'b01;
    end

endmodule
```

Thư viện sử dụng: *slow_vdd1v2_basicCells_hvt.lib*

4.1. Chuẩn bị các files cần thiết để chạy trên server:

4.1.1. TCL file (synth.tcl)

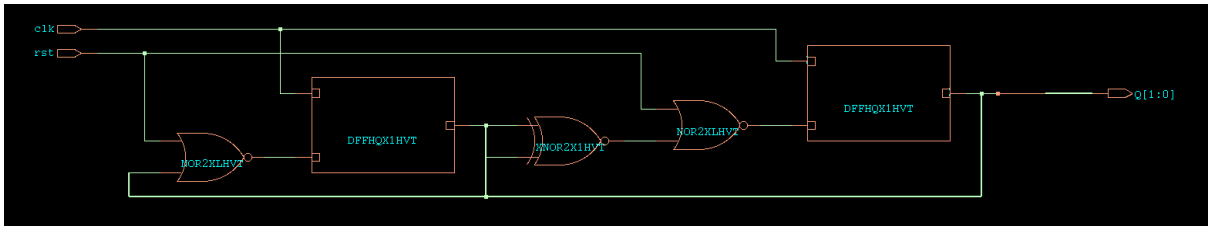
```
set_db init_lib_search_path {/home/yellow/ee3201_16/LAB2/gpdk045_lib}

set_db init_hdl_search_path {/home/yellow/ee3201_16/LAB2/Exp4/00_src}
read_libs slow_vdd1v2_basicCells_hvt.lib
read_hdl -sv counter.sv
elaborate
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
# syn_opt
report_timing > 02_reports/timing.rpt
report_power  > 02_reports/power.rpt
report_area   > 02_reports/area.rpt
report_qor    > 02_reports/qor.rpt
write_hdl     > 03_outputs/counter_netlist.sv
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -
setuphold split > 03_outputs/delay.sdf
```

4.1.2. Testbench file (counter_tb.sv)

```
``timescale 1ns/1ps

`ifdef GATE_LEVEL
    `include "./../03_outputs/counter_netlist.sv"
    `include "./../slow_vdd1v2_basicCells_hvt.v"
`else
    `include "./../00_src/counter.sv"
`endif

module counter_tb;
    parameter T = 50;
    logic clk;
    logic rst;
    logic [1:0] Q;

    counter DUT (
        .clk(clk),
        .rst(rst),
        .Q(Q)
    );

`ifdef ANNOTATION
    initial begin
        $sdf_annotate("./../Exp4/03_outputs/delay.sdf", DUT, ,
"annotate.log", "MAXIMUM");
    end
`endif

    initial begin
        clk = 0;
        forever #(T/2) clk = ~clk;
    end

    initial begin
        $dumpfile("counter_wave_T50.vcd");
        $dumpvars(0, counter_tb);

        rst = 1;
        #(2*T);
        rst = 0;
        #(5*T);
        $finish;
    end
endmodule
```

4.2. Kết quả sau khi chạy tool Genus trên server

4.2.1. *Kết quả synthesis*



Hình 4.1: Kết quả synthesis bộ Counter

Schematic bao gồm 2 khối D-FlipFlop quy định 2 bit của ngõ ra Q (Q[1], Q[0]), 2 cổng NOR và 1 cổng XNOR, đóng vai trò trong việc bắt reset và thực hiện phép cộng 1, tín hiệu từ ngõ ra được đưa trở lại ngõ vào để thực hiện cộng liên tục.

4.2.2. *Đọc các file outputs*

a) Netlist file (counter_netlist.sv)

File netlist chứa toàn bộ các cells trong schematic, và cách kết nối giữa chúng thông qua dây nối (wires)

```
// Generated by Cadence Genus(TM) Synthesis Solution 21.19-s055_1
// Generated on: Nov 20 2025 17:43:28 +07 (Nov 20 2025 10:43:28 UTC)
// Verification Directory fv/counter

module counter(clk, rst, Q);
  input clk, rst;
  output [1:0] Q;
  wire clk, rst;
  wire [1:0] Q;
  wire n_0, n_1, n_2;
  DFFHQX1HVT \Q_reg[1] (.CK (clk), .D (n_2), .Q (Q[1]));
  NOR2XLHVT g20__2398(.A (rst), .B (n_1), .Y (n_2));
  DFFHQX1HVT \Q_reg[0] (.CK (clk), .D (n_0), .Q (Q[0]));
  XNOR2X1HVT g22__5107(.A (Q[0]), .B (Q[1]), .Y (n_1));
  NOR2XLHVT g23__6260(.A (rst), .B (Q[0]), .Y (n_0));
endmodule
```

b) SDF file (delay.sdf)

```
(DELAYFILE
  (SDFVERSION  "OVI 3.0")
  (DESIGN      "counter")
  (DATE        "Thu Nov 20 17:43:28 +07 2025")
  (VENDOR      "Cadence, Inc.")
  (PROGRAM     "Genus(TM) Synthesis Solution")
  (VERSION     "21.19-s055_1")
  (DIVIDER     .)
```

```
(VOLTAGE     ::1.08)
(PROCESS     "::1.0")
(TEMPERATURE ::125.0)
(TIMESCALE   1ns)
(CELL
   (CELLTYPE "DFFHQX1HVT")
   (INSTANCE Q_reg\[1\])
   (DELAY
      (ABSOLUTE
        (PORT CK (::0.000))
        (PORT D (::0.000))
        (IOPATH CK Q (::0.119) (::0.140))
      )
   )
   (TIMINGCHECK
      (HOLD (negedge D) (posedge CK) (::0.009))
      (HOLD (posedge D) (posedge CK) (::0.000))
      (SETUP (negedge D) (posedge CK) (::0.037))
      (SETUP (posedge D) (posedge CK) (::0.087))
   )
)
(CELL
   (CELLTYPE "NOR2XLHVT")
   (INSTANCE g20__2398)
   (DELAY
      (ABSOLUTE
        (PORT A (::0.000))
        (PORT B (::0.000))
        (IOPATH A Y (::0.028) (::0.022))
        (IOPATH B Y (::0.030) (::0.025))
      )
   )
)
(CELL
   (CELLTYPE "DFFHQX1HVT")
   (INSTANCE Q_reg\[0\])
   (DELAY
      (ABSOLUTE
        (PORT CK (::0.000))
        (PORT D (::0.000))
        (IOPATH CK Q (::0.119) (::0.140))
      )
   )
   (TIMINGCHECK
      (HOLD (negedge D) (posedge CK) (::0.009))
      (HOLD (posedge D) (posedge CK) (::0.000))
      (SETUP (negedge D) (posedge CK) (::0.037))
```

```
        (SETUP (posedge D) (posedge CK) (::0.087))
      )
  )
  (CELL
      (CELLTYPE "XNOR2X1HVT")
      (INSTANCE g22__5107)
      (DELAY
        (ABSOLUTE
          (PORT A (::0.000))
          (PORT B (::0.000))
          (IOPATH A Y (::0.131) (::0.110))
          (IOPATH B Y (::0.109) (::0.077))
        )
      )
  )
  (CELL
      (CELLTYPE "NOR2XLHVT")
      (INSTANCE g23__6260)
      (DELAY
        (ABSOLUTE
          (PORT A (::0.000))
          (PORT B (::0.000))
          (IOPATH A Y (::0.028) (::0.022))
          (IOPATH B Y (::0.031) (::0.024))
        )
      )
  )
)
```

File này chứa các thông số liên quan đến hiện tượng trễ tín hiệu (delay) của từng standard cell theo thư viện đã cho. Cụ thể:

(TIMESCALE 1ns): Đơn vị thời gian được sử dụng trong file này là nano giây

(CELLTYPE "NOR2XLHVT"): Tên của cell trong thư viện standard cells (cổng NOR 2 ngõ vào, loại High-Vt).

(INSTANCE g23__6260): Tên instance của cell trong mạch — được gán bởi công cụ synthesis.

ABSOLUTE: Các thông số tuyệt đối về độ trễ, là kết quả đo được của cell ở một số điều kiện

(IOPATH A Y (::0.028) (::0.022)):

- ***IOPATH A Y:*** Đường đi của tín hiệu từ ngõ vào A đến ngõ ra Y.
- ***(::0.028) (::0.022):***

– Giá trị đầu tiên (0.028 ns) độ trễ của cạnh lên (0 → 1).

– Giá trị thứ hai (0.022 ns) độ trễ của cạnh xuống (1 → 0).

Tương tự, **IOPATH B Y** là đường đi của tín hiệu từ ngõ vào B đến ngõ ra Y

4.3. Kiểm thử và kết luận

4.3.1. Testcases

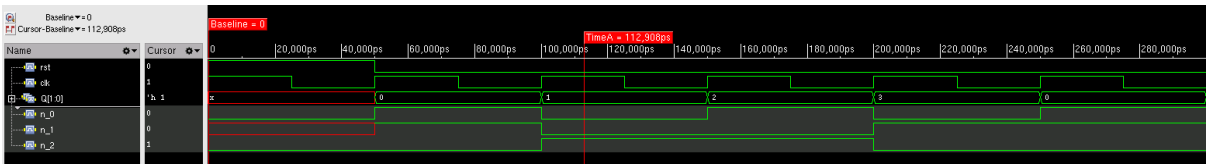
Bài kiểm thử bao gồm 3 giá trị chu kỳ khác nhau T = 50 ns, 100 ns và 200 ns. Mỗi trường hợp chu kỳ sẽ phân tích dạng sóng (mỗi dạng sóng thực hiện phân tích 5 chu kỳ) và giải thích nguyên nhân gây trễ tại ngõ ra

Bảng 4.1: Các trường hợp kiểm thử của khối Counter

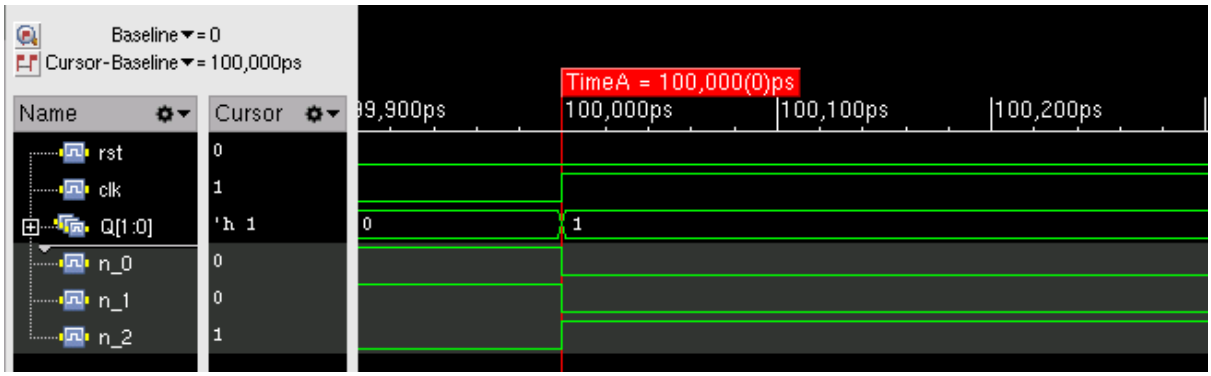
Chu kỳ	**Thời gian theo chu kỳ (T)**			**Q mong muốn**
	T = 50 ns	**T = 100 ns**	**T = 200 ns**	
0 (Để bật/tắt reset)	0 → 50	0 → 100	0 → 200	xx
1	50 → 100	100 → 200	200 → 400	0
2	100 → 150	200 → 300	400 → 600	1
3	150 → 200	300 → 400	600 → 800	2
4	200 → 250	400 → 500	800 → 1000	3
5	250 → 300	500 → 600	1000 → 1200	0

4.3.2. Chạy trên Xcelium và phân tích dạng sóng trên Simvision

a) Trường hợp không có delay (với T = 50 ns)



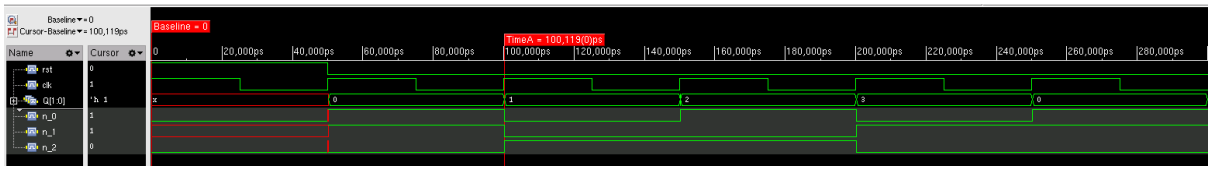
Hình 4.2: Dạng sóng lý tưởng (không có delay)



Hình 4.3: Dạng sóng phóng to tại thời điểm 100 ns

Ở dạng sóng lý tưởng, ngõ ra Q và các đường dây cục bộ trong mạch thay đổi đồng thời với cạnh lên của xung clock.

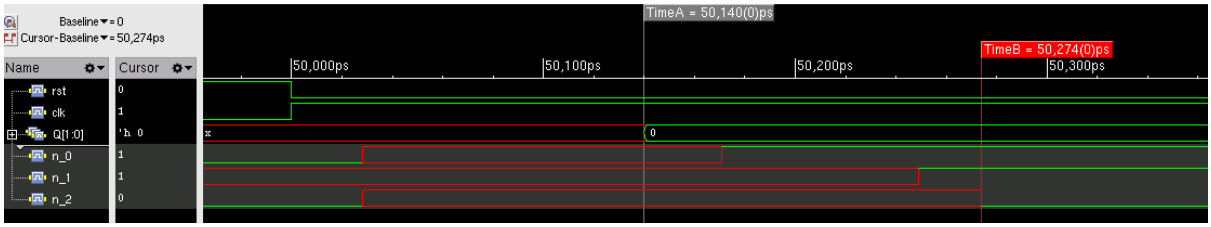
b) Trường hợp T = 50 ns



Hình 4.4: Dạng sóng của toàn bộ bài kiểm thử với T = 50 ns

Từ dạng sóng, ta dễ thấy ngõ ra Q thay đổi và ổn định muộn hơn các cạnh lên xung clock tại các mốc thời gian đang xét. Ngõ ra còn xuất hiện giá trị khác với giá trị mong muốn, dù tồn tại trong một khoảng thời gian rất nhỏ. Sau đây, ta sẽ phân tích cụ thể từng trường hợp để hiểu rõ tại sao lại có hiện tượng này.

- **Chu kỳ 1 $(50\ ns \rightarrow 100\ ns)$ (Ngay sau reset)**

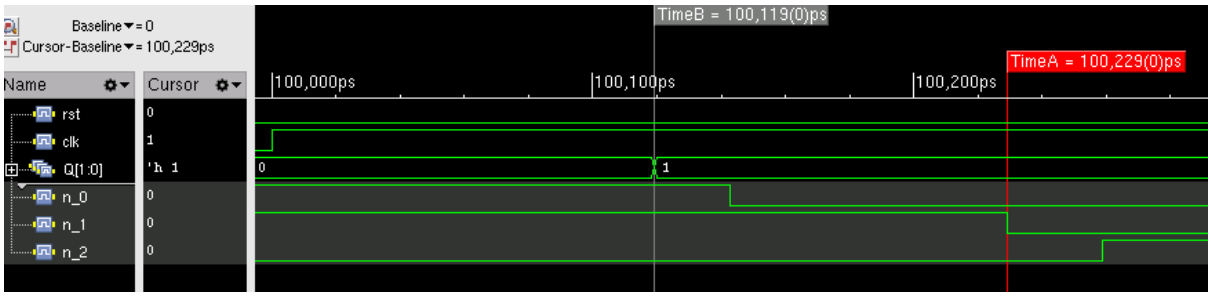


Hình 4.5: Trạng thái các ngõ vào/ra và các đường dây tại giai đoạn khởi tạo

Tại 50 ns, cạnh lên xung clock diễn ra đồng thời với tắt reset, nếu không áp dụng delay, ngõ ra Q sẽ mang giá trị 0 ngay lập tức. Trên thực tế, ngõ ra Q được xác định sau một khoảng thời gian chính xác là $\Delta t = 140\ ps$ kể từ khi có cạnh lên và tắt reset.

Khoảng thời gian trễ 140 ps này là độ trễ của D-FF khi có cạnh xuống, như đã liệt kê trong file SDF, kết quả này đúng với giá trị của Q sau khi ổn định là 00.

- **Chu kỳ 2 $(100\ ns \rightarrow 150\ ns)$**

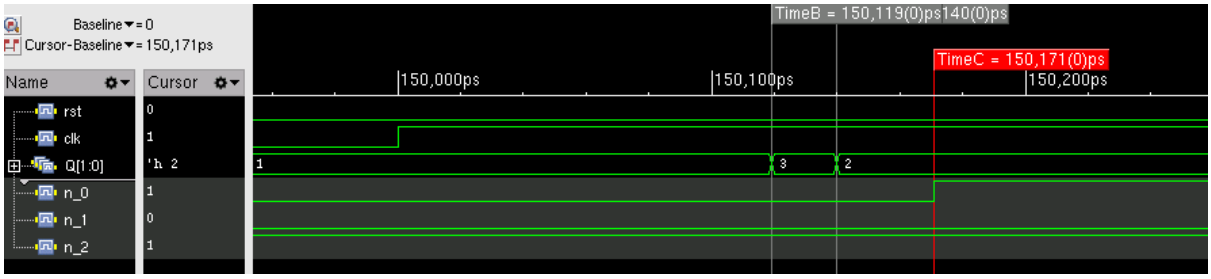


Hình 4.6: Trạng thái các biến tại đầu chu kỳ 2, t = 100 ns

Tại 100 ns có cạnh lên của xung clock. Ngõ ra Q theo lý thuyết sẽ bằng với kết quả ở chu kỳ trước cộng thêm 1, tức chuyển sang 1 ngay tại thời điểm có cạnh lên. Trong thực tế, ngõ ra Q chỉ chuyển sang 1 và ổn định sau mốc thời gian 100,119 ps, tức là sau một khoảng $\Delta t = 119\ ps$. Đây chính là giá trị delay của D-FF khi ngõ ra Q có thay đổi cạnh lên.

Tại 100 ns, Q theo lý thuyết sẽ chuyển từ 0 lên 1, tức là Q[0] sẽ có cạnh lên, Q[1] vẫn giữ nguyên giá trị. Vậy tổng thời gian trễ của ngõ ra Q sẽ là thời gian trễ của ngõ ra Q[0] khi có cạnh lên, tức là 119 ps.

- **Chu kỳ 3 ($150\ ns \rightarrow 200\ ns$)**



Hình 4.7: Trạng thái các biến tại đầu chu kỳ 3, t = 150 ns

Tại t = 150 ns = 150,000 ps có cạnh lên của xung clock. Ngõ ra Q mong muốn là 2 (10). Trên thực tế, ngõ ra Q thay đổi như sau:

- Từ 150,000 ps đến 150,119 ps: Ngõ ra Q vẫn giữ giá trị từ chu kỳ trước (01)
- Từ 150,119 ps đến 150,140 ps: Q chuyển sang giá trị 3 (11)
- Sau 150,140 ps: Q chuyển sang 2 (10), là giá trị ta mong muốn đạt được

Hiện tượng ngõ ra Q xuất hiện giá trị 3 trước khi chuyển sang giá trị mong muốn là do sự chênh lệch giữa thời gian trễ của việc thay đổi cạnh lên/xuống của D Flip-Flop. Cụ thể, khi Q chuyển từ 1 (01) sang 2 (10), sự thay đổi các bit trong ngõ ra Q diễn ra như sau:

- Q[1] chuyển từ 0 sang 1 (cạnh lên, thời gian trễ 119 ps)
- Q[0] chuyển từ 1 sang 0 (cạnh xuống, thời gian trễ 140 ps)

Vì Q[1] có thời gian trễ thay đổi sớm hơn Q[0], nên trong khoảng thời gian từ 150,119 ps đến 150,140 ps bit Q[1] thay đổi giá trị trước, đến sau mốc 150,140 ps thì Q[0] mới được xác định. Đó là lý do trên dạng sóng xuất hiện giá trị trung gian 11

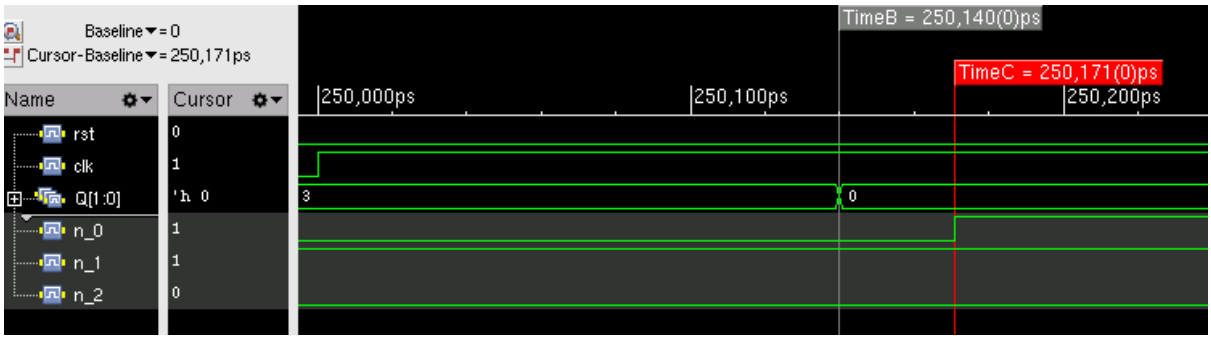
- **Chu kỳ 4 ($200\ ns \rightarrow 250\ ns$)**

Hình 4.8: Trạng thái các biến tại đầu chu kỳ 4, t = 200 ns

Tại t = 200 ns = 200,000 ps có cạnh lên xung clock. Ngõ ra Q mong muốn lúc này là 3 (11). Trên thực tế, Q chỉ chuyển sang giá trị mong muốn sau mốc 200,119 ps, tức là Q đã bị trễ một khoảng thời gian $\Delta t = 119\ ps$.

Cụ thể, khi Q chuyển từ 2 (10) sang 3 (11), chỉ có bit Q[0] thay đổi cạnh lên, vì vậy thời gian trễ của Q lúc này đúng bằng thời gian Q[0] thực hiện cạnh lên.

- **Chu kỳ 5 ($250\ ns \rightarrow 300\ ns$)**

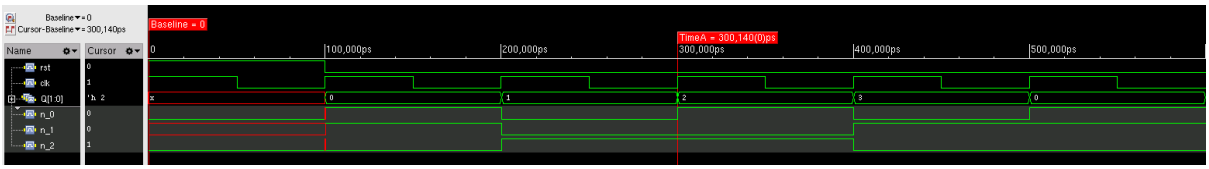


Hình 4.9: Trạng thái các biến tại đầu chu kỳ 5, t = 250 ns

Tại t = 250 ns = 250,000 ps có cạnh lên xung clock. Ngõ ra Q mong muốn lúc này là 4 (100), nhưng Q chỉ có phạm vi biểu diễn là 2 bit, nên Q sẽ trở về lại giá trị 0. Trên thực tế, Q chỉ chuyển sang giá trị mong muốn sau mốc 250,140 ps, tức là Q đã bị trễ một khoảng thời gian $\Delta t = 140\ ps$.

Cụ thể, khi Q chuyển từ 3 (11) sang 0 (00), cả hai bit Q[0] và Q[1] đều thay đổi cạnh xuống, vì vậy thời gian trễ của Q lúc này đúng bằng thời gian Q[0] và Q[1] thực hiện cạnh xuống.

c) Trường hợp T = 100 ns

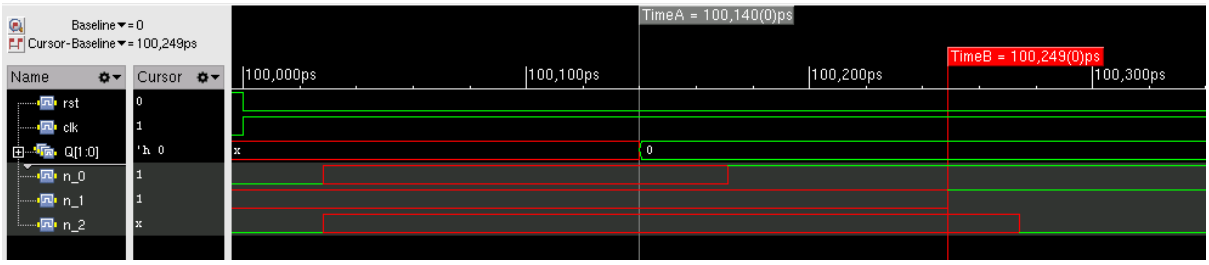


Hình 4.10: Dạng sóng của toàn bộ bài kiểm thử với T = 100 ns

Khi tăng T đến 100 ns, dạng sóng đã gần đạt đến dạng lý thuyết (không có delay), nhưng các giá trị trung gian vẫn còn hiện hữu nếu nhìn kỹ

Khi phân tích sâu vào từng trường hợp, ta thấy ngõ ra vẫn đạt giá trị mong muốn trễ hơn so với mốc thời gian thay đổi các ngõ vào

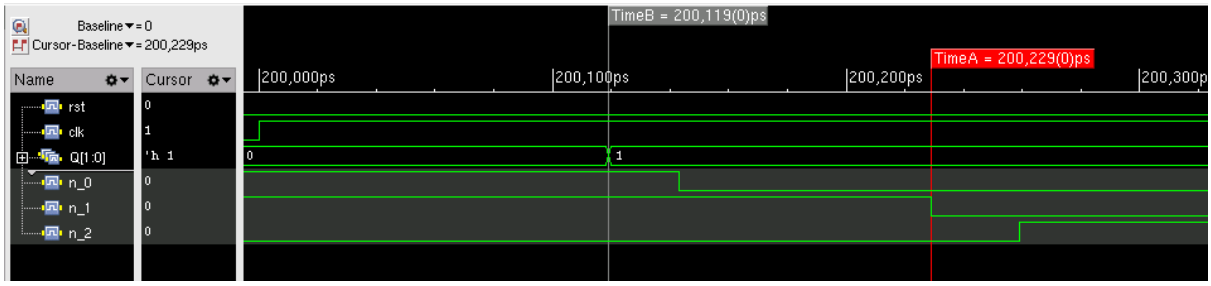
- **Chu kỳ 1 ($100\ ns \rightarrow 200\ ns$) – Ngay sau khi Reset**



Hình 4.11: Trạng thái các biến tại đầu chu kỳ 1, t = 100 ns

Chu kỳ 1 với T = 100 ns vẫn giống như chu kỳ 1 với T = 50 ns, ngõ ra Q vẫn đạt giá trị mong muốn (0) sau $\Delta t = 140\ ps$ kể từ khi có cạnh lên xung clock

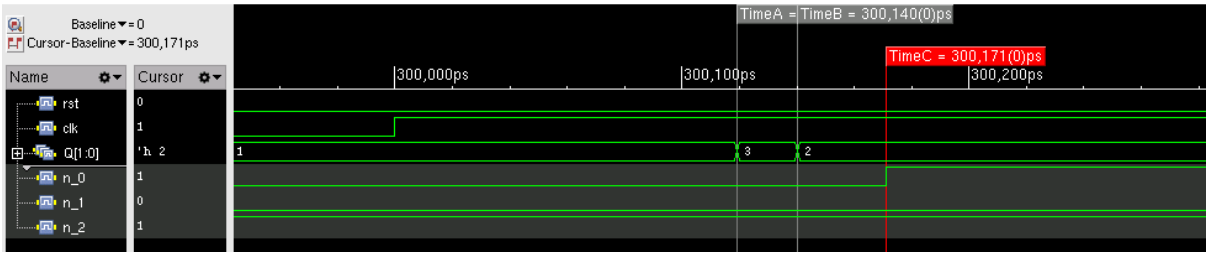
- **Chu kỳ 2 ($200\ ns \rightarrow 300\ ns$)**



Hình 4.12: Trạng thái các biến tại đầu chu kỳ 2, t = 200 ns

Tương tự như trường hợp t = 100 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 119 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

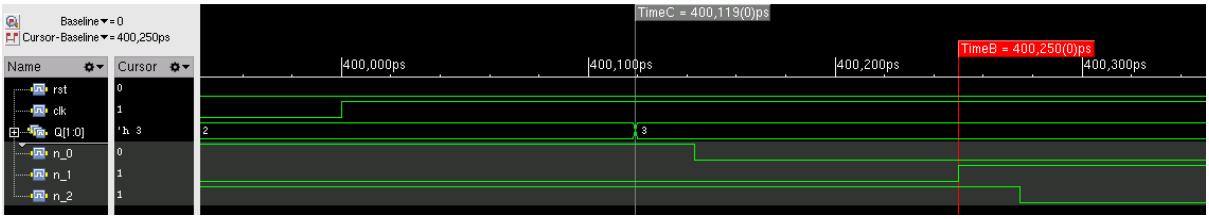
- **Chu kỳ 3 ($300\ ns \rightarrow 400\ ns$)**



Hình 4.13: Trạng thái các biến tại đầu chu kỳ 3, t = 300 ns

Tương tự như trường hợp t = 150 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 140 ps, và có giá trị trung gian trong khoảng từ 300,119 ps đến 300,140 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

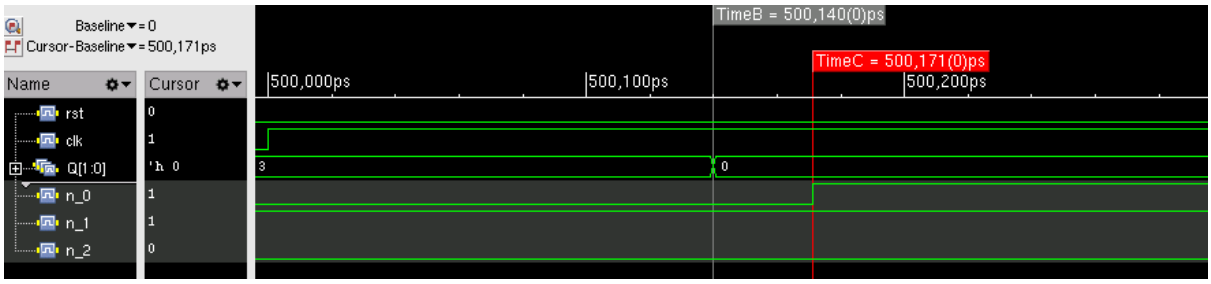
- **Chu kỳ 4 ($400\ ns \rightarrow 500\ ns$)**



Hình 4.14: Trạng thái các biến tại đầu chu kỳ 4, t = 400 ns

Tương tự như trường hợp t = 200 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 119 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

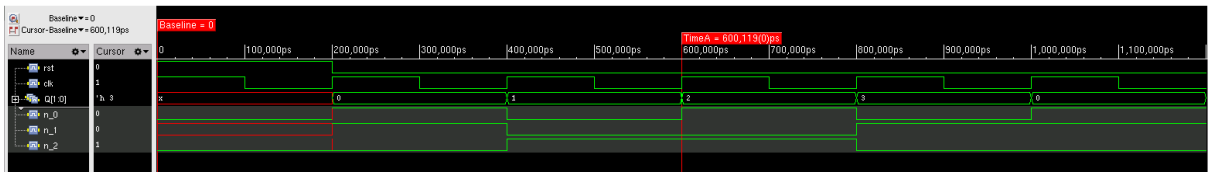
- **Chu kỳ 5 ($500\ ns \rightarrow 600\ ns$)**



Hình 4.15: Trạng thái các biến tại đầu chu kỳ 5, t = 500 ns

Tương tự như trường hợp t = 250 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 140 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

d) Trường hợp T = 200 ns

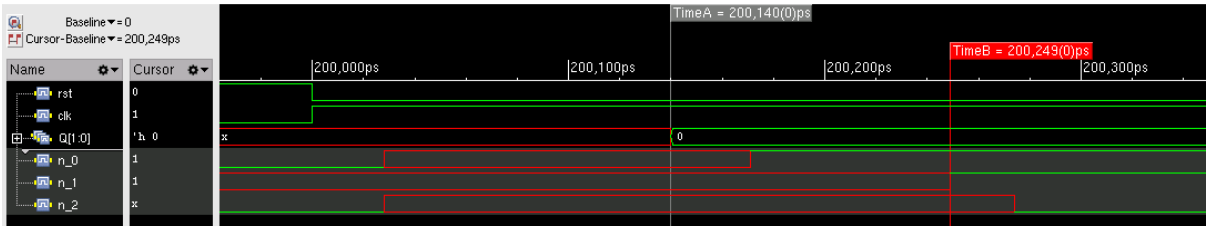


Hình 4.16: Dạng sóng của toàn bộ bài kiểm thử với T = 100 ns

Khi tăng T đến 200 ns, dạng sóng hầu như đạt đến dạng lý thuyết (không có delay), giá trị trung gian giờ chỉ còn được thể hiện bằng một chấm nhỏ thể hiện rằng có giá trị trung gian tại thời điểm đó

Khi phân tích sâu vào từng trường hợp, ta thấy ngõ ra vẫn đạt giá trị mong muốn trễ hơn so với mốc thời gian thay đổi các ngõ vào

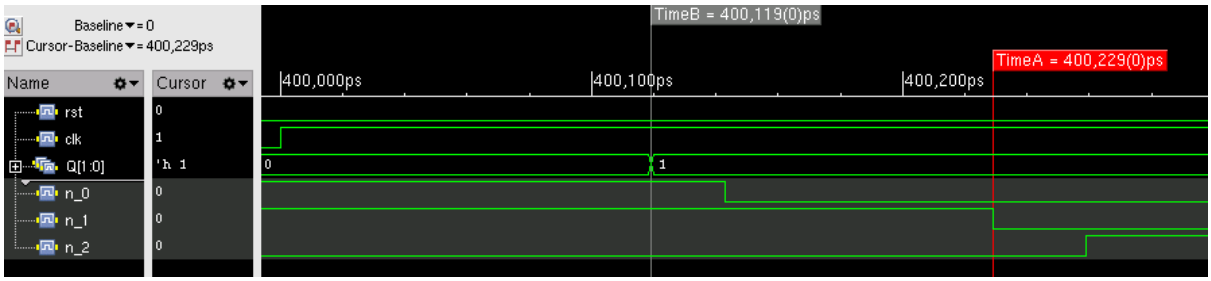
- **Chu kỳ 1 ($200\ ns \rightarrow 400\ ns$) – Ngay sau khi Reset**



Hình 4.17: Trạng thái các biến tại đầu chu kỳ 1, t = 200 ns

Chu kỳ 1 với T = 200 ns vẫn giống như chu kỳ 1 với T = 50 ns, ngõ ra Q vẫn đạt giá trị mong muốn (0) sau $\Delta t = 140\ ps$ kể từ khi có cạnh lên xung clock

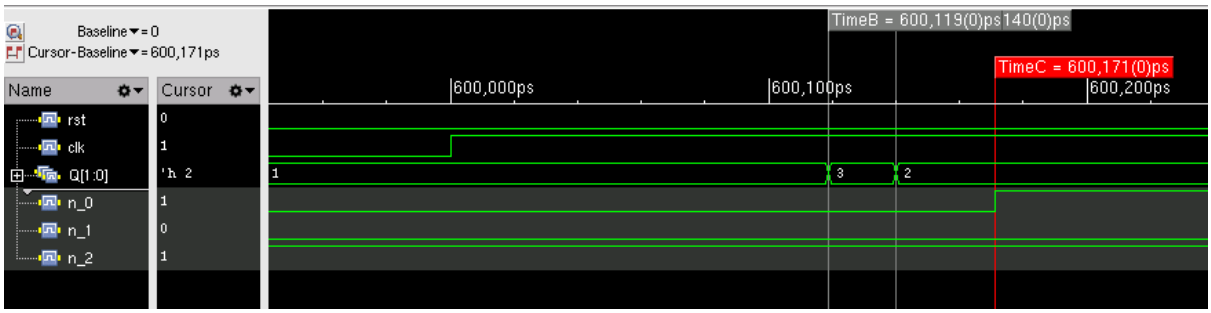
- **Chu kỳ 2 ($400\ ns \rightarrow 600\ ns$)**



Hình 4.18: Trạng thái các biến tại đầu chu kỳ 2, t = 400 ns

Tương tự như trường hợp t = 100 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 119 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

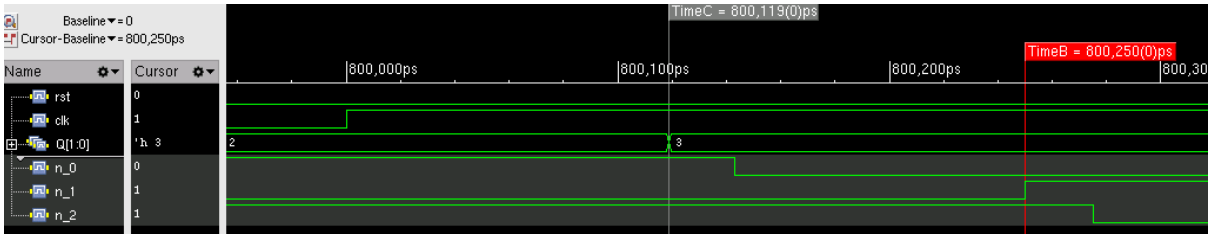
- **Chu kỳ 3 ($600\ ns \rightarrow 800\ ns$)**



Hình 4.19: Trạng thái các biến tại đầu chu kỳ 3, t = 600 ns

Tương tự như trường hợp t = 150 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 140 ps, và có giá trị trung gian trong khoảng từ 600,119 ps đến 600,140 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

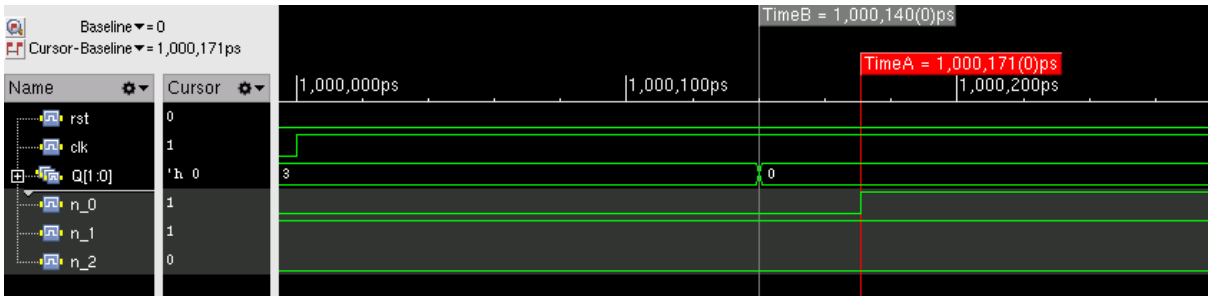
- **Chu kỳ 4 $(800\ ns \rightarrow 1000\ ns)$**



Hình 4.20: Trạng thái các biến tại đầu chu kỳ 4, t = 800 ns

Tương tự như trường hợp t = 200 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 119 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

- **Chu kỳ 5 $(1000\ ns \rightarrow 1200\ ns)$**



Hình 4.21: Trạng thái các biến tại đầu chu kỳ 5, t = 1000 ns

Tương tự như trường hợp t = 250 ns khi T = 50 ns, ngõ ra Q ổn định sau mốc thời gian 140 ps. Việc phân tích và giải thích đã được trình bày ở trường hợp nêu trên

4.3.3. Tổng kết

a) Tổng hợp kết quả của bài kiểm thử:

Bảng 4.2: Tổng hợp các kết quả kiểm thử của khối Counter

Chu kỳ	Q mong muốn	Thời gian trễ (delay)
0 (Để bật/tắt reset)	xx	x
1	0	140 ps
2	1	119 ps
3	2	140 ps
4	3	119 ps
5	0	140 ps

b) Độ trễ lan truyền

Trong mạch tuần tự sử dụng D-FlipFlop, độ trễ quan sát được ở ngõ ra là T_{cq}. Đây là khoảng thời gian cần thiết để tín hiệu truyền từ ngõ vào clock, đi qua các cổng logic bên trong Flip-Flop để cập nhật trạng thái mới tại ngõ ra Q.

Dựa trên file SDF và các kết quả trên dạng sóng:

- Độ trễ cạnh lên $(0 \rightarrow 1)$của Flip-Flop là 119 ps.
- Độ trễ cạnh xuống $(1 \rightarrow 0)$của Flip-Flop là 140 ps.

Thời gian để ngõ ra ổn định phụ thuộc vào sự chuyển đổi của các bit. Nếu việc chuyển đổi có cạnh xuống, hệ thống sẽ chịu độ trễ lớn nhất là 140 ps.

c) Hiện tượng xuất hiện giá trị trung gian

Trong quá trình ngõ ra chuyển trạng thái từ 01 (1) sang 10 (2), mạch xuất hiện giá trị trung gian 11 (3) trong khoảng thời gian ngắn $(\Delta t = 21\ ps)$. Hiện tượng này được giải thích như sau:

Để ngõ ra Q chuyển từ 01 sang 10: Bit Q[0] cần chuyển từ $1 \rightarrow 0$ (cạnh xuống) và bit Q[1] cần chuyển từ $0 \rightarrow 1$ (Cạnh lên).

Theo file SDF, độ trễ cạnh lên (119 ps) nhỏ hơn trễ cạnh xuống (140 ps).

Do đó, bit Q[1] lên mức cao trước, trong khi bit Q[0] vẫn chưa kịp xuống mức thấp. Tại khoảng thời gian giữa 119 ps và 140 ps, cả hai bit đều ở mức cao (11), tạo ra giá trị sai tạm thời trước khi mạch đi vào ổn định.

d) Kết luận về tham số T

Kết quả mô phỏng với 3 giá trị T (50 ns, 100 ns, 200 ns) cho thấy thời gian trễ (t_{delay}) là một thông số vật lý cố định của các cells và công nghệ chế tạo (trong Lab này là 45nm), không phụ thuộc vào chu kỳ T.

Để mạch hoạt động đúng (không bị vi phạm Setup/Hold time), chu kỳ T phải luôn thỏa mãn điều kiện: $T > T_{cq} + T_{logic} + T_{setup}$.

Ở bài thí nghiệm này, do T (hàng chục ns) lớn hơn rất nhiều so với tổng delay (hàng trăm ps), ta luôn đảm bảo tín hiệu tại ngõ vào D luôn ổn định từ rất lâu trước khi cạnh lên clock tiếp theo xuất hiện, do đó loại bỏ khả năng vi phạm Setup Time Violation.

Nếu ta giảm chu kỳ T xuống tiệm cận với tổng trễ, ví dụ $T \approx 200\,ps$, khoảng thời gian dự trữ sẽ bị cắt ngắn, hoặc không còn. Khi đó, nếu dữ liệu tại ngõ vào D chưa kịp xác lập giá trị đúng mà cạnh lên clock đã tới, Flip-Flop sẽ bắt sai dữ liệu, dẫn đến ngõ ra Q bị sai lệch.

Các giá trị T lớn trong bài thí nghiệm này giúp đảm bảo mạch hoạt động đúng mà không cần quan tâm đến các vấn đề tối ưu hóa thời gian khi thiết kế có tần số cao.